



Manuela Fischer, Felix Friedrich

Informatik

Vorlesung vom D-INFK der ETH Zürich

Herbst 2023

Willkommen

zur Vorlesung Informatik

am MATH/PHYS Department der ETH Zürich.

Ort und Zeit:

Dienstag 14:15 - 16:00, HG F 7 und HG F 5 (Video).

Pause 15:00 - 15:15, leichte Verschiebung möglich.

Vorlesungs-Webseite

`http://lec.inf.ethz.ch/ifmp`

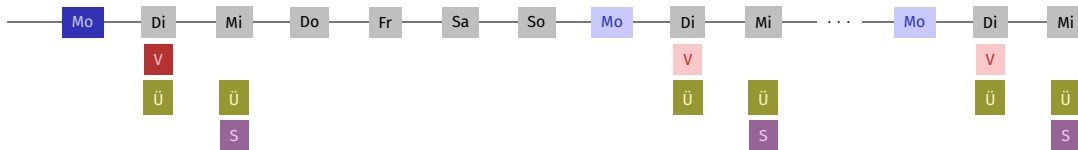
Team

Chefassistent Manuel Kansy

Assistierende	Adriana De-Sassi	Alain Flückiger
	Cajetan Tulej	Chen Guo
	Christopher Otto	Dong Kang
	Esther Petat	Fatemeh Zargarbashi
	Fiona Sacher	Flavio De Vincenti
	Hannah Vogel	Johannes Luber
	Katharina Knist	Levin Ceglie
	Maxence Aleksa	Michelle Egger
	Nick Herrmann	Renas Sahin
	Siyuan Shen	Vincent Junker

Dozierende	Dr. Manuela Fischer	Dr. Felix Friedrich
-------------------	---------------------	---------------------

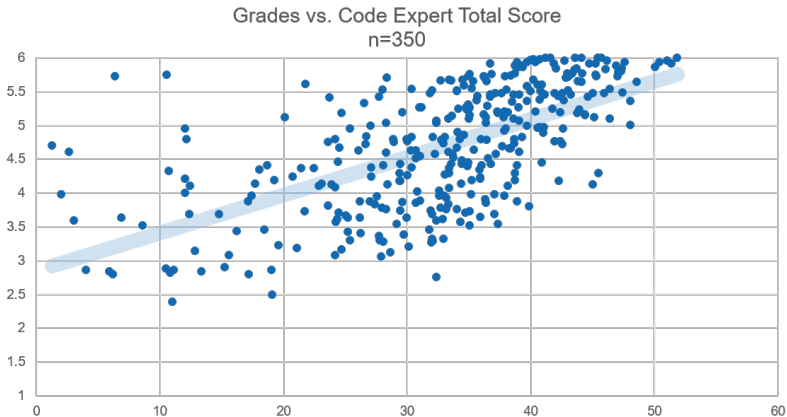
Ablauf



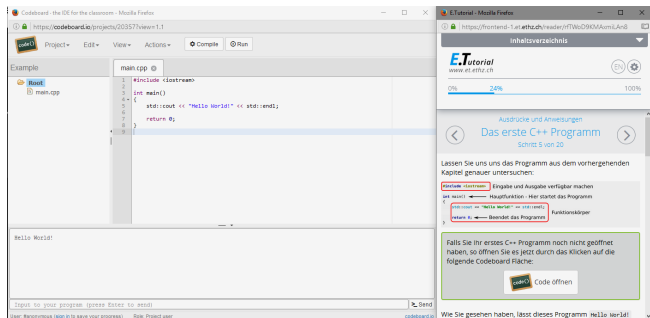
- Übungsblattausgabe am Montag morgen (online)
- Vorlesung am Dienstag
- Vorbesprechung in der folgenden Übung (am Dienstag/Mittwoch)
- StudyCenter (studycenter.ethz.ch)
- Abgabe der Serie spätestens am Montag in der nächste Woche (18:00h)
- Nachbesprechung der Serie in der übernächsten Übung. Feedback zu den Abgaben innerhalb einer Woche nach Abgabe-Termin.

Zu den Übungen

- Bearbeitung der wöchentlichen Übungsserien ist also freiwillig, wird aber dringend empfohlen!



Online Tutorial



Zum Einstieg stellen wir ein Online-C++ Tutorial zur Verfügung.
Ziel: Ausgleich der unterschiedlichen Programmierkenntnisse.
⇒ Online Tutorial bis nächste Woche durcharbeiten.

Relevantes für die Prüfung

Prüfungsstoff für die Endprüfung schliesst ein

- Vorlesungsinhalt (Vorlesung, Handout) und
- Übungsinhalte (Übungsstunden, Übungsaufgaben).

Prüfung ist schriftlich am Computer.

Es wird sowohl praktisches Wissen (Programmierfähigkeit) als auch theoretisches Wissen (Hintergründe, Systematik) geprüft. Bewertung Prüfung prozentual nach bestandenen Tests, Bonusaufgaben ebenso. Serien verlangen alle Tests um die XP zu erhalten.

(Beim üben:) Moodle zeigt nur eine Antwort als richtig an, hinter den Kulissen werden alle semantisch richtigen akzeptiert.

Unser Angebot (VVZ)

- Ihre Programmierübungen werden (halb)automatisch bewertet. Durch Bearbeitung der wöchentlichen Übungsserien kann ein Bonus von maximal 0.25 Notenpunkten erarbeitet werden, der an die Prüfung mitgenommen wird.
- Der Bonus ist proportional zur erreichten Punktzahl von speziell markierten Bonusaufgaben, wobei volle Punktzahl einem Bonus von 0.25 entspricht. Die Zulassung zu speziell markierten Bonusaufgaben hängt von der erfolgreichen Absolvierung anderer Übungsaufgaben ab. Der erreichte Notenbonus verfällt, sobald die Vorlesung neu gelesen wird.

Unser Angebot (konkret)

- Insgesamt 3 Bonusaufgaben; 2/3 der Punkte reichen für 0.25 Notenbonus für die Prüfung
- Sie können also z.B. 2 Bonusaufgaben zu 100% lösen, oder 3 Bonusaufgaben zu je 66%, oder ...
- Bonusaufgaben müssen durch erfolgreich gelöste Übungsserien freigeschaltet (→ Experience Points) werden
- Es müssen wiederum nicht alle Übungsserien vollständig gelöst werden, um eine Bonusaufgabe freizuschalten
- Details: Kurswebseite, Übungsstunden, Online-Übungssystem (Code Expert)

Akademische Lauterkeit

Wir ermuntern Sie explizit, Lösungsideen und -ansätze mit Ihren Kollegen zu diskutieren. Teamwork ist auch in der Informatik wichtig. Es ist aber insbesondere wichtig, dass Sie aktiv dazulernen und nicht nur reproduzieren. Daher:

Regeln

Sie geben nur eigene Lösungen ab, welche Sie selbst verfasst und verstanden haben. Copy-Paste ist nicht erlaubt, ebensowenig Teamimplementierungen.

■ Vorlesung:

- Ursprüngliche Fassung von Prof. B. Gärtner und Dr. F. Friedrich
- Mit Änderungen von Dr. F. Friedrich, Dr. H. Lehner, Dr. M. Schwerhoff

■ Skript: Prof. B. Gärtner

■ Code Expert: Dr. H. Lehner, David Avanthay und anderen

Absichtlich leer

1. Einführung

Informatik: Definition und Geschichte, Algorithmen, Turing Maschine, Höhere Programmiersprachen, Werkzeuge der Programmierung

Was ist Informatik?

Informatik = Automatisierung geistiger Tätigkeiten

Was ist Informatik?

- Die Wissenschaft der systematischen Verarbeitung von *Informationen*, (...)
- ...insbesondere der automatischen Verarbeitung mit Hilfe von Digitalrechnern.

(Wikipedia, nach dem „Duden Informatik“)

Informatik vs. Computer

Computer science is not about machines, in the same way that astronomy is not about telescopes.

Mike Fellows, US-Informatiker (1991)

- Die Informatik beschäftigt sich heute auch mit dem Entwurf von schnellen Computern und Netzwerken...
- ...aber nicht als Selbstzweck, sondern zur effizienteren **systematischen Verarbeitung von Informationen.**

Informatik $\neq$ EDV-Kenntnisse

EDV-Kenntnisse: *Anwenderwissen („Computer Literacy“)*

- Umgang mit dem Computer
- Bedienung von Computerprogrammen (für Texterfassung, E-Mail, Präsentationen,...)

Informatik: *Grundlagenwissen*

- Wie funktioniert ein Computer?
- Wie schreibt man ein Computerprogramm?

Inhalt dieser Vorlesung

- Systematisches Problemlösen mit Algorithmen und der Programmiersprache C++.
- Also: nicht nur,
aber auch Programmierkurs.

Warum Programmieren?

Na, weil Computer und
Software heutzutage
allgegenwärtig sind!



Warum Programmieren?

- Jedes **Verständnis** moderner Technologie erfordert Wissen über die grundlegende Funktionsweise – und Grenzen – eines Computers
- Wir leben mitten in einer **Revolution**, die nicht nur alle Wissenschaften, sondern die ganze Gesellschaft betrifft: Die Automatisierung der geistigen Tätigkeiten schreitet fort von reinen Berechnungen zur künstlichen Intelligenz. Wer mitwirken will, sollte besser programmieren können.
- Programmieren (mit dem Werkzeug Computer) wird zu einer **Kulturtechnik** wie Lesen und Schreiben (mit den Werkzeugen Papier und Bleistift)
- Programmieren bedeutet, etwas zu **erschaffen/erzeugen** – es macht Freude und ist nützlich

Algorithmus: Kernbegriff der Informatik

Algorithmus:

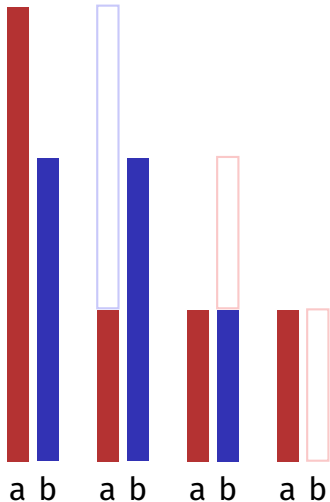
- Handlungsanweisung zur schrittweisen Lösung eines Problems
- Ausführung erfordert keine Intelligenz, nur Genauigkeit (sogar Computer können es)
- nach *Muhammed al-Chwarizmi*,
Autor eines arabischen
Rechen-Lehrbuchs (um 825)



“Dixit algorizmi...” (lateinische Übersetzung)

Der älteste nichttriviale Algorithmus

Euklidischer Algorithmus (aus Euklids *Elementen*, 3. Jh. v. Chr.)



- Eingabe: ganze Zahlen $a > 0, b > 0$
- Ausgabe: ggT von a und b

Solange $b \neq 0$

Wenn $a > b$ dann

$$a \leftarrow a - b$$

Sonst:

$$b \leftarrow b - a$$

Ergebnis: a .

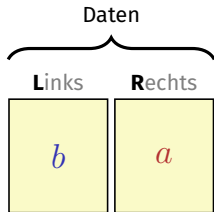
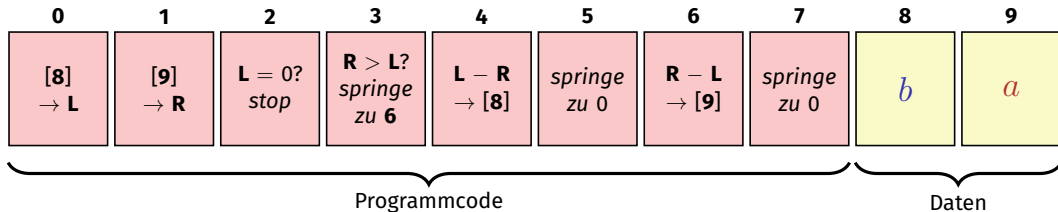
Algorithmen: 3 Abstraktionsstufen

1. **Kernidee** (abstrakt):
Die Essenz eines Algorithmus' („Heureka-Moment“)
2. **Pseudocode** (semi-detailliert):
Für Menschen gemacht (Bildung, Korrektheit- und Effizienzdiskussionen, Beweise)
3. **Implementierung** (sehr detailliert):
Für Mensch & Computer gemacht (les- & ausführbar, bestimmte Programmiersprache, verschiedene Implementierungen möglich)

Euklid: Kernidee und Pseudocode gesehen, Implementierung noch nicht

Euklid in der Box

Speicher



Register

Solange $b \neq 0$

Wenn $a > b$ dann

$$a \leftarrow a - b$$

Sonst:

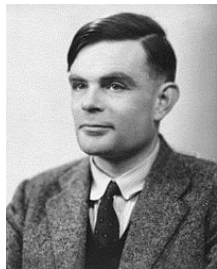
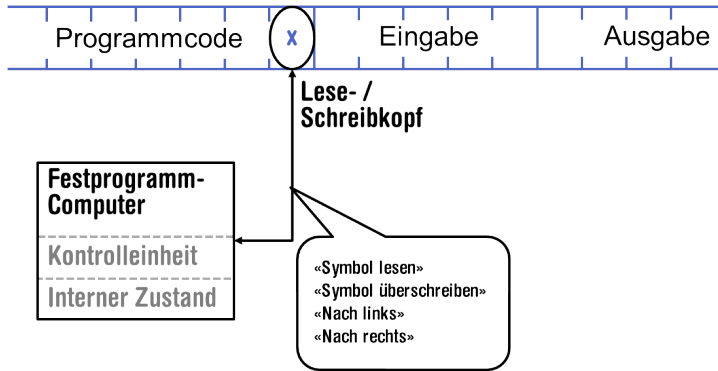
$$b \leftarrow b - a$$

Ergebnis: a .

Computer – Konzept

Eine geniale Idee: Universelle Turingmaschine (Alan Turing, 1936)

Folge von Symbolen auf Ein- und Ausgabeband

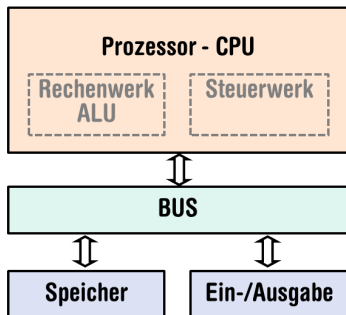


Alan Turing

Computer – Umsetzung

- Z1 – Konrad Zuse (1938)
- ENIAC – John Von Neumann (1945)

Von Neumann Architektur



Konrad Zuse



John von Neumann

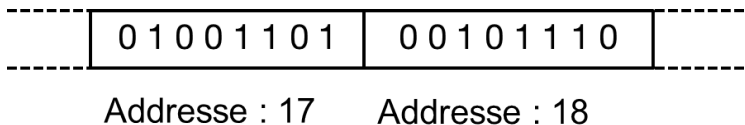
Zutaten der *Von Neumann Architektur*:

- Hauptspeicher (RAM) für Programme und Daten
- Prozessor (CPU) zur Verarbeitung der Programme und Daten
- I/O Komponenten zur Kommunikation mit der Aussenwelt

Speicher für Daten *und* Programm



- Folge von Bits aus $\{0, 1\}$.
- Programmzustand: Werte aller Bits.
- Zusammenfassung von Bits zu Speicherzellen (oft: 8 Bits = 1 Byte).
- Jede Speicherzelle hat eine Adresse.
- Random Access: Zugriffszeit auf Speicherzelle (nahezu) unabhängig von ihrer Adresse.



Der Prozessor (CPU)

- führt Befehle in Maschinensprache aus
- hat eigenen "schnellen" Speicher (Register)
- kann vom Hauptspeicher lesen und in ihn schreiben
- beherrscht eine Menge einfachster Operationen (z.B. Addieren zweier Registerinhalte)

Rechengeschwindigkeit

- Computer sind, anderslautenden Berichten zum Trotz, nicht intelligent
- Aber sie führen Befehle unglaublich schnell aus!
- Zum Beispiel (Befehle pro Sekunde (IPS), grob gerundet):
 - Raspberry Pi 2: 5×10^9
 - Standard Intel/AMD: $200 - 800 \times 10^9$
 - AMD Ryzen Threadripper: 2×10^{12}

(Quelle: https://en.wikipedia.org/wiki/Instructions_per_second)

Programmieren

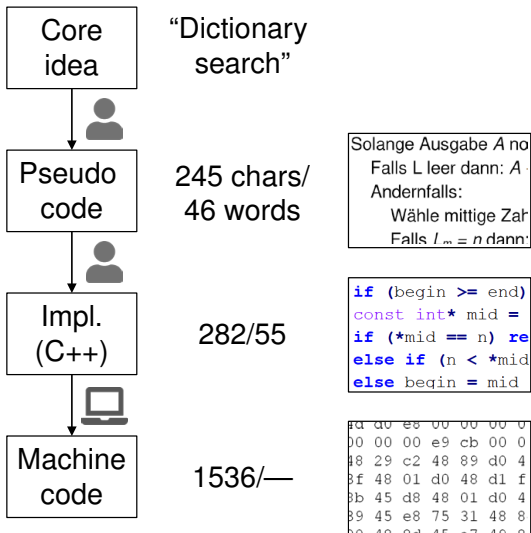
- Mit Hilfe einer Programmiersprache wird dem Computer eine Folge von Befehlen erteilt, damit er genau das macht, was wir wollen.
- Die Folge von Befehlen ist das (Computer)-Programm.



The Harvard Computers, Menschliche Berufsrechner, ca.1890

Programmiersprachen

- Sprache, die der Computer "verstehen", ist sehr primitiv (Maschinensprache).
- Einfache Operationen müssen in (extrem) viele Einzelschritte aufgeteilt werden.
- Sprache variiert von Computer zu Computer.



Warum C++?

Andere populäre Programmiersprachen: Java, C#, Python, JavaScript, Swift, Kotlin, Go, and many more.

- C++ ist relevant in der Praxis, weit verbreitet und „läuft überall“
- C++ ist eine der „schnellsten“ Programmiersprachen
- C++ eignet sich gut für Systemprogrammierung, da es einen sorgfältigen Umgang mit Ressourcen (Speicher, ...) erlaubt/verlangt

Warum C++?

C++ als Einführungssprache? Warum nicht Python oder JavaScript?

- C++ macht viele Entscheidungen explizit, bzw. zwingt Programmierer dazu, explizit eine Wahl zu treffen
- Dies erleichtert einen späteren Umstieg auf „simplere“ Sprachen
- Analogie: Führerausweis auf Schaltwagen machen, aber später Automatik fahren

Syntax und Semantik

Programme müssen, wie unsere Sprache, nach gewissen Regeln geformt werden.

- Syntax: Zusammenfügingsregeln für elementare Zeichen (Buchstaben).
- Semantik: Interpretationsregeln für zusammengefügte Zeichen.

Entsprechende Regeln für ein Computerprogramm sind einfacher, aber auch strenger, denn Computer sind vergleichsweise dumm.

Deutsch vs. C++

Deutsch

Alleen sind nicht gefährlich, Rasen ist gefährlich!
(Wikipedia: Mehrdeutigkeit)

C++

```
// computation  
int b = a * a; //  $b = a^2$   
b = b * b;     //  $b = a^4$ 
```

C++: Fehlerarten illustriert an deutscher Sprache

■ Das Auto fuhr zu schnell.

Syntaktisch und semantisch korrekt.

■ DasAuto fuh r zu sxhnell.

Syntaxfehler: Wortbildung.

■ Rot das Auto ist.

Syntaxfehler: Satzstellung.

■ Man empfiehlt dem Dozenten
nicht zu widersprechen

Syntaxfehler: Satzzeichen fehlen .

■ Sie ist nicht gross und rothaarig.

Syntaktisch korrekt aber mehrdeutig. [kein Analogon]

■ Die Auto ist rot.

Syntaktisch korrekt, doch semantisch fehlerhaft: Falscher Artikel.
[Typfehler]

■ Das Fahrrad galoppiert schnell.

Syntaktisch und grammatikalisch korrekt! Semantisch fehlerhaft.
[Laufzeitfehler]

■ Manche Tiere riechen gut.

Syntaktisch und semantisch korrekt. Semantisch mehrdeutig. [kein Analogon]

Syntax und Semantik von C++

Syntax:

- Wann ist ein Text ein C++Programm?
- D.h. ist es *grammatikalisch* korrekt?
- → Kann vom Computer überprüft werden

Semantik:

- Was *bedeutet* ein Programm?
- Welchen Algorithmus *implementiert* ein Programm?
- → Braucht menschliches Verständnis

Der ISO/IEC Standard 14822 (1998, 2011, 2014, ...)

- ist das „Gesetz“ von C++
- legt Grammatik und Bedeutung von C++-Programmen fest
- wird seit 2011 regelmässig durch Neuerungen für *fortgeschrittenes* Programmieren erweitert

Verhalten eines Programmes

Zur Compilationszeit:

- vom Compiler akzeptiertes Programm (syntaktisch korrektes C++)
- Compiler-Fehler

Zur Laufzeit:

- korrektes Resultat
- inkorrektes Resultat
- Programmabsturz
- Programm *terminiert* nicht (Endlosschleife)

2. Ein C++-Programm und seine Bestandteile

Programmiertools, Bestandteile

Was braucht es zum Programmieren?

- **Editor:** Programm zum Ändern, Erfassen und Speichern von C++-Programmtext
- **Compiler:** Programm zum Übersetzen des Programmtexts in Maschinsprache
- **Computer:** Gerät zum Ausführen von Programmen in Maschinsprache
- **Betriebssystem:** Programm zur Organisation all dieser Abläufe (Dateiverwaltung, Editor-, Compiler- und Programmaufruf)

- Kommentare/Layout
- Include-Direktiven
- Die main-Funktion
- Werte, Effekte
- Typen, Funktionalität
- Literale
- Variablen
- Konstanten
- Bezeichner, Namen
- Ausdrücke
- L- und R-Werte
- Operatoren
- Anweisungen

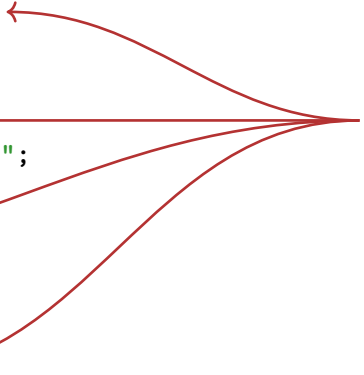
Das erste C++ Programm

```
// Program: power8.cpp
// Raise a number to the eighth power.
#include <iostream>
int main() {
    // input
    std::cout << "Compute a^8 for a =? ";
    int a;
    std::cin >> a;  Anweisungen: Mache etwas (lies  $a$  ein)!
    // computation
    int b = a * a; // b = a^2  Ausdrücke: Berechne einen Wert ( $a^2$ )!
    b = b * b;      // b = a^4
    // output b * b, i.e., a^8
    std::cout << a << "^8 = " << b * b << "\n";
    return 0;
}
```

"Beiwerk": Kommentare

```
// Program: power8.cpp
// Raise a number to the eighth power.
#include <iostream>
int main() {
    // input
    std::cout << "Compute a^8 for a =? ";
    int a;
    std::cin >> a;
    // computation
    int b = a * a; // b = a^2
    b = b * b;      // b = a^4
    // output b * b, i.e., a^8
    std::cout << a << "^8 = " << b * b << "\n";
    return 0;
}
```

Kommentare



Kommentare und Layout

Kommentare

- hat jedes gute Programm,
- dokumentieren, *was* das Programm *wie* macht und wie man es verwendet und
- werden vom Compiler ignoriert.
- Syntax: "Doppelslash" `//` bis Zeilenende.

Ignoriert werden vom Compiler ausserdem

- Leerzeilen, Leerzeichen,
- Einrückungen, die die Programmlogik widerspiegeln (sollten)

Kommentare und Layout

Dem Compiler ist's egal...

```
#include <iostream>
int main(){std::cout << "Compute a^8 for a =? ";
int a; std::cin >> a; int b = a * a; b = b * b;
std::cout << a << "^8 = " << b*b << "\n";return 0;}
```

... uns aber nicht!

"Beiwerk": Include und Main-Funktion

```
// Program: power8.cpp
// Raise a number to the eighth power.
#include <iostream> ← Include-Direktive
int main() { ← Funktionsdeklaration der main-Funktion
    // input
    std::cout << "Compute a^8 for a =? ";
    int a;
    std::cin >> a;
    // computation
    int b = a * a; // b = a^2
    b = b * b;     // b = a^4
    // output b * b, i.e., a^8
    std::cout << a << "^8 = " << b * b << "\n";
    return 0;
}
```

Include-Direktiven

C++ besteht aus

- Kernsprache
- Standardbibliothek
 - Ein/Ausgabe (Header `iostream`)
 - Mathematische Funktionen (`cmath`)
 - ...

```
#include <iostream>
```

- macht Ein/Ausgabe verfügbar

Die Hauptfunktion

Die `main`-Funktion

- existiert in jedem C++ Programm
- wird vom Betriebssystem aufgerufen
- wie eine mathematische Funktion ...
 - Argumente (bei `power8.cpp`: keine)
 - Rückgabewert (bei `power8.cpp`: 0)
- ... aber mit zusätzlichem Effekt.
 - Lies eine Zahl ein und gib die 8-te Potenz aus.

Anweisungen: Mache etwas!

```
int main() {  
    // input  
    std::cout << "Compute a^8 for a =? ";  
    int a;  
    std::cin >> a;  
    // computation  
    int b = a * a; // b = a^2  
    b = b * b; // b = a^4  
    // output b * b, i.e., a^8  
    std::cout << a << "^8 = " << b * b << "\n";  
    return 0;  
}
```

Ausdrucksanweisungen

Rückgabeanweisung

- Bausteine eines C++ Programms
- werden (sequenziell) *ausgeführt*
- enden mit einem Semikolon
- Jede Anweisung hat (potenziell) einen Effekt.

Ausdrucksanweisungen

- haben die Form

`expr;`

wobei *expr* ein Ausdruck ist

- Effekt ist der Effekt von *expr*, der Wert von *expr* wird ignoriert.

```
b = b*b;
```

Rückgabeanweisungen

- treten nur in Funktionen auf und sind von der Form

`return expr;`

wobei *expr* ein Ausdruck ist

- spezifizieren Rückgabewert der Funktion

```
return 0;
```

Anweisungen – Effekte

```
int main() {  
    // input  
    std::cout << "Compute a^8 for a =? ";  
    int a;  
    std::cin >> a;  
    // computation  
    int b = a * a;  
    b = b * b;  
    // output b * b, i.e., a^8  
    std::cout << a << "^8 = " << b * b << "\n";  
    return 0;  
}
```

Effekt: Ausgabe des Strings Compute ...

Effekt: Eingabe einer Zahl und Speichern in a

Effekt: Speichern des berechneten Wertes von $a \cdot a$ in b

Effekt: Speichern des berechneten Wertes von $b \cdot b$ in b

Effekt: Rückgabe des Wertes 0

Effekt: Ausgabe des Wertes von a und des berechneten Wertes von $b \cdot b$

Werte und Effekte

- bestimmen, was das Programm macht,
- sind rein semantische Konzepte:
 - Zeichen 0 bedeutet Wert $0 \in \mathbb{Z}$
 - `std::cin >> a;` bedeutet Effekt "Einlesen einer Zahl"
- hängen vom Programmzustand (Speicherinhalte / Eingaben) ab

Anweisungen – Variablendefinitionen

```
int main() {  
    // input  
    std::cout << "Compute a^8 for a =? ";  
    int a; ← Deklarationsanweisungen  
    std::cin >> a;  
    // computation  
    int b = a * a; ← // b = a^2  
    b = b * b;      // b = a^4  
    // output b * b, i.e., a^8  
    std::cout << a << "^8 = " << b * b << "\n";  
    return 0;  
}
```

Typ-
namen

- führen neue Namen im Programm ein,
- bestehen aus Deklaration + Semikolon Beispiel: `int a;`
- können Variablen auch initialisieren Beispiel: `int b = a * a;`

Typen und Funktionalität

`int`:

- C++ Typ für ganze Zahlen,
- entspricht $(\mathbb{Z}, +, \times)$ in der Mathematik

In C++ hat jeder Typ einen Namen sowie

- Wertebereich (z.B. ganze Zahlen)
- Funktionalität (z.B. Addition/Multiplikation)

Fundamentaltypen

C++ enthält fundamentale Typen für

- Ganze Zahlen (`int`)
- Natürliche Zahlen (`unsigned int`)
- Reelle Zahlen (`float`, `double`)
- Wahrheitswerte (`bool`)
- ...

Variablen

- repräsentieren (wechselnde) Werte
- haben
 - Name
 - Typ
 - Wert
 - Adresse
- sind im Programmtext "sichtbar"

`int a;` definiert Variable mit

- Name: a
- Typ: `int`
- Wert: (vorerst) undefiniert
- Adresse: durch Compiler bestimmt

Bezeichner und Namen

(Variablen-)Namen sind Bezeichner:

- erlaubt: A,...,Z; a,...,z; 0,...,9; _
- erstes Zeichen ist Buchstabe.

Es gibt noch andere Namen:

- `std::cin` (qualifizierter Name)

Ausdrücke: Berechne einen Wert!

Ausdrücke

- repräsentieren *Berechnungen*
- sind entweder primär (b)
- oder zusammengesetzt ($b*b$)...
- ...aus anderen Ausdrücken, mit Hilfe von Operatoren
- haben einen Typ und einen Wert

Analogie: Baukasten

`// input`  Zusammengesetzter Ausdruck

```
std::cout << "Compute a^8 for a =? ";
```

```
int a;
```

```
std::cin >> a;
```

`// computation`

```
int b = a * a; // b = a^2
```

```
b = b * b;
```

 Zweifach zusammengesetzter Ausdruck

~~`// output b * b, i.e., a^8`~~

```
std::cout << a << "^8 = " << b * b << ".\n";
```

`return`  Vierfach zusammengesetzter Ausdruck

Ausdrücke (Expressions)



- repräsentieren *Berechnungen*,
- sind *primär* oder *zusammengesetzt* (aus anderen Ausdrücken und Operationen)

$a * a$

zusammengesetzt aus

Variablenname, Operatorsymbol, Variablenname

Variablenname: primärer Ausdruck

- können geklammert werden

$a * a$ ist äquivalent zu $(a * a)$

Ausdrücke (Expressions)

haben Typ, Wert und Effekt (potenziell).

`a * a`

- Typ: `int` (Typ der Operanden)
- Wert: Produkt von `a` und `a`
- Effekt: keiner.

`b = b * b`

- Typ: `int` (Typ der Operanden)
- Wert: Produkt von `b` und `b`
- Effekt: Weise `b` diesen Wert zu.

Typ eines Ausdrucks ist fest, aber Wert und Effekt werden erst durch die *Auswertung* des Ausdrucks bestimmt.

Literale

- repräsentieren konstante Werte
 - haben festen Typ und Wert
 - sind "syntaktische Werte"
-
- 0 hat Typ `int`, Wert 0.
 - `1.2e5` hat Typ `double`, Wert $1.2 \cdot 10^5$.

Diagram illustrating C++ code with annotations for operators and operands:

- Linker Operand (Ausgabestrom)**: Points to `std::cout` in the first line.
- Ausgabe-Operator**: Points to the first `<<` operator in the first line.
- Rechter Operand (String)**: Points to the string `"Compute a^8 for a =? "` in the first line.
- Rechter Operand (Variablenname)**: Points to `a` in the second line.
- Eingabe-Operator**: Points to the `>>` operator in the second line.
- Linker Operand (Eingabestrom)**: Points to `std::cin` in the second line.
- Zuweisungsoperator**: Points to the `=` operator in the third line.
- Multiplikationsoperator**: Points to the `*` operator in the fourth line.

```
// input
std::cout << "Compute a^8 for a =? ";
int a;
std::cin >> a;

// computation
int b = a;
b = b * b;    // b = a^4

// output
std::cout << a << "^8 = " << b * b << "\n";
return 0;
```

Operatoren

- machen aus Ausdrücken (*Operanden*) neue zusammengesetzte Ausdrücke
- spezifizieren für die Operanden und das Ergebnis die Typen, und ob sie L- oder R-Werte sein müssen
- haben eine Stelligkeit

Zuweisungsoperator =

- Linker Operand ist **L**-Wert (hat Adresse),
- Rechter Operand ist **R**-Wert des gleichen Typs.
- Weist linkem Operanden den Wert des rechten Operanden zu und gibt den linken Operanden als L-Wert zurück

Beispiele: $b = b * b$ und $a = b$

Vorsicht, Falle!

Der Operator = entspricht dem Zuweisungsoperator in der Mathematik ($:=$), nicht dem Vergleichsoperator ($=$).

Multiplikationsoperator *



- erwartet zwei R-Werte vom gleichen Typ als Operanden (Stelligkeit 2)
- "gibt Produkt als R-Wert des gleichen Typs zurück", das heisst formal:
 - Der zusammengesetzte Ausdruck ist ein R-Wert; sein Wert ist das Produkt der Werte der beiden Operanden

Beispiele: $a * a$ und $b * b$

Eingabeoperator >>



- linker Operand ist L-Wert (*Eingabestrom*)
- rechter Operand ist L-Wert
- weist dem rechten Operanden den nächsten Wert aus der Eingabe zu, *entfernt ihn aus der Eingabe* und gibt den Eingabestrom als L-Wert zurück Beispiel: `std::cin >> a` (meist Tastatureingabe)
- Eingabestrom wird verändert und muss deshalb ein L-Wert sein!

Ausgabeoperator <<



- linker Operand ist L-Wert (*Ausgabestrom*)
- rechter Operand ist R-Wert
- gibt den Wert des rechten Operanden aus, fügt ihn dem Ausgabestrom hinzu und gibt den Ausgabestrom als L-Wert zurück Beispiel:
`std::cout << a` (meist Bildschirmausgabe)
- Ausgabestrom wird verändert und muss deshalb ein L-Wert sein!

Ausgabeoperator <<

Warum Rückgabe des Ausgabestroms?

- erlaubt Bündelung von Ausgaben

```
std::cout << a << "^8 = " << b * b << "\n"
```

ist wie folgt logisch geklammert

```
(((((std::cout << a) << "^8 = ") << b * b) << "\n"))
```

- `std::cout << a` dient als linker Operand des nächsten << und ist somit ein L-Wert, der kein Variablenname ist.

L-Werte und R-Werte

```
// input
std::cout << "Compute a^8 for a =? ";
int a;
std::cin >> a;
```

R-Wert (points to the string literal "Compute a^8 for a =? ")

L-Wert (Ausdruck + Adresse) (points to the variable `a`)


```
// computation
int b = a * a; // b = a^2
b = b * b; // b = a^4
```

L-Wert (Ausdruck + Adresse) (points to the variable `b` in `b = b * b;`)

R-Wert (points to the expression `b * b` in `b = b * b;`)


```
// output b * b, i.e., a^8
std::cout << a << "^8 = " << b * b << ".\n";
return 0;
```

R-Wert (Ausdruck, der kein L-Wert ist) (points to the expression `b * b` in the output statement)

R-Wert (points to the constant `0` in `return 0;`)

L-Wert (“**L**inks vom Zuweisungsoperator”)

- Ausdruck mit Adresse
- Wert ist der Inhalt an der Speicheradresse entsprechend dem Typ des Ausdrucks.
- L-Wert kann seinen Wert ändern (z.B. per Zuweisung).

Beispiel: Variablenname

L-Werte und R-Werte

R-Wert (“**R**echts vom Zuweisungsoperator”)

- Ausdruck der kein L-Wert ist
- Jeder L-Wert kann als R-Wert benutzt werden (aber nicht umgekehrt).
- Ein R-Wert kann seinen Wert *nicht ändern*.

Beispiel: Literal 0

Terminologie: L-Werte und R-Werte

L-Wert (“**L**inks vom Zuweisungsoperator”)

- Ausdruck der einen Speicherplatz identifiziert
- Zum Beispielen eine Variable *a*
(später im Kurs lernen wir weitere L-Werte kennen)
- Wert ist der Inhalt an der Speicheradresse entsprechend dem Typ des Ausdrucks (für *a* z.B. Wert 2).
- L-Wert kann seinen Wert *ändern* (z.B. per Zuweisung).

Terminologie: L-Werte und R-Werte

R-Wert (“**R**echts vom Zuweisungsoperator”)

- Ausdruck der kein L-Wert ist
- Beispiel: Integer-Literal 0
- Jeder L-Wert kann als R-Wert benutzt werden (aber nicht umgekehrt) ...
- ... indem der *Wert* des L-Werts genommen wird
(z.B. könnte der L-Wert a den Wert 2 haben, der dann als R-Wert verwendet wird)
- Ein R-Wert kann seinen Wert *nicht ändern*

L-Werte und R-Werte

```
std::cout << "Compute a^8 for a = ? ";  
int a;  
std::cin >> a;  
int r = a * a; // r = a^2  
r = r * r; // r = a^4  
std::cout << a << "^8 = " << r * r << ".\n";  
return 0;
```

R-Wert

L-Wert (Ausdruck + Adresse)

L-Wert (Ausdruck + Adresse)

R-Wert

R-Wert (Ausdruck, der kein L-Wert ist)

Der Compiler als Freund: Konstanten

Konstanten

- sind Variablen mit unveränderbarem Wert

```
const int speed_of_light = 299792458;
```

- Verwendung: `const` vor der Definition

Der Compiler als Freund: Konstanten

- Compiler kontrolliert Einhaltung des `const`-Versprechens

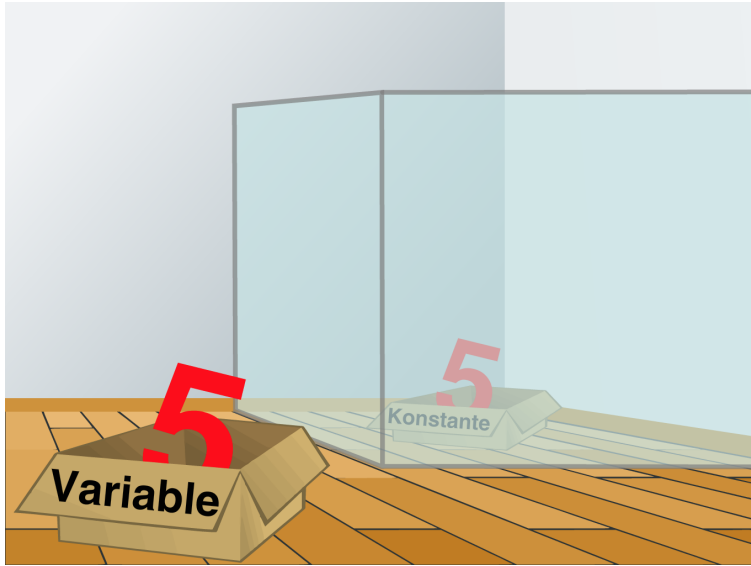
```
const int speed_of_light = 299792458;  
...  
speed_of_light = 3000000000;
```

Compilerfehler!



- Hilfsmittel zur Vermeidung von Fehlern: Konstanten erlauben garantierte Einhaltung des Versprechens „Wert ändert sich nicht“

Konstanten: Variablen hinter Glas



Die `const`-Richtlinie

Const-Richtlinie

Denke bei *jeder Variablen* darüber nach, ob sie im Verlauf des Programmes jemals ihren Wert ändern wird oder nicht. Im letzteren Falle verwende das Schlüsselwort `const`, um die Variable zu einer Konstanten zu machen.

Ein Programm, welches diese Richtlinie befolgt, heisst `const`-korrekt.

3. Ganze Zahlen

Auswertung arithmetischer Ausdrücke, Assoziativität und Präzedenz, arithmetische Operatoren, Wertebereich der Typen **int**, **unsigned int**

Beispiel: power8.cpp

```
int a; // Input
int r; // Result

std::cout << "Compute a^8 for a = ?";
std::cin >> a;

r = a * a; // r = a^2
r = r * r; // r = a^4

std::cout << "a^8 = " << r*r << '\n';
```

Celsius to Fahrenheit

```
// Program: fahrenheit.cpp
// Convert temperatures from Celsius to Fahrenheit.
#include <iostream>

int main() {
    // Input
    std::cout << "Temperature in degrees Celsius =? ";
    int celsius;
    std::cin >> celsius;

    // Computation and output
    std::cout << celsius << " degrees Celsius are "
              << 9 * celsius / 5 + 32 << " degrees Fahrenheit.\n";
    return 0;
}
```

Arithmetischer Ausdruck

```
9 * celsius / 5 + 32
```

- Arithmetischer Ausdruck,
- enthält drei Literale, eine Variable, drei Operatorsymbole

Wie ist der Ausdruck geklammert?

Präzedenz

Punkt vor Strichrechnung

`9 * celsius / 5 + 32`

bedeutet

`(9 * celsius / 5) + 32`

Regel 1: Präzedenz

Multiplikative Operatoren (*, /, %) haben höhere Präzedenz („binden stärker“) als additive Operatoren (+, -)

Assoziativität

Von links nach rechts

`9 * celsius / 5 + 32`

bedeutet

`((9 * celsius) / 5) + 32`

Regel 2: Assoziativität

Arithmetische Operatoren (`*`, `/`, `%`, `+`, `-`) sind linksassoziativ: bei gleicher Präzedenz erfolgt Auswertung von links nach rechts

Stelligkeit

Vorzeichen

$-3 - 4$

bedeutet

$(-3) - 4$

Regel 3: Stelligkeit

Unäre Operatoren $+$, $-$ vor binären $+$, $-$.

Klammerung

Jeder Ausdruck kann mit Hilfe der

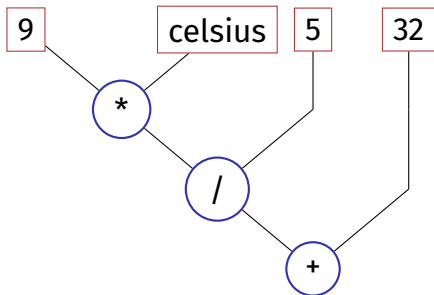
- Assoziativitäten
- Präzedenzen
- Stelligkeiten (Anzahl Operanden)

der beteiligten Operatoren eindeutig geklammert werden (Details im Skript).

Ausdrucksbäume

Klammerung ergibt Ausdrucksbaum

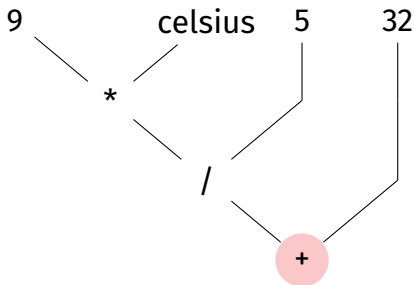
`((9 * celsius) / 5) + 32)`



Auswertungsreihenfolge

Von oben nach unten im Ausdrucksbaum

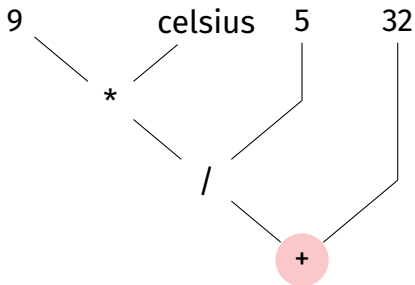
9 * celsius / 5 + 32



Auswertungsreihenfolge

Reihenfolge nicht eindeutig bestimmt:

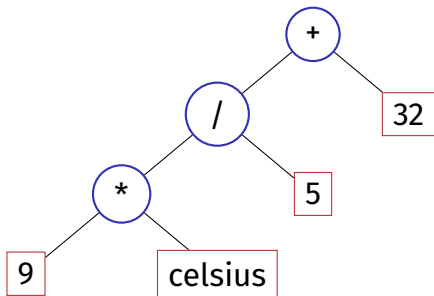
`9 * celsius / 5 + 32`



Ausdrucksbäume – Notation

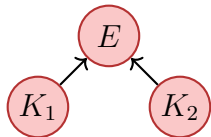
Üblichere Notation: Wurzel oben

`9 * celsius / 5 + 32`



Auswertungsreihenfolge – formaler

Gültige Reihenfolge: Jeder Knoten wird erst nach seinen Kindern ausgewertet.



C++: anzuwendende gültige Reihenfolge nicht spezifiziert.

- "Guter Ausdruck": jede gültige Reihenfolge führt zum gleichen Ergebnis.
- Beispiel eines "schlechten Ausdrucks": $a * (a=2)$

Auswertungsreihenfolge

Richtlinie

Vermeide das Verändern von Variablen, welche im selben Ausdruck noch einmal verwendet werden!

Arithmetische Operatoren

	Symbol	Stelligkeit	Präzedenz	Assoziativität
Unäres +	+	1	16	rechts
Negation	-	1	16	rechts
Multiplikation	*	2	14	links
Division	/	2	14	links
Modulo	%	2	14	links
Addition	+	2	13	links
Subtraktion	-	2	13	links

Alle Operatoren: $[R\text{-Wert} \times] R\text{-Wert} \rightarrow R\text{-Wert}$

CPP Reference Link, Vorsicht: dort Bindung stärker bei kleinerer Zahl

https://en.cppreference.com/w/cpp/language/operator_precedence

Einschub: Zuweisungsausdruck – nun genauer

- Bereits bekannt: $a = b$ bedeutet Zuweisung von b (R-Wert) an a (L-Wert). Rückgabe: L-Wert.
- Was bedeutet $a = b = c$?
- Antwort: Zuweisung rechtsassoziativ, also

$$a = b = c \quad \Longleftrightarrow \quad a = (b = c)$$

Mehrfachzuweisung: $a = b = 0 \implies b=0; a=0$

Division

- Operator / realisiert ganzzahlige Division

`5 / 2` hat Wert 2

- In `fahrenheit.cpp`

```
9 * celsius / 5 + 32
```

15 degrees Celsius are 59 degrees Fahrenheit

- Mathematisch äquivalent...aber nicht in C++!

```
9 / 5 * celsius + 32
```

15 degrees Celsius are 47 degrees Fahrenheit

Präzisionsverlust

Richtlinie

- Auf möglichen Präzisionsverlust achten
- Potentiell verlustbehaftete Operationen möglichst spät durchführen, um „Fehlereskalation“ zu vermeiden

Division und Modulo

- Modulo-Operator berechnet Rest der ganzzahligen Division

$5 / 2$ hat Wert 2, $5 \% 2$ hat Wert 1.

- Es gilt

$$(-a)/b == -(a/b)$$

- Es gilt auch:

$$(a / b) * b + a \% b \text{ hat den Wert von } a.$$

- Daraus lässt sich herleiten, welche Ergebnisse Division und Modulo mit negativen Zahlen ergeben (müssen)

Inkrement und Dekrement

- Erhöhen / Erniedrigen einer Zahl um 1 ist eine häufige Operation
- geht für einen L-Wert so:

```
expr = expr + 1.
```

Nachteile

- relativ lang
- `expr` wird zweimal ausgewertet
 - Später: L-wertige Ausdrücke deren Auswertung „teuer“ ist
 - `expr` könnte einen Effekt haben (aber sollte nicht, siehe Richtlinie)

In-/Dekrement Operatoren

Post-Inkrement

```
expr++
```

Wert von `expr` wird um 1 erhöht, der alte Wert von `expr` wird (als R-Wert) zurückgegeben

Prä-Inkrement

```
++expr
```

Wert von `expr` wird um 1 erhöht, der neue Wert von `expr` wird (als L-Wert) zurückgegeben

Post-Dekrement

```
expr--
```

Wert von `expr` wird um 1 verringert, der alte Wert von `expr` wird (als R-Wert) zurückgegeben

Prä-Dekrement

```
--expr
```

Wert von `expr` wird um 1 verringert, der neue Wert von `expr` wird (als L-Wert) zurückgegeben

In-/Dekrement Operatoren

	Gebrauch	Stelligkeit	Präz	Assoz.	L/R-Werte
Post-Inkrement	<code>expr++</code>	1	17	links	L-Wert $\rightarrow$ R-Wert
Prä-Inkrement	<code>++expr</code>	1	16	rechts	L-Wert $\rightarrow$ L-Wert
Post-Dekrement	<code>expr--</code>	1	17	links	L-Wert $\rightarrow$ R-Wert
Prä-Dekrement	<code>--expr</code>	1	16	rechts	L-Wert $\rightarrow$ L-Wert

In-/Dekrement Operatoren

```
int a = 7;  
std::cout << ++a << "\n"; // 8  
std::cout << a++ << "\n"; // 8  
std::cout << a << "\n"; // 9
```


In-/Dekrement Operatoren

Ist die Anweisung

`++expr;` ← wir bevorzugen dies

äquivalent zu

`expr++;?`

Ja, aber

- Prä-Inkrement ist effizienter (alter Wert muss nicht gespeichert werden)
- Post-In/Dekrement sind die einzigen linksassoziativen unären Operatoren (nicht sehr intuitiv)

C++ VS. ++C

Eigentlich sollte unsere Sprache ++C heissen, denn

- sie ist eine Weiterentwicklung der Sprache C,
- während C++ ja immer noch das alte C liefert.

Arithmetische Zuweisungen

a += b

$\Leftrightarrow$

a = a + b

Analog für -, *, / und %

Arithmetische Zuweisungen

Gebrauch		Bedeutung
<code>+=</code>	<code>expr1 += expr2</code>	<code>expr1 = expr1 + expr2</code>
<code>-=</code>	<code>expr1 -= expr2</code>	<code>expr1 = expr1 - expr2</code>
<code>*=</code>	<code>expr1 *= expr2</code>	<code>expr1 = expr1 * expr2</code>
<code>/=</code>	<code>expr1 /= expr2</code>	<code>expr1 = expr1 / expr2</code>
<code>%=</code>	<code>expr1 %= expr2</code>	<code>expr1 = expr1 % expr2</code>

Arithmetische Zuweisungen werten `expr1` nur einmal aus.
Zuweisungen haben Präzedenz 4 und sind rechtsassoziativ

Binäre Zahlendarstellungen

Binäre Darstellung (Bits aus $\{0, 1\}$)

$$b_n b_{n-1} \dots b_1 b_0$$

entspricht der Zahl $b_n \cdot 2^n + \dots + b_1 \cdot 2 + b_0$

101011 entspricht 43.

Niedrigstes Bit, Least Significant Bit (LSB)

Höchstes Bit, Most Significant Bit (MSB)

Rechentricks

■ Abschätzung der Grössenordnung von Zweierpotenzen¹:

$$2^{10} = 1024 = 1\text{Ki} \approx 10^3.$$

$$2^{20} = 1\text{Mi} \approx 10^6,$$

$$2^{30} = 1\text{Gi} \approx 10^9,$$

$$2^{32} = 4 \cdot (1024)^3 = 4\text{Gi} \approx 4 \cdot 10^9.$$

$$2^{64} = 16\text{Ei} \approx 16 \cdot 10^{18}.$$

¹Dezimal vs. Binäre Einheiten: MB - Megabyte vs. MiB - Megabibyte (etc.)
kilo (K, Ki) – mega (M, Mi) – giga (G, Gi) – tera(T, Ti) – peta(P, Pi) – exa (E, Ei)

Hexadezimale Zahlen

Zahlen zur Basis 16. Darstellung

$$h_n h_{n-1} \dots h_1 h_0$$

entspricht der Zahl

$$h_n \cdot 16^n + \dots + h_1 \cdot 16 + h_0.$$

Schreibweise in C++: vorangestelltes 0x

0xff entspricht 255.

Hex Nibbles

hex	bin	dec
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
a	1010	10
b	1011	11
c	1100	12
d	1101	13
e	1110	14
f	1111	15

Wozu Hexadezimalzahlen?

- Ein Hex-Nibble entspricht genau 4 Bits. Die Zahlen 1, 2, 4 und 8 repräsentieren Bits 0, 1, 2 und 3.
- Kompakte Darstellung von Binärzahlen.

32-bit Zahlen bestehen aus acht Hex-Nibbles: 0x00000000 -- 0xffffffff

.

0x400 = 1Ki = 1'024.

0x100000 = 1Mi = 1'048'576.

0x40000000 = 1Gi = 1'073.741,824.

0x80000000: höchstes Bit einer 32-bit Zahl gesetzt.

0xffffffff: alle Bits einer 32-bit Zahl gesetzt.

“0x8a20aaf0 ist eine Adresse in den oberen 2G des 32-bit Addressraumes”

Wozu Hexadezimalzahlen?

Für Programmierer und Techniker (Bedienungsanleitung des Schachcomputers *Mephisto II*, 1981)

Beispiele:

8200

- a) Anzeige 8200
MEPHISTO ist mit genau 2 Bauern-Einheiten im Vorteil.

7F00

- b) Anzeige 7F00
MEPHISTO ist mit genau 1 Bauern-Einheit im Nachteil.

Die Anzeige erfolgt in **hexadezimaler Schreibweise**. Im Gegensatz zum gewohnten Dezimalsystem gehen die Ziffern an jeder Stelle von 0 bis F (A = 10, B = 11, ..., F = 15).

Für mathematisch Vorgebildete nachstehend die Umrechnungsformel in das dezimale Punktsystem:

$$ABCD = (A \times 16^3) + (B \times 16^2) + (C \times 16^1) + (D \times 16^0)$$

Für A gilt: 7 = -1; 8 = 0; 9 = +1 usw.

Eine Bauereinheit (B) wird ausgedrückt in $16^2 = 256$ Punkten. Dieses auf den ersten Blick vielleicht etwas komplizierte System dient der Service-Freundlichkeit von MEPHISTO, sowie insbesondere der Entwicklungsarbeit an zukünftigen, noch stärkeren Programmen, ist also mehr für unsere Programmierer und Techniker vorgesehen.

Beispiele:

805E

- c) Anzeige 805E
(E=14) Umrechnung nach folgendem Verfahren:
 $(14 \times 16^0) + (5 \times 16^1) + (0 \times 16^2) + (8 \times 16^3) = 14 + 80 + 0 + 0 =$
 $= +94 \text{ Punkte.}$

7F80

- d) Anzeige 7F80
(7=-1; F=15) Umrechnung wie folgt:
 $(0 \times 16^0) + (8 \times 16^1) + (15 \times 16^2) - (1 \times 16^3) = 0 + 128 + 3840 - 4096 =$

Beispiel: Hex-Farben

#00FF00



r g b

Die NZZ hätte viel Platz sparen können...

4e 5a 5a

www.nzz.ch • Fr. 4.00 • €3.50

```

011000011 011010000 01101000 01101000 01101001 01101110 01100111 01110001 01100101 01101000 01100101 01101110 01100100 0010
0000 01101001 01101110 00100000 01101000 01100001 01110100 01100101 01100001 01110011 000001101 000001010 01000100 01101010

```

01100011 01101000 01110100 01100101

01110011 01102001

01100110 01100101

```

01110010 01101110 01100111 01100101 011
01000 01100001 01101100 01101000 0110
0101 0110111000001101 00001010 0000110
00001010 01001100 01100001 01110101 01

```

00100000 01110110

```
01000101 01110010 01100001 01101110 0111-
0100 01110111 01101111 01110010 01110100
01101100 01101001 01100011 01101000 001-
01110 00001011 00001010 00001011 000-
01010 01001010 11111100 01110100 011001-
11 00100010 00100010 01101001 01110011
```

Wertebereich des Typs int

```
// Output the smallest and the largest value of type int.
#include <iostream>
#include <limits>

int main() {
    std::cout << "Minimum int value is "
               << std::numeric_limits<int>::min() << "...\n"
               << "Maximum int value is "
               << std::numeric_limits<int>::max() << "...\n";
    return 0;
}
```

Minimum int value is -2147483648.
Maximum int value is 2147483647.
Woher kommen diese Zahlen?

Wertebereich des Typs `int`

- Repräsentation mit B Bits. Wertebereich umfasst die 2^B ganzen Zahlen:

$$\{-2^{B-1}, -2^{B-1} + 1, \dots, -1, 0, 1, \dots, 2^{B-1} - 2, 2^{B-1} - 1\}$$

- Auf den meisten Plattformen $B = 32$
- Für den Typ `int` garantiert C++ $B \geq 16$
- Hintergrund: Abschnitt 2.2.8 (Binary Representation) im Skript

Überlauf und Unterlauf

- Arithmetische Operationen (+, -, *) können aus dem Wertebereich herausführen.
- Ergebnisse können inkorrekt sein.

`power8.cpp`: $15^8 = -1732076671$

- In C++, Über-/Unterlauf auf `int` sind C++ **undefiniertes Verhalten**, d.h. **keine Fehlermeldungen**, keine Garantien für das Ergebnis

Der Typ `unsigned int`

- Wertebereich

$$\{0, 1, \dots, 2^B - 1\}$$

- Alle arithmetischen Operationen gibt es auch für `unsigned int`.

- Literale: `1u`, `17u` ...

int VS. unsigned int

- Für `int` sind Überlauf und Unterlauf undefiniertes Verhalten.
- Für `unsigned int` sind Überlauf und Unterlauf klar definiert, durch Rechnung modulo $2^B - 1$.
- Im Rahmen dieses Kurses ignorieren wir diese Fehler zumeist, und Sie können das auch, ausser wo wir es explizit verlangen.
- Andere Sprachen handhaben dieses Problem verschieden: einige geben Fehlermeldungen, andere vergrössern den Datentyp bis der Speicher gefüllt ist.

https://en.cppreference.com/w/cpp/language/operator_arithmetic#Overflows

https://en.wikipedia.org/wiki/Integer_overflow#Methods_to_address_integer_overflow_problems

Gemischte Ausdrücke

- Operatoren können Operanden verschiedener Typen haben (z.B. `int` und `unsigned int`).

```
17 + 17u
```

- Solche gemischten Ausdrücke sind vom "allgemeineren" Typ `unsigned int`.
- `int`-Operanden werden konvertiert nach `unsigned int`.

Konversion

int Wert	Vorzeichen	unsigned int Wert
x	≥ 0	x
x	< 0	$x + 2^B$

Konversion "andersherum"

Die Deklaration

```
int a = 3u;
```

konvertiert `3u` nach `int`.

Der Wert bleibt erhalten, weil er im Wertebereich von `int` liegt; andernfalls ist das Ergebnis implementierungsabhängig.

Zahlen mit Vorzeichen

Hinweis: Die verbleibenden Folien zur vorzeichenbehafteten Zahlendarstellung, dem Rechnen mit Binärzahlen sowie der Zweierkomplementdarstellung sind *nicht* klausurrelevant

Vorzeichenbehaftete Zahlendarstellung

- Soweit klar (hoffentlich): Binäre Zahlendarstellung ohne Vorzeichen, z.B.

$$[b_{31}b_{30} \dots b_0]_u \quad \hat{=} \quad b_{31} \cdot 2^{31} + b_{30} \cdot 2^{30} + \dots + b_0$$

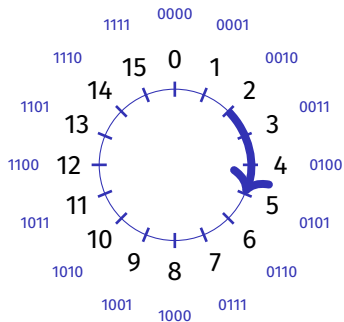
- Suche möglichst konsistente Lösung

Die Darstellung mit Vorzeichen sollte möglichst viel mit der vorzeichenlosen Lösung „gemein haben“. Positive Zahlen sollten sich in beiden Systemen algorithmisch möglichst gleich verhalten.

Rechnen mit Binärzahlen (4 Stellen)

Einfache Addition

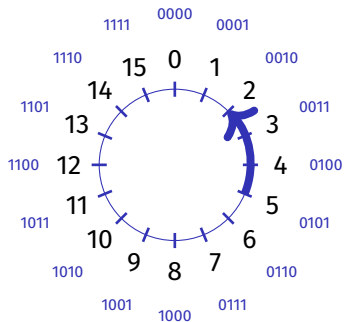
$$\begin{array}{r} 2 \\ +3 \\ \hline 5 \end{array} \quad \begin{array}{r} 0010 \\ +0011 \\ \hline 0101_2 = 5_{10} \end{array}$$



Rechnen mit Binärzahlen (4 Stellen)

Einfache Subtraktion

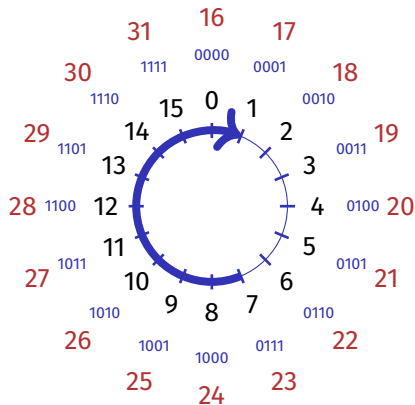
$$\begin{array}{r} 5 \qquad 0101 \\ -3 \qquad -0011 \\ \hline 2 \qquad 0010_2 = 2_{10} \end{array}$$



Rechnen mit Binärzahlen (4 Stellen)

Addition mit Überlauf

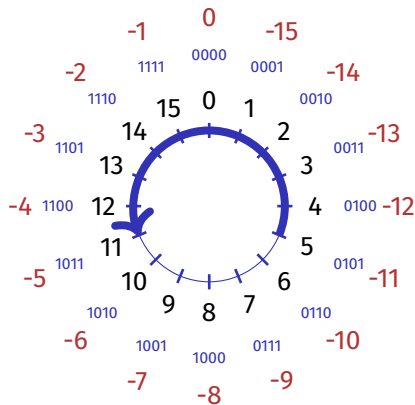
$$\begin{array}{r} 7 \qquad 0111 \\ +10 \quad +1010 \\ \hline 17 \qquad (1)0001_2 = 1_{10}(= 17 \bmod 16) \end{array}$$



Rechnen mit Binärzahlen (4 Stellen)

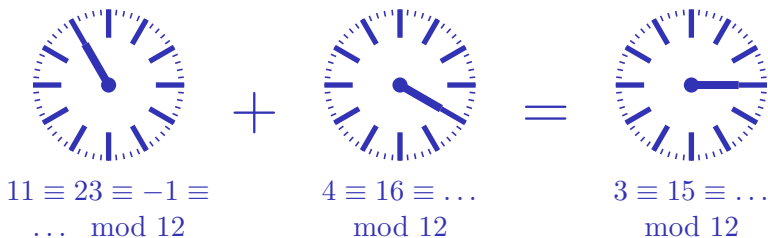
Subtraktion mit Unterlauf

$$\begin{array}{r} 5 \quad 0101 \\ -10 \quad -1010 \\ \hline -5 \quad (\dots 11)1011_2 = 11_{10} (= -5 \bmod 16) \end{array}$$



Analogie

Modulo-Arithmetik: Rechnen im Kreis mit der Uhr²



²Die Arithmetik funktioniert auch für die Multiplikation

Negative Zahlen (3 Stellen)

	a	$-a$	
0	000	000	0
1	001	111	-1
2	010	110	-2
3	011	101	-3
4	100	100	-4
5	101		
6	110		
7	111		

Das höchste Bit entscheidet über das Vorzeichen *und* es trägt zum Zahlwert bei.

- Negation durch bitweise Negation und Addition von 1.

$$-2 = -[0010] = [1101] + [0001] = [1110]$$

- Arithmetik der Addition und Subtraktion identisch zur vorzeichenlosen Arithmetik.

$$3 - 2 = 3 + (-2) = [0011] + [1110] = [0001]$$

- Intuitive „Wrap-Around“ Konversion negativer Zahlen.

$$-n \rightarrow 2^B - n$$

- Wertebereich: $-2^{B-1} \dots 2^{B-1} - 1$

4. Wahrheitswerte

Boolesche Funktionen; der Typ **bool**; logische und relationale Operatoren; Kurzschlussauswertung

Wo wollen wir hin?

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";  
else  
    std::cout << "odd";
```

Verhalten hängt ab vom Wert eines Booleschen Ausdrucks

Boolesche Werte in der Mathematik

Boolesche Ausdrücke können zwei mögliche Werte annehmen:

0 oder 1

- 0 entspricht „falsch“
- 1 entspricht „wahr“

Der Typ `bool` in C++

- Repräsentiert Wahrheitswerte
- Literale `false` und `true`
- Wertebereich `{false, true}`

```
bool b = true; // Variable mit Wert true (wahr)
```


Relationale Operatoren

$a < b$ (kleiner als)
 $a \geq b$ (größer gleich)
 $a == b$ (gleich)
 $a \neq b$ (ungleich)

$\text{Zahlentyp} \times \text{Zahlentyp} \rightarrow \text{bool}$

$\text{R-Wert} \times \text{R-Wert} \rightarrow \text{R-Wert}$

Relationale Operatoren: Tabelle

	Symbol	Stelligkeit	Präzedenz	Assoziativität
Kleiner	<	2	11	links
Grösser	>	2	11	links
Kleiner gleich	<=	2	11	links
Grösser gleich	>=	2	11	links
Gleich	==	2	10	links
Ungleich	!=	2	10	links

$\text{Zahlentyp} \times \text{Zahlentyp} \rightarrow \text{bool}$

$\text{R-Wert} \times \text{R-Wert} \rightarrow \text{R-Wert}$

Boolesche Funktionen in der Mathematik

- Boolesche Funktion

$$f : \{0, 1\}^2 \rightarrow \{0, 1\}$$

- 0 entspricht „falsch“.
- 1 entspricht „wahr“.

$$x \wedge y$$

■ „Logisches Und“

$$f : \{0, 1\}^2 \rightarrow \{0, 1\}$$

■ 0 entspricht „falsch“.

■ 1 entspricht „wahr“.

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

Logischer Operator &&

`a && b` (logisches Und)

`bool` $\times$ `bool` $\rightarrow$ `bool`

R-Wert $\times$ R-Wert $\rightarrow$ R-Wert

```
int n = -1;  
int p = 3;  
bool b = (n < 0) && (0 < p); // b = true (wahr)
```

$$x \vee y$$

■ „Logisches Oder“

$$f : \{0, 1\}^2 \rightarrow \{0, 1\}$$

■ 0 entspricht „falsch“.

■ 1 entspricht „wahr“.

x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

Logischer Operator ||

a || b (logisches Oder)

$\text{bool} \times \text{bool} \rightarrow \text{bool}$

R-Wert $\times$ R-Wert $\rightarrow$ R-Wert

```
int n = 1;  
int p = 0;  
bool b = (n < 0) || (0 < p); // b = false (falsch)
```

$\neg x$

■ „Logisches Nicht“

$$f : \{0, 1\} \rightarrow \{0, 1\}$$

■ 0 entspricht „falsch“.

■ 1 entspricht „wahr“.

x	$\neg x$
0	1
1	0

Logischer Operator !

!b (logisches Nicht)

bool → bool

R-Wert → R-Wert

```
int n = 1;  
bool b = !(n < 0); // b = true (wahr)
```

Logische Operatoren: Tabelle



	Symbol	Stelligkeit	Präzedenz	Assoziativität
Logisches Und (AND)	&&	2	6	links
Logisches Oder (OR)	 	2	5	links
Logisches Nicht (NOT)	!	1	16	rechts

Präzedenzen

$!b \ \&\& \ a$
 $\Updownarrow$
 $(!b) \ \&\& \ a$

$a \ \&\& \ b \ || \ c \ \&\& \ d$
 $\Updownarrow$
 $(a \ \&\& \ b) \ || \ (c \ \&\& \ d)$

$a \ || \ b \ \&\& \ c \ || \ d$
 $\Updownarrow$
 $a \ || \ (b \ \&\& \ c) \ || \ d$

Präzedenzen

Der unäre logische Operator !

bindet stärker als

binäre arithmetische Operatoren. Diese

binden stärker als

relationale Operatoren,

und diese binden stärker als

binäre logische Operatoren.

```
7 + x < y && y != 3 * z || ! b
((7 + x) < y) && (y != (3 * z)) || (!b)
```

Vollständigkeit

- $\wedge$, $\vee$ und $\neg$ sind die in C++ verfügbaren Booleschen Funktionen.
- Alle anderen *binären* Booleschen Funktionen sind daraus erzeugbar.
- Beispiel: Exclusives Oder $\oplus$.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Vollständigkeit: $x \oplus y$

$$x \oplus y = (x \vee y) \wedge \neg(x \wedge y).$$

$$(x \mid\mid y) \ \&\& \ ! (x \ \&\& \ y)$$

Vollständigkeit Beweis

- Identifiziere binäre Boolesche Funktionen mit ihrem charakteristischen Vektor.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Charakteristischer Vektor: 0110

$$x \oplus y = f_{0110}(x, y)$$

Vollständigkeit Beweis

- Schritt 1: erzeuge die *elementaren* Funktionen $f_{0001}, f_{0010}, f_{0100}, f_{1000}$

$$f_{0001}(x, y) = x \wedge y$$

$$f_{0010}(x, y) = x \wedge \neg y$$

$$f_{0100}(x, y) = \neg x \wedge y$$

$$f_{1000}(x, y) = \neg(x \vee y)$$

Vollständigkeit Beweis

- Schritt 2: erzeuge alle Funktionen durch „Veroderung“ elementarer Funktionen

$$f_{\textcolor{red}{1}10\textcolor{red}{1}}(x, y) = f_{\textcolor{red}{1}000}(x, y) \vee f_{01\textcolor{red}{1}00}(x, y) \vee f_{0000\textcolor{red}{1}}(x, y).$$

- Schritt 3: erzeuge f_{0000}

$$f_{0000} = 0.$$

bool vs int: Konversion

- bool kann überall dort verwendet werden, wo int gefordert ist – und umgekehrt.
- Viele existierende Programme verwenden statt bool den Typ int.

Das ist schlechter Stil, der noch auf die Sprache C zurückgeht.

bool	→	int
<i>true</i>	→	1
<i>false</i>	→	0
int	→	bool
$\neq 0$	→	<i>true</i>
0	→	<i>false</i>

```
bool b = 3; // b=true
```

DeMorgansche Regeln

■ $!(a \ \&\& \ b) == (!a \ || \ !b)$

■ $!(a \ || \ b) == (!a \ \&\& \ !b)$

! (reich *und* schön) == (arm *oder* hässlich)

Anwendung: Entweder ... Oder (XOR)

$(x \mid\mid y) \quad \&\& \neg(x \&\& y)$ x oder y, und nicht beide

$(x \mid\mid y) \quad \&\& (\neg x \mid\mid \neg y)$ x oder y, und eines nicht

$\neg(\neg x \&\& \neg y) \quad \&\& \neg(x \&\& y)$ nicht keines, und nicht beide

$\neg(\neg x \&\& \neg y \mid\mid x \&\& y)$ nicht: keines oder beide

Kurzschlussauswertung

- Logische Operatoren `&&` und `||` werten den *linken Operanden zuerst* aus.
- Falls das Ergebnis dann schon feststeht, wird der rechte Operand *nicht mehr* ausgewertet.

```
x != 0 && z / x > y
```

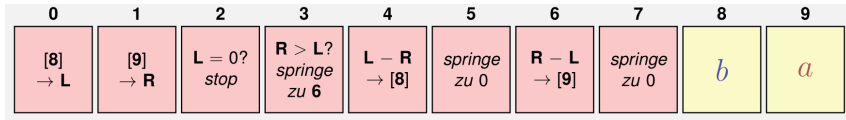
⇒ Keine Division durch 0

5. Kontrollanweisungen I

Auswahanweisungen, Iterationsanweisungen, Terminierung, Blöcke

Kontrollfluss

- Bisher: *linear* (von oben nach unten)
- Interessante Programme nutzen „Verzweigungen“ und „Sprünge“



Auswahanweisungen

realisieren Verzweigungen

- `if` Anweisung
- `if-else` Anweisung

if-Anweisung

```
if ( condition )  
    statement
```

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";
```

Ist *condition* wahr, dann wird *statement* ausgeführt.

- *statement*: beliebige Anweisung (*Rumpf* der if-Anweisung)
- *condition*: konvertierbar nach `bool`

if-else-Anweisung

```
if ( condition )  
    statement1  
else  
    statement2
```

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";  
else  
    std::cout << "odd";
```

Ist *condition* wahr, so wird *statement1* ausgeführt, andernfalls wird *statement2* ausgeführt.

- *condition*: konvertierbar nach bool.
- *statement1*: Rumpf des if-Zweiges
- *statement2*: Rumpf des else-Zweiges

Layout!

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";  
else  
    std::cout << "odd";
```



The diagram illustrates the indentation of the code. Two blue arrows point from the right side of the code to the word "Einrückung" (which means "indentation" in German). The first arrow points from the "even" string in the first branch, and the second arrow points from the "odd" string in the second branch. Both arrows originate from a blue rectangular box containing the word "Einrückung".

Iterationsanweisungen

realisieren Schleifen:

- `for`-Anweisung
- `while`-Anweisung
- `do`-Anweisung

Berechne $1 + 2 + \dots + n$

```
#include <iostream>

int main() {
    // input
    std::cout << "Compute the sum 1+...+n for n =? ";
    unsigned int n;
    std::cin >> n;
    // computation of sum_{i=1}^n i
    unsigned int s = 0;
    for (unsigned int i = 1; i <= n; ++i) s += i;
    // output
    std::cout << "1+...+" << n << " = " << s << ".\n";
    return 0;
}
```

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2$, $s == 0$

i		s
i==1	wahr	s == 1
i==2	wahr	s == 3
i==3	falsch	
		s == 3

Der kleine Gauß (1777 - 1855)



- Wie sie vermutlich wissen, gibt es einen effizienteren Weg, um die Summe der ersten n natürlichen Zahlen zu berechnen. Dazu folgende Anekdote:
- Mathe-Lehrer wollte seine Schüler mit folgender Aufgabe beschäftigen:

Berechne die Summe der Zahlen von 1 bis 100!

- Gauß war nach einer Minute fertig.

Die Lösung von Gauß



- Die gesuchte Zahl ist

$$1 + 2 + 3 + \cdots + 98 + 99 + 100.$$

- Das ist die Hälfte von

$$\begin{array}{rcccccccc} & 1 & + & 2 & + & \cdots & + & 99 & + & 100 \\ + & 100 & + & 99 & + & \cdots & + & 2 & + & 1 \\ \hline = & 101 & + & 101 & + & \cdots & + & 101 & + & 101 \end{array}$$

- Antwort: $100 \cdot 101/2 = 5050$

for-Anweisung: Syntax

```
for (init statement; condition; expression)  
    body statement
```

- *init statement*: Ausdrucksanweisung, Deklarationsanweisung, Nullanweisung
- *condition*: konvertierbar nach `bool`
- *expression*: beliebiger Ausdruck
- *body statement*: beliebige Anweisung (*Rumpf* der for-Anweisung)

for-Anweisung: Semantik

```
for ( init statement condition ; expression )  
    statement
```

- *init-statement* wird ausgeführt
 - *condition* wird ausgewertet
 - true: Iteration beginnt
 statement wird ausgeführt
 expression wird ausgeführt
 - falsch: for-Anweisung wird beendet.
- 

for-Anweisung: Terminierung

```
for (unsigned int i = 1; i <= n; ++i)  
    s += i;
```

Hier und meistens:

- *expression* ändert einen Wert, der in *condition* vorkommt.
- Nach endlich vielen Iterationen wird *condition* falsch: Terminierung.

Endlosschleifen

- Endlosschleifen sind leicht zu produzieren:

```
for ( ; ; ) ;
```

- Die *leere condition* ist wahr.
 - Die *leere expression* hat keinen Effekt.
 - Die *Nullanweisung* hat keinen Effekt.
- ... aber nicht automatisch zu erkennen.

```
for (init; cond; expr) stmt;
```

Halteproblem

Unentscheidbarkeit des Halteproblems

Es gibt kein C++ Programm, das für jedes C++- Programm P und jede Eingabe I korrekt feststellen kann, ob das Programm P bei Eingabe von I terminiert.

Das heisst, die Korrektheit von Programmen kann *nicht* automatisch überprüft werden.³

³Alan Turing, 1936. Theoretische Fragestellungen dieser Art waren für Alan Turing die Hauptmotivation für die Konstruktion seiner Rechenmaschine.

Beispiel: Primzahltest

Def.: Eine natürliche Zahl $n \geq 2$ ist eine Primzahl, wenn kein $d \in \{2, \dots, n-1\}$ ein Teiler von n ist.

Eine Schleife, die das testet:

```
unsigned int d;  
for (d=2; n%d != 0; ++d);
```

Primzahltest: Terminierung

```
unsigned int d;  
for (d=2; n%d != 0; ++d); // for n >= 2
```

- Fortschritt: Startwert $d=2$, dann in jeder Iteration plus 1 ($++d$)
- Abbruch: $n\%d \neq 0$ evaluiert zu `false` sobald ein Teiler erreicht wurde
— spätestens, wenn $d == n$
- Fortschritt garantiert, dass Abbruchbedingung erreicht wird

Primzahltest: Korrektheit

```
unsigned int d;  
for (d=2; n%d != 0; ++d); // for n >= 2
```

Jeder mögliche Teiler $2 \leq d \leq n$ wird ausprobiert. Falls die Schleife mit $d == n$ terminiert, dann und genau dann ist n prim.

Blöcke

- Blöcke gruppieren mehrere Anweisungen zu einer neuen Anweisung

```
{statement1 statement2 ... statementN}
```

- Beispiel: Rumpf der main Funktion

```
int main() {  
    ...  
}
```

- Beispiel: Schleifenrumpf

```
for (unsigned int i = 1; i <= n; ++i) {  
    s += i;  
    std::cout << "partial sum is " << s << "\n";  
}
```

6. Kontrollanweisungen II

Sichtbarkeit, Lokale Variablen, While-Anweisung, Do-Anweisung, Sprunganweisungen

Sichtbarkeit

Deklaration in einem Block ist ausserhalb des Blocks nicht *sichtbar*.

```
int main()
{
  {
    int i = 2;
  }
  std::cout << i; // Fehler: undeklariierter Name
  return 0;
}
```

main block

block

„Blickrichtung“

Kontrollanweisung definiert Block

Kontrollanweisungen verhalten sich in diesem Zusammenhang wie Blöcke.

```
int main()
{
    int s = 0;
    block | for (unsigned int i = 0; i < 10; ++i)
           s += i;
    std::cout << i; // Fehler: undeklariertes Name
    return 0;
}
```

Gültigkeitsbereich einer Deklaration

Potentieller Gültigkeitsbereich: Ab Deklaration bis Ende des Programnteils, der die Deklaration enthält.

Im Block

```
{  
    ...  
    int i = 2;  
    ...  
}
```

scope

Im Funktionsrumpf

```
int main() {  
    ...  
    int i = 2;  
    ...  
    return 0;  
}
```

scope

In Kontrollanweisung

```
for (int i = 0; i < 10; ++i) {s += i; ... }
```

scope

Gültigkeitsbereich einer Deklaration

Wirklicher Gültigkeitsbereich = Potentieller Gültigkeitsbereich minus darin enthaltene potentielle Gültigkeitsbereiche von Deklarationen des gleichen Namens

```
int main()
{
    int i = 2;
    for (int i = 0; i < 5; ++i)
        // outputs 0,1,2,3,4
        std::cout << i;
        // outputs 2
        std::cout << i;
    return 0;
}
```

Gültigkeit von i in main
i₂ in for

Lokale Variablen (Deklaration in Block)

- werden bei jedem Erreichen ihrer Deklaration neu „angelegt“, d.h.
 - Speicher / Adresse wird zugewiesen
 - evtl. Initialisierung wird ausgeführt
- werden am Ende ihrer deklarativen Region „abgebaut“ (Speicher wird freigegeben, Adresse wird ungültig)

Lokale Variablen

```
int main()
{
    int i = 5;
    for (int j = 0; j < 5; ++j) {
        std::cout << ++i; // outputs 6, 7, 8, 9, 10
        int k = 2;
        std::cout << --k; // outputs 1, 1, 1, 1, 1
    }
}
```

Lokale Variablen (Deklaration in einem Block) haben *automatische Speicherdauer*.

while Anweisung



```
while (condition)  
    statement
```

- *statement*: beliebige Anweisung, Rumpf der `while` Anweisung.
- *condition*: konvertierbar nach `bool`.

while Anweisung

```
while (condition)  
    statement
```

ist äquivalent zu

```
for (; condition; )  
    statement
```

while-Anweisung: Semantik



```
while (expression)  
    statement
```

- *condition* wird ausgewertet
 - true: Iteration beginnt
 statement wird ausgeführt
 - false: while-Anweisung wird beendet.
-

while-Anweisung: Warum?

- Bei `for`-Anweisung ist oft expression allein für den Fortschritt zuständig („Zählschleife“)

```
for (unsigned int i = 1; i <= n; ++i)
    s += i;
```

- Falls der Fortschritt nicht so einfach ist, kann `while` besser lesbar sein.

Beispiel: Die Collatz-Folge

$(n \in \mathbb{N})$

- $n_0 = n$
- $n_i = \begin{cases} \frac{n_{i-1}}{2} & , \text{ falls } n_{i-1} \text{ gerade} \\ 3n_{i-1} + 1 & , \text{ falls } n_{i-1} \text{ ungerade} \end{cases}, i \geq 1.$

n=5: 5 , 16 , 8 , 4 , 2 , 1 , 4 , 2 , 1 , ... (Repetition bei 1)

Die Collatz-Folge in C++

// Program collatz.cpp. Computes the Collatz sequence of a number n.

```
#include <iostream>
```

```
int main() {
```

```
    // Input
```

```
    std::cout << "Compute the Collatz sequence for n =? ";
```

```
    unsigned int n;
```

```
    std::cin >> n;
```

```
    // Iteration
```

```
    while (n > 1) {
```

```
        if (n % 2 == 0) n = n / 2;
```

```
        else n = 3 * n + 1;
```

```
        std::cout << n << " ";
```

```
    }
```

```
    std::cout << "\n";
```

Die Collatz-Folge in C++

n = 27:

82, 41, 124, 62, 31, 94, 47, 142, 71, 214, 107, 322, 161, 484, 242,
121, 364, 182, 91, 274, 137, 412, 206, 103, 310, 155, 466, 233, 700,
350, 175, 526, 263, 790, 395, 1186, 593, 1780, 890, 445, 1336, 668,
334, 167, 502, 251, 754, 377, 1132, 566, 283, 850, 425, 1276, 638,
319, 958, 479, 1438, 719, 2158, 1079, 3238, 1619, 4858, 2429, 7288,
3644, 1822, 911, 2734, 1367, 4102, 2051, 6154, 3077, 9232, 4616,
2308, 1154, 577, 1732, 866, 433, 1300, 650, 325, 976, 488, 244, 122,
61, 184, 92, 46, 23, 70, 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8,
4, 2, 1

Erscheint die 1 für jedes n ?

- Man vermutet es, aber niemand kann es beweisen!
- Falls nicht, so ist die `while`-Anweisung zur Berechnung der Collatz-Folge für einige n theoretisch eine Endlosschleife.

do-Anweisung



```
do  
    statement  
while (condition);
```

- *statement*: beliebige Anweisung, Rumpf der do Anweisung.
- *condition*: konvertierbar nach `bool`.

do-Anweisung

```
do  
    statement  
while (condition);
```

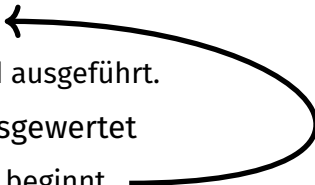
ist äquivalent zu

```
statement  
while (condition)  
    statement
```

do-Anweisung: Semantik



```
do  
  statement  
while (condition);
```

- Iteration beginnt 
 - *statement* wird ausgeführt.
- *condition* wird ausgewertet
 - true: Iteration beginnt
 - false: do-Anweisung wird beendet.

do-Anweisung: Beispiel Taschenrechner

Summiere ganze Zahlen (bei 0 ist Schluss):

```
int a;    // next input value
int s = 0; // sum of values so far
do {
    std::cout << "next number =? ";
    std::cin >> a;
    s += a;
    std::cout << "sum = " << s << "\n";
} while (a != 0);
```

- Auswahl (bedingte *Verzweigungen*)

- `if` und `if-else`-Anweisung

- Iteration (bedingte *Sprünge*)

- `for`-Anweisung
 - `while`-Anweisung
 - `do`-Anweisung

- Blöcke und Gültigkeit von Deklarationen

Sprunganweisungen

- `break;`
- `continue;`

```
break;
```

- umschliessende Iterationsanweisung wird sofort beendet
- nützlich, um Schleife „in der Mitte“ abbrechen zu können ⁴

⁴und unverzichtbar bei switch-Anweisungen

Taschenrechner mit break

Summiere ganze Zahlen (bei 0 ist Schluss):

```
int a;  
int s = 0;  
do {  
    std::cout << "next number =? ";  
    std::cin >> a;  
    s += a; /* irrelevant in last iteration */  
    std::cout << "sum = " << s << "\n";  
} while (a != 0);
```


Taschenrechner mit break

Unterdrücke irrelevante Addition von 0:

```
int a;
int s = 0;
do {
    std::cout << "next number =? ";
    std::cin >> a;
    if (a == 0) break; // Abbruch in der Mitte
    s += a;
    std::cout << "sum = " << s << "\n";
} while (a != 0)
```

Taschenrechner mit break

Äquivalent und noch etwas einfacher:

```
int a;  
int s = 0;  
while (true) {  
    std::cout << "next number =? ";  
    std::cin >> a;  
    if (a == 0) break; // Abbruch in der Mitte  
    s += a;  
    std::cout << "sum = " << s << "\n";  
}
```

Taschenrechner *ohne* break



Version ohne break wertet `a != 0` zweimal aus (und benötigt zusätzlichen Block).

```
int a = 1; // Init. mit nicht-null notwendig
int s = 0;
while (a != 0) {
    std::cout << "next number =? ";
    std::cin >> a;
    if (a != 0) {
        s += a;
        std::cout << "sum = " << s << "\n";
    }
}
```

```
continue;
```

- Kontrolle überspringt den Rest des Rumpfes der umschliessenden Iterationsanweisung
- Iterationsanweisung wird aber *nicht* abgebrochen

Taschenrechner mit `continue`

Ignoriere alle negativen Eingaben:

```
int a;
int s = 0;
while (true) {
    std::cout << "next number =? ";
    std::cin >> a;
    if (a < 0) continue; // springe zu }
    if (a == 0) break;
    s += a;
    std::cout << "sum = " << s << "\n";
}
```

break und continue in der Praxis

- Vorteil: Können verschachtelte `if-else`-Blöcke (oder komplexe Disjunktionen) vermeiden
- Aber führen zu mehr Sprüngen und somit zu potentiell komplexerem Kontrollfluss
- Ihr Einsatz ist daher umstritten und sollte mit Vorsicht geschehen

Äquivalenz von Iterationsanweisungen



Wir haben gesehen:

- `while` und `do` können mit Hilfe von `for` simuliert werden

Es gilt aber sogar:

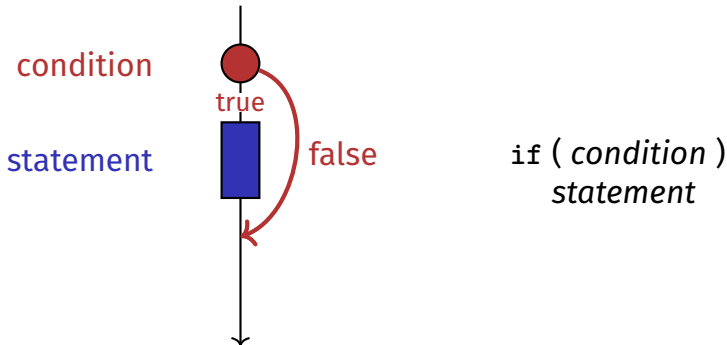
- Alle drei Iterationsanweisungen haben die gleiche „Ausdruckskraft“ (Skript)
- Nicht ganz so einfach falls ein `continue` im Spiel ist

- realisieren unbedingte Sprünge.
- sind wie `while` und `do` praktisch, aber nicht unverzichtbar
- sollten vorsichtig eingesetzt werden: nur dort wo sie den Kontrollfluss *vereinfachen*, statt ihn *komplizierter* zu machen

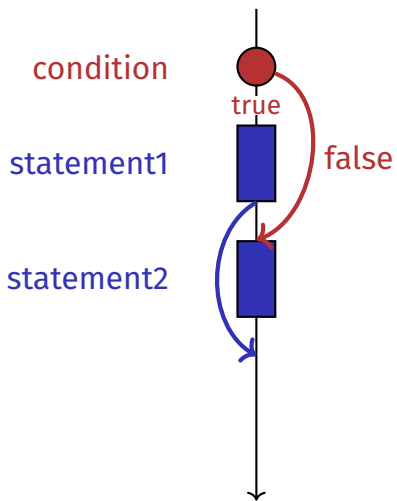
Kontrollfluss

Reihenfolge der (wiederholten) Ausführung von Anweisungen

- Grundsätzlich von oben nach unten...
- ...ausser in Auswahl- und Kontrollanweisungen



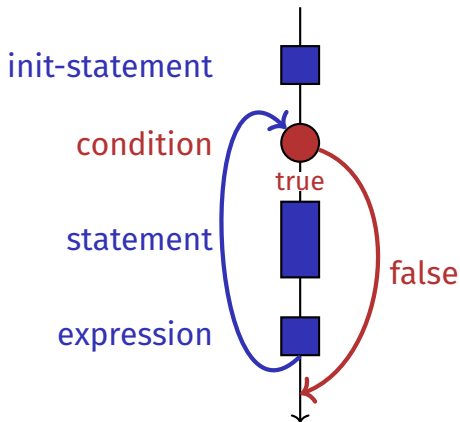
Kontrollfluss if else



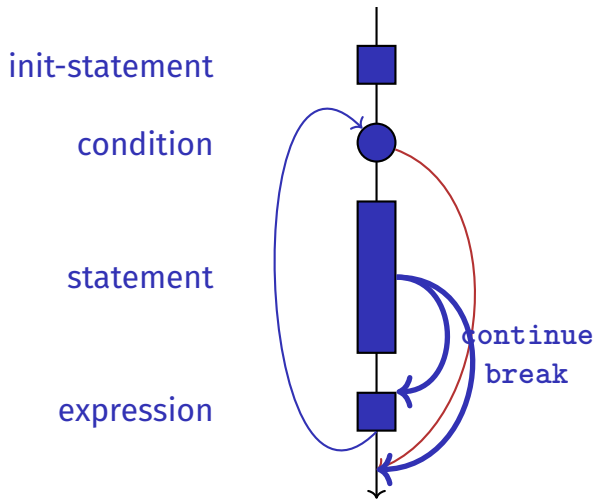
```
if ( condition )  
    statement1  
else  
    statement2
```

Kontrollfluss for

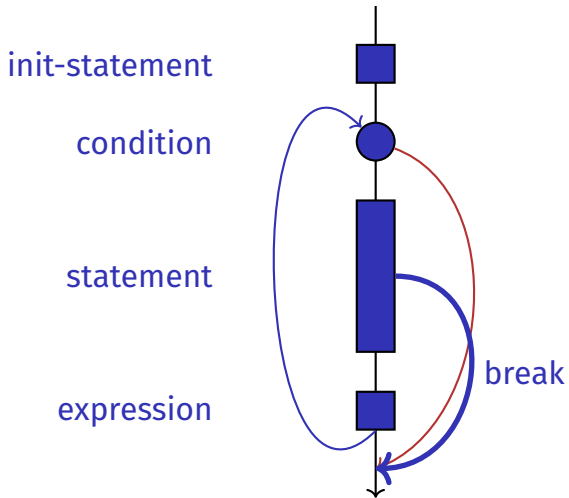
`for ( init statement ; condition ; expression )`
`statement`



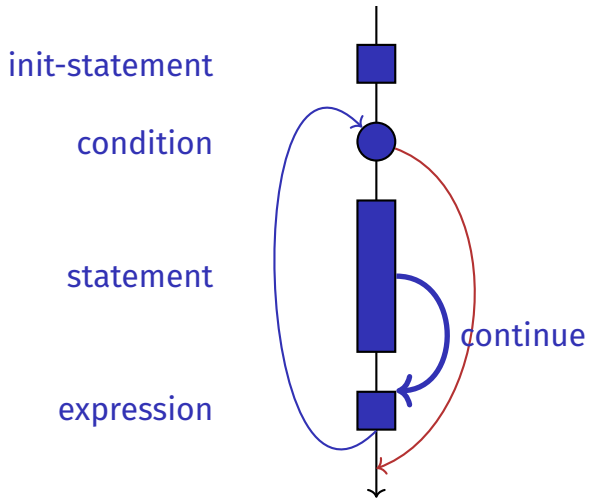
Kontrollfluss *break* und *continue* in for



Kontrollfluss break in for



Kontrollfluss `continue` in `for`



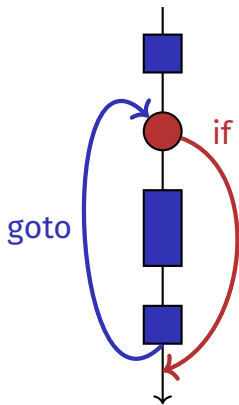
Kontrollfluss: Die guten alten Zeiten?

Beobachtung

Wir brauchen eigentlich nur `ifs` und Sprünge an beliebige Stellen im Programm (`goto`).

Sprachen, die darauf basieren:

- Maschinensprache
- Assembler („höhere“ Maschinensprache)
- BASIC, die erste Programmiersprache für ein allgemeines Publikum (1964)



BASIC und die Home-Computer...

...ermöglichten einer ganzen Generation von Jugendlichen das Programmieren.

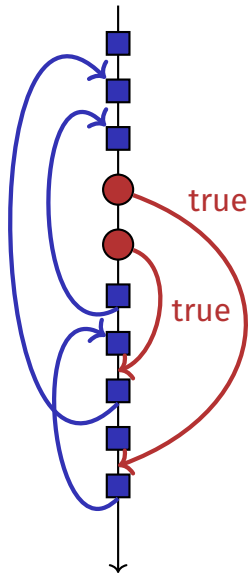


Home-Computer Commodore C64 (1982)

Spaghetti-Code mit goto

Ausgabe aller Primzahlen
mit der Programmiersprache BASIC:

```
10 N=2
20 D=1
30 D=D+1
40 IF N=D GOTO 100
50 IF N/D = INT(N/D) GOTO 70
60 GOTO 30
70 N=N+1
80 GOTO 20
100 PRINT N
110 GOTO 70
```



Die „richtige“ Iterationsanweisung

Ziele: Lesbarkeit, Prägnanz. Insbesondere

- Wenige Anweisungen
- Wenige Zeilen Code
- Einfacher Kontrollfluss
- Einfache Ausdrücke

Ziele sind oft nicht gleichzeitig erreichbar.

Ungerade Zahlen in $\{0, \dots, 100\}$

Erster (korrekter) Versuch:

```
for (unsigned int i = 0; i < 100; ++i) {  
    if (i % 2 == 0)  
        continue;  
    std::cout << i << "\n";  
}
```

Ungerade Zahlen in $\{0, \dots, 100\}$

Weniger Anweisungen, weniger Zeilen:

```
for (unsigned int i = 0; i < 100; ++i) {  
    if (i % 2 != 0)  
        std::cout << i << "\n";  
}
```

Ungerade Zahlen in $\{0, \dots, 100\}$

Weniger Anweisungen, einfacherer Kontrollfluss:

```
for (unsigned int i = 1; i < 100; i += 2)  
    std::cout << i << "\n";
```

Das ist hier die „richtige“ Iterationsanweisung

Notenausgabe

1. Funktionale Anforderung:

6 → "Excellent ... You passed!"

5,4 → "You passed!"

3 → "Close, but ... You failed!"

2,1 → "You failed!"

sonst → "Error!"

2. Ausserdem: Text- und Codeduplikation vermeiden

Notenausgabe mit if-Anweisungen

```
int grade;
...
if (grade == 6) std::cout << "Excellent ... ";
if (4 <= grade && grade <= 6) {
    std::cout << "You passed!";
} else if (1 <= grade && grade < 4) {
    if (grade == 3) std::cout << "Close, but ... ";
    std::cout << "You failed!";
} else std::cout << "Error!";
```

Nachteil: Kontrollfluss – und somit Programmverhalten – nicht gerade offensichtlich

Notenausgabe mit switch-Anweisung

```
switch (grade) {  
    case 6: std::cout << "Excellent ... ";  
    case 5:  
    case 4: std::cout << "You passed!";  
        break;  
    case 3: std::cout << "Close, but ... ";  
    case 2:  
    case 1: std::cout << "You failed!";  
        break;  
    default: std::cout << "Error!";  
}
```

Springe zu passendem case

Durchfallen

Verlasse switch

Durchfallen

Verlasse switch

In allen anderen Fällen

Vorteil: Kontrollfluss klar erkennbar

Die switch-Anweisung

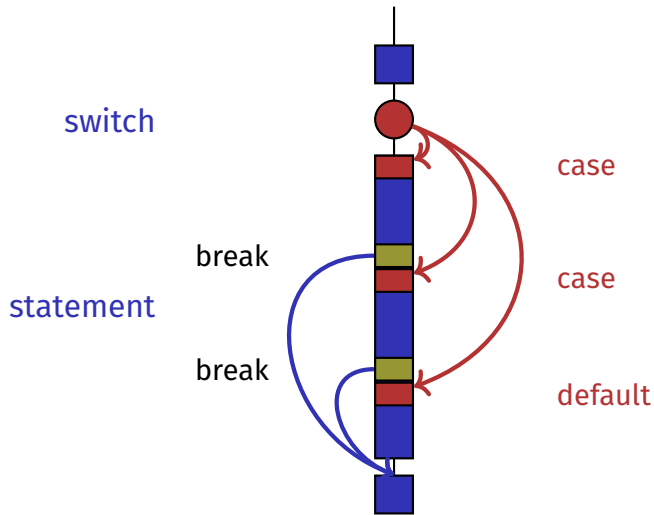
```
switch (expression)  
statement
```

- *expression*: Ausdruck, konvertierbar in einen integralen Typ
- *statement* : beliebige Anweisung, in welcher `case` und `default`-Marken erlaubt sind, `break` hat eine spezielle Bedeutung.
- Benutzung des Durchfallens in der Praxis umstritten, Einsatz gut abwägen (entsprechende Compilerwarnung kann aktiviert werden)

```
switch (expression)  
    statement
```

- `expression` wird ausgewertet.
- Beinhaltet `statement` eine `case`-Marke mit (konstantem) Wert von `condition`, wird dorthin gesprungen.
- Sonst wird, sofern vorhanden, an die `default`-Marke gesprungen. Wenn nicht vorhanden, wird `statement` übersprungen.
- Die `break`-Anweisung beendet die `switch`-Anweisung.

Kontrollfluss switch



7. Fließkommazahlen I

Typen **float** und **double**; Gemischte Ausdrücke und Konversionen; Lücken im Wertebereich

„Richtig“ Rechnen

```
// Program: fahrenheit_float.cpp
// Convert temperatures from Celsius to Fahrenheit.

#include <iostream>

int main() {
    // Input
    std::cout << "Temperature in degrees Celsius =? ";
    float celsius;
    std::cin >> celsius;

    // Computation and output
    std::cout << celsius << " degrees Celsius are "
              << 9 * celsius / 5 + 32 << " degrees Fahrenheit.\n";
    return 0;
}
```

Fixkommazahlen

- feste Anzahl Vorkommastellen (z.B. 7)
- feste Anzahl Nachkommastellen (z.B. 3)

0.0824 = 0000000.082 ← dritte Stelle abgeschnitten

Nachteile

- Wertebereich wird *noch* kleiner als bei ganzen Zahlen.
- Repräsentierbarkeit hängt von der Stelle des Kommas ab.

Fliesskommazahlen

- Beobachtung: Unterschiedlich „effiziente“ Darstellungen einer Zahl, z.B.

$$\begin{aligned}0.0824 &= 0.00824 \cdot 10^1 = 0.824 \cdot 10^{-1} \\ &= 8.24 \cdot 10^{-2} = 824 \cdot 10^{-4}\end{aligned}$$

Anzahl *signifikanter Stellen* bleibt konstant

- Fließkommarepräsentation daher:
 - Feste Anzahl signifikanter Stellen (z.B. 10),
 - Plus Position des Kommas mittels Exponenten
 - Zahl ist $\text{Signifikand} \times 10^{\text{Exponent}}$

Typen `float` und `double`

- sind die fundamentalen C++ Typen für Fließkommazahlen
- approximieren den Körper der reellen Zahlen $(\mathbb{R}, +, \times)$ in der Mathematik
- haben grossen Wertebereich, ausreichend für viele Anwendungen:
 - `float`: ca. 7 Stellen, Exponent bis ± 38
 - `double`: ca. 15 Stellen, Exponent bis ± 308
- sind auf den meisten Rechnern sehr schnell (Hardwareunterstützung)

Arithmetische Operatoren

Wie bei `int`, aber ...

- Divisionsoperator `/` modelliert „echte“ (reelle, nicht ganzzahlige) Division
- Kein Modulo-Operator, d.h. kein `%`

Literale

unterscheiden sich von Ganzzahlliteralen durch Angabe von

■ Dezimalkomma

`1.0` : Typ `double`, Wert 1

`1.27f` : Typ `float`, Wert 1.27

■ und / oder Exponent.

`1e3` : Typ `double`, Wert 1000

`1.23e-7` : Typ `double`, Wert $1.23 \cdot 10^{-7}$

`1.23e-7f` : Typ `float`, Wert $1.23 \cdot 10^{-7}$

1.23e-7f

ganzzahliger Teil

Exponent

fraktionaler Teil

Rechnen mit `float`: Beispiel

Approximation der Euler-Zahl

$$e = \sum_{i=0}^{\infty} \frac{1}{i!} \approx 2.71828 \dots$$

mittels der ersten 10 Terme.

Rechnen mit float: Eulersche Zahl

```
std::cout << "Approximating the Euler number... \n";

// values for i-th iteration, initialized for i = 0
float t = 1.0f; // term 1/i!
float e = 1.0f; // i-th approximation of e

// iteration 1, ..., n
for (unsigned int i = 1; i < 10; ++i) {
    t /= i;    // 1/(i-1)! -> 1/i!
    e += t;
    std::cout << "Value after term " << i << ": "
                << e << "\n";
}
```

Rechnen mit `float`: Eulersche Zahl

```
Value after term 1: 2
Value after term 2: 2.5
Value after term 3: 2.66667
Value after term 4: 2.70833
Value after term 5: 2.71667
Value after term 6: 2.71806
Value after term 7: 2.71825
Value after term 8: 2.71828
Value after term 9: 2.71828
```

Gemischte Ausdrücke, Konversion

- Fließkommazahlen sind allgemeiner als ganzzahlige Typen.
- In gemischten Ausdrücken werden ganze Zahlen zu Fließkommazahlen konvertiert.

```
9 * celsius / 5 + 32
```

Löcher im Wertebereich

```
float n1;  
std::cout << "First number =? ";  
std::cin >> n1;
```

Eingabe 1.1

```
float n2;  
std::cout << "Second number =? ";  
std::cin >> n2;
```

Eingabe 1.0

```
float d;  
std::cout << "Their difference =? ";  
std::cin >> d;
```

Eingabe 0.1

```
std::cout << "Computed difference - input difference = "  
          << n1 - n2 - d << "\n";
```

Ausgabe 2.23517e-8

Was ist denn hier los?

Wertebereich

Ganzzahlige Typen:

- Über- und Unterlauf häufig, aber ...
- Wertebereich ist zusammenhängend (keine Löcher): $\mathbb{Z}$ ist „diskret“.

Fliesskommatypen:

- Über- und Unterlauf selten, aber ...
- es gibt Löcher: $\mathbb{R}$ ist „kontinuierlich“.

8. Fließkommazahlen II

Fließkommazahlensysteme, IEEE Standard, Grenzen der Fließkommaarithmetik, Fließkomma-Richtlinien, Harmonische Zahlen

Fliesskommazahlensysteme

Ein Fliesskommazahlensystem ist durch vier natürliche Zahlen definiert:

- $\beta \geq 2$, die Basis,
- $p \geq 1$, die Präzision (Stellenzahl),
- $e_{\min}$, der kleinste Exponent,
- $e_{\max}$, der grösste Exponent.

Bezeichnung:

$$F(\beta, p, e_{\min}, e_{\max})$$

Fliesskommazahlensysteme

$F(\beta, p, e_{\min}, e_{\max})$ enthält die Zahlen

$$\pm \sum_{i=0}^{p-1} d_i \beta^{-i} \cdot \beta^e,$$

$$d_i \in \{0, \dots, \beta - 1\}, \quad e \in \{e_{\min}, \dots, e_{\max}\}.$$

In Basis- β -Darstellung:

$$\pm d_0 \bullet d_1 \dots d_{p-1} \times \beta^e$$

Fliesskommazahlensysteme

Darstellungen der Dezimalzahl 0.1 (mit $\beta = 10$):

$$1.0 \cdot 10^{-1}, \quad 0.1 \cdot 10^0, \quad 0.01 \cdot 10^1, \quad \dots$$

Unterschiedliche Darstellungsmöglichkeiten durch Wahl des Exponenten

Normalisierte Darstellung

Normalisierte Zahl:

$$\pm d_0 \bullet d_1 \dots d_{p-1} \times \beta^e, \quad d_0 \neq 0$$

Bemerkung 1

Die normalisierte Darstellung ist eindeutig und deshalb zu bevorzugen.

Bemerkung 2

Die Zahl 0, sowie alle Zahlen kleiner als $\beta^{e_{\min}}$, haben keine normalisierte Darstellung (greifen wir später wieder auf).

Menge der normalisierten Zahlen

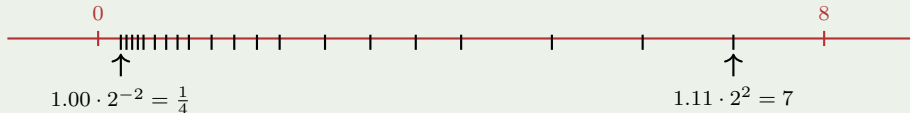
$$F^*(\beta, p, e_{\min}, e_{\max})$$

Normalisierte Darstellung

Beispiel $F^*(2, 3, -2, 2)$

(nur positive Zahlen)

$d_0 \bullet d_1 d_2$	$e = -2$	$e = -1$	$e = 0$	$e = 1$	$e = 2$
1.00_2	0.25	0.5	1	2	4
1.01_2	0.3125	0.625	1.25	2.5	5
1.10_2	0.375	0.75	1.5	3	6
1.11_2	0.4375	0.875	1.75	3.5	7



Binäre und dezimale Systeme

- Intern rechnet der Computer mit $\beta = 2$
(binäres System)
- Literale und Eingaben haben $\beta = 10$
(dezimales System)
- Eingaben müssen umgerechnet werden!

Umrechnung

$$(0 < x < 2)$$

Berechnung der *Binärdarstellung*:

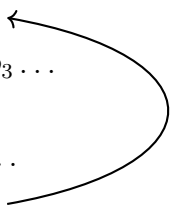
$$x = \sum_{i=0}^{\infty} b_i 2^{-i}$$

$$= b_0 \bullet b_1 b_2 b_3 \dots$$

$$= b_0 + 0 \bullet b_1 b_2 b_3 \dots$$

$\Rightarrow$

$$(x - b_0) = 0 \bullet b_1 b_2 b_3 b_4 \dots$$

$$2 \cdot (x - b_0) = b_1 \bullet b_2 b_3 b_4 \dots$$


```
for (int b_0; x != 0; x = 2 * (x - b_0)) {  
    if (x >= 1) b_0 = 1;  
    else b_0 = 0;  
    std::cout << b_0;  
}
```

Umrechnung dezimal $\rightarrow$ binär

Angenommen, $0 < x < 2$. Binärdarstellung:

$$\begin{aligned}x &= \sum_{i=-\infty}^0 b_i 2^i = b_0 \bullet b_{-1} b_{-2} b_{-3} \dots \\&= b_0 + \sum_{i=-\infty}^{-1} b_i 2^i = b_0 + \sum_{i=-\infty}^0 b_{i-1} 2^{i-1} \\&= b_0 + \underbrace{\left(\sum_{i=-\infty}^0 b_{i-1} 2^i \right)}_{x' = b_{-1} \bullet b_{-2} b_{-3} b_{-4}} / 2\end{aligned}$$

Umrechnung dezimal $\rightarrow$ binär



Angenommen, $0 < x < 2$.

■ Also: $x' = b_{-1}b_{-2}b_{-3}b_{-4}\dots = 2 \cdot (x - b_0)$

■ Schritt 1 (für x): Berechnen von b_0 :

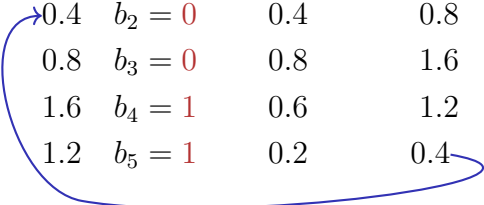
$$b_0 = \begin{cases} 1, & \text{falls } x \geq 1 \\ 0, & \text{sonst} \end{cases}$$

■ Schritt 2 (für x): Berechnen von $b_{-1}, b_{-2}, \dots$:

Gehe zu Schritt 1 (für $x' = 2 \cdot (x - b_0)$)

Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$	0.2	0.4
0.4	$b_2 = 0$	0.4	0.8
0.8	$b_3 = 0$	0.8	1.6
1.6	$b_4 = 1$	0.6	1.2
1.2	$b_5 = 1$	0.2	0.4



$\Rightarrow 1.\overline{00011}$, periodisch, *nicht* endlich

Binärdarstellungen von 1.1 und 0.1

- sind nicht endlich, also gibt es Fehler bei der Konversion in ein (endliches) binäres Fließkommazahlensystem.
- `1.1f` und `0.1f` sind nicht 1.1 und 0.1, sondern geringfügig fehlerhafte Approximationen dieser Zahlen.
- In `diff.cpp`: $1.1 - 1.0 \neq 0.1$

Binärdarstellungen von 1.1 und 0.1

auf meinem Computer:

$$\begin{aligned} 1.1 &= \underline{1.100000000000000000}888178\dots \\ 1.1f &= \underline{1.1000000}238418\dots \end{aligned}$$

Rechnen mit Fließkommazahlen

Beispiel ($\beta = 2, p = 4$):

$$\begin{array}{r} 1.111 \cdot 2^{-2} \\ + 1.011 \cdot 2^{-1} \\ \hline = 1.001 \cdot 2^0 \end{array}$$

1. Exponenten anpassen durch Denormalisieren einer Zahl 2. Binäre Addition der Signifikanden 3. Renormalisierung 4. Runden auf p signifikante Stellen, falls nötig

Der IEEE Standard 754

legt Fließkommazahlensysteme und deren Rundungsverhalten fest und wird fast überall benutzt

- Single precision (`float`) Zahlen:

$F^*(2, 24, -126, 127)$ (32 bit) plus 0, ∞ , ...

- Double precision (`double`) Zahlen:

$F^*(2, 53, -1022, 1023)$ (64 bit) plus 0, ∞ , ...

- Alle arithmetischen Operationen runden das *exakte* Ergebnis auf die nächste darstellbare Zahl

Warum

$$F^*(2, 24, -126, 127)?$$

- 1 Bit für das Vorzeichen
- 23 Bit für den Signifikanden (führendes Bit ist 1 und wird nicht gespeichert)
- 8 Bit für den Exponenten (256 mögliche Werte)(254 mögliche Exponenten, 2 Spezialwerte: 0, ∞ ,...)

⇒ insgesamt 32 Bit.

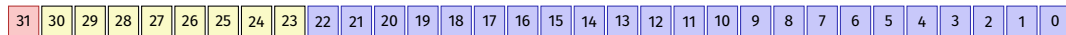
Warum

$$F^*(2, 53, -1022, 1023)?$$

- 1 Bit für das Vorzeichen
- 52 Bit für den Signifikanden (führendes Bit ist 1 und wird nicht gespeichert)
- 11 Bit für den Exponenten (2046 mögliche Exponenten, 2 Spezialwerte: 0, $\infty, \dots$)

⇒ insgesamt 64 Bit.

Beispiel: 32-bit Darstellung einer Fließkommazahl



± Exponent

Mantisse

± $2^{-126}, \dots, 2^{127}$
 $0, \infty, \dots$

1.000000000000000000000000
...
1.111111111111111111111111

Regel 1

Teste keine gerundeten Fließkommazahlen auf Gleichheit!

```
for (float i = 0.1; i != 1.0; i += 0.1)
    std::cout << i << "\n";
```

Endlosschleife, weil i niemals exakt 1 ist!

Regel 2

Addiere keine zwei Zahlen sehr unterschiedlicher Grösse!

$$\begin{aligned} & 1.000 \cdot 2^5 \\ & + 1.000 \cdot 2^0 \\ & = 1.00001 \cdot 2^5 \\ & \text{„}=\text{“ } 1.000 \cdot 2^5 \text{ (Rundung auf 4 Stellen)} \end{aligned}$$

Addition von 1 hat keinen Effekt!

- Die n -te Harmonische Zahl ist

$$H_n = \sum_{i=1}^n \frac{1}{i} \approx \ln n.$$

- Diese Summe kann vorwärts oder rückwärts berechnet werden, was mathematisch gesehen natürlich äquivalent ist.

Beobachtung:

- Die Vorwärtssumme wächst irgendwann nicht mehr und ist „richtig“ falsch.
- Die Rückwärtssumme approximiert H_n gut.

Erklärung:

- Bei $1 + 1/2 + 1/3 + \dots$ sind späte Terme zu klein, um noch beizutragen.
- Problematik wie bei $2^5 + 1$ „=“ 2^5

```
#include <iostream>
#include <iomanip> // for std::setprecision

int main() {
    unsigned int n = 0;

    while (true){
        std::cout << "n = ? ";
        std::cin >> n;

        std::cout << "Computing H_n for n = " << n << "\n";

        float h = 0;
        for (unsigned int i = 1; i <= n; ++i)
            h += 1.0 / i;

        std::cout << std::setprecision(100);
```


Ergebnisse:



Compute H_n for $n = ?$ 10000000
Forward sum = 15.4037
Backward sum = 16.686



Compute H_n for $n = ?$ 100000000
Forward sum = 15.4037
Backward sum = 18.8079

Regel 3

Subtrahiere keine zwei Zahlen sehr ähnlicher Grösse!

Auslöschungsproblematik, nicht vertieft.

Literatur

David Goldberg: What Every Computer Scientist Should Know About Floating-Point Arithmetic (1991)



Randy Glasbergen, 1996

9. Funktionen I

Funktionsdefinitionen- und Aufrufe, Auswertung von Funktionsaufrufen,
Der Typ **void**

- kapseln häufig gebrauchte Funktionalität (z.B. Potenzberechnung) und machen sie einfach verfügbar
- strukturieren das Programm: Unterteilung in kleine Teilaufgaben, jede davon durch eine Funktion realisiert

⇒ Prozedurales Programmieren; Prozedur: anderes Wort für Funktion.

Beispiel: Potenzberechnung

```
double a;  
int n;  
std::cin >> a; // Eingabe a  
std::cin >> n; // Eingabe n
```

```
double result = 1.0;  
if (n < 0) { //  $a^n = (1/a)^{-n}$   
    a = 1.0/a;  
    n = -n;  
}  
for (int i = 0; i < n; ++i)  
    result *= a;
```



"Funktion pow"

```
std::cout << a << "^" << n << " = " << result << ".\n";
```

Funktion zur Potenzberechnung

```
// assumes e >= 0 || b != 0.0
// returns the value b^e
double pow(double b, int e)
{
    double result = 1.0;
    if (e < 0) { // b^e = (1/b)^(-e)
        b = 1.0/b;
        e = -e;
    }
    for (int i = 0; i < e; ++i)
        result *= b;
    return result;
}
```

Funktion zur Potenzberechnung

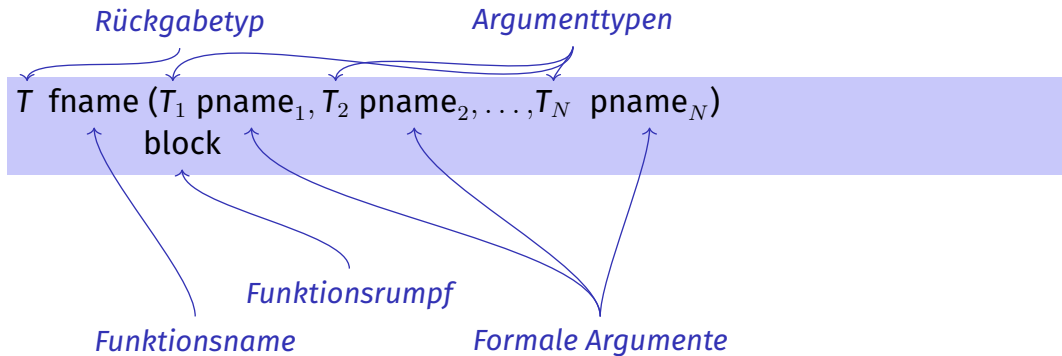
```
// Prog: callpow.cpp
// Define and call a function for computing powers.
#include <iostream>
```

```
double pow(double b, int e){...}
```

```
int main()
{
    std::cout << pow( 2.0, -2) << "\n"; // outputs 0.25
    std::cout << pow( 1.5, 2) << "\n"; // outputs 2.25
    std::cout << pow(-2.0, 9) << "\n"; // outputs -512

    return 0;
}
```


Funktionsdefinitionen



Funktionsdefinitionen

- dürfen nicht *lokal* auftreten, also nicht in Blocks, nicht in anderen Funktionen und nicht in Kontrollanweisungen
- können im Programm ohne Trennsymbole aufeinander folgen

```
double pow (double b, int e)
```

```
{
```

```
    ...
```

```
}
```

```
int main ()
```

```
{
```

```
    ...
```

```
}
```

Beispiel: Xor

```
// returns 1 XOR r
bool Xor(bool l, bool r)
{
    return l && !r || !l && r;
}
```

Beispiel: Harmonic

```
// assumes n >= 0
// returns nth harmonic number
// computed with backward sum
float Harmonic(int n)
{
    float res = 0;
    for (unsigned int i = n; i >= 1; --i)
        res += 1.0f / i;
    return res;
}
```

Beispiel: min

```
// returns the minimum of a and b
int min(int a, int b)
{
    if (a<b)
        return a;
    else
        return b;
}
```

Funktionsaufrufe

$fname (expression_1, expression_2, \dots, expression_N)$

- Alle Aufrufargumente müssen konvertierbar sein in die entsprechenden Argumenttypen.
- Der Funktionsaufruf selbst ist ein Ausdruck vom Rückgabetyt. Wert und Effekt wie in der Nachbedingung der Funktion *fname* angegeben.

Beispiel: `pow(a,n)`: Ausdruck vom Typ `double`

Funktionsaufrufe

Für die Typen, die wir bisher kennen, gilt:

- Aufrufargumente sind R-Werte
 $\hookrightarrow$ *call-by-value* (auch *pass-by-value*), dazu gleich mehr
- Funktionsaufruf selbst ist R-Wert.

$fname: \text{R-Wert} \times \text{R-Wert} \times \dots \times \text{R-Wert} \longrightarrow \text{R-Wert}$

- Auswertung der Aufrufargumente
- Initialisierung der formalen Argumente mit den resultierenden Werten
- Ausführung des Funktionsrumpfes: formale Argumente verhalten sich dabei wie lokale Variablen
- Ausführung endet mit `return expression;`



Rückgabewert ergibt den Wert des Funktionsaufrufes.

Beispiel: Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}  
  
...  
pow (2.0, -2)
```

Aufruf von pow

Rückgabe

^amanchmal *formale Parameter*

- Deklarative Region: Funktionsdefinition
- sind ausserhalb der Funktionsdefinition *nicht* sichtbar
- werden bei jedem Aufruf der Funktion neu angelegt (automatische Speicherdauer)
- Änderungen ihrer Werte haben keinen Einfluss auf die Werte der Aufrufargumente (Aufrufargumente sind R-Werte)

Gültigkeit formaler Argumente

```
double pow(double b, int e){  
    double r = 1.0;  
    if (e<0) {  
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        r * = b;  
    return r;  
}
```

```
int main(){  
    double b = 2.0;  
    int e = -2;  
    double z = pow(b, e);  
  
    std::cout << z; // 0.25  
    std::cout << b; // 2  
    std::cout << e; // -2  
    return 0;  
}
```

Nicht die formalen Argumente `b` und `e` von `pow`, sondern die hier definierten Variablen lokal zum Rumpf von `main`

Der Typ void

```
// writes "(i, j)" to standard output
void print_pair(int i, int j) {
    std::cout << "(" << i << ", " << j << ")\n";
}

int main() {
    print_pair(3,4); // outputs (3, 4)
    return 0;
}
```

Der Typ `void`

- Fundamentaler Typ mit leerem Wertebereich
- Verwendung als Rückgabetyt für Funktionen, die *nur* einen Effekt haben

void-Funktionen

- benötigen kein `return`.
- Ausführung endet, wenn
 - Ende des Funktionsrumpfes erreicht wird oder
 - `return`; erreicht wird
 - oder
 - `return expression`; erreicht wird.



Ausdruck vom Typ `void` (z.B. Aufruf einer Funktion mit Rückgabetyp `void`)

Funktionen und return

Das Verhalten einer Funktion mit Rückgabotyp ungleich `void` ist **undefiniert** wenn das Ende des Funktionsrumpfes ohne `return` Anweisung erreicht wird.

Falsch:

```
bool compare(float x, float y) {  
    float delta = x - y;  
    if (delta*delta < 0.001f) return true;  
}
```

Der Wert von `compare(10,20)` ist hier undefiniert

Funktionen und return

Das Verhalten einer Funktion mit Rückgabotyp ungleich `void` ist **undefiniert** wenn das Ende des Funktionsrumpfes ohne `return` Anweisung erreicht wird.

Besser:

```
bool compare(float x, float y) {  
    float delta = x - y;  
    if (delta*delta < 0.001f)  
        return true;  
    else  
        return false;  
}
```

Alle Ausführungspfade erreichen ein `return`

Funktionen und return

Das Verhalten einer Funktion mit Rückgabebetyp ungleich `void` ist **undefiniert** wenn das Ende des Funktionsrumpfes ohne `return` Anweisung erreicht wird.

Noch besser (weil einfacher)

```
bool compare(float x, float y) {  
    float delta = x - y;  
    return delta*delta < 0.001f;  
}
```

10. Funktionen II

Vor- und Nachbedingungen, Stepwise Refinement, Gültigkeitsbereich, Bibliotheken, Standardfunktionen

- beschreiben (möglichst vollständig) was die Funktion „macht“
- dokumentieren die Funktion für Benutzer / Programmierer (wir selbst oder andere)
- machen Programme lesbarer: wir müssen nicht verstehen, *wie* die Funktion es macht
- werden vom Compiler ignoriert
- Vor- und Nachbedingungen machen – unter der Annahme ihrer Korrektheit – Aussagen über die Korrektheit eines Programmes möglich.

Vorbedingungen

Vorbedingung (precondition):

- Was muss bei Funktionsaufruf gelten?
- Spezifiziert *Definitionsereich* der Funktion.

⇒ Wert 0^e ist für $e < 0$ undefiniert

```
// PRE: e >= 0 || b != 0.0
double pow(double b, int e){
    ...
}
```

Nachbedingungen

Nachbedingung (postcondition):

- Was gilt nach Funktionsaufruf?
- Spezifiziert *Wert* und *Effekt* des Funktionsaufrufes.

```
// PRE: e >= 0 || b != 0.0
// POST: return value is b^e
double pow(double b, int e){
    ...
}
```

Hier nur Wert, kein Effekt.

Vor- und Nachbedingungen

- sollten korrekt sein:
- *Wenn* die Vorbedingung beim Funktionsaufruf gilt, *dann* gilt auch die Nachbedingung nach dem Funktionsaufruf.

Funktion `pow`: funktioniert für alle Basen $b \neq 0$

- Gilt Vorbedingung beim Funktionsaufruf nicht, so machen wir keine Aussage.
- C++-Standard-Jargon: „Undefined behavior“.

Funktion `pow`: Division durch 0

- Vorbedingung sollte so schwach wie möglich sein (möglichst grosser Definitionsbereich)
- Nachbedingung sollte so stark wie möglich sein (möglichst detaillierte Aussage)

Fromme Lügen...

```
// PRE: e >= 0 || b != 0.0  
// POST: return value is  $b^e$ 
```

ist formal inkorrekt :

- Überlauf, falls e oder b zu gross sind
- b^e vielleicht nicht als double Wert darstellbar (Löcher im Wertebereich)

Fromme Lügen...sind erlaubt.

```
// PRE: e >= 0 || b != 0.0  
// POST: return value is be
```

Die exakten Vor- und Nachbedingungen sind plattformabhängig und meist sehr kompliziert. Wir abstrahieren und geben die mathematischen Bedingungen an.
⇒ Kompromiss zwischen formaler Korrektheit und lascher Praxis.

Prüfen von Vorbedingungen...

- Vorbedingungen sind nur Kommentare.
- Wie können wir *sicherstellen*, dass sie beim Funktionsaufruf gelten?

...mit Assertions

```
#include <cassert>

...
// PRE: e >= 0 || b != 0.0
// POST: return value is b^e
double pow(double b, int e) {
    assert (e >= 0 || b != 0);
    double result = 1.0;
    ...
}
```

Assertions: Syntax und Semantik

```
assert ( expression )
```

- *expression*: Ausdruck, dessen Wert nach `bool` konvertierbar ist.
- Semantik:
 - Falls *expression* == `true`: kein Effekt.
 - Falls *expression* == `false`: Programm wird mit entsprechender Fehlermeldung abgebrochen.
- benötigt `#include <cassert>`

Nachbedingungen mit Assertions

- Das Ergebnis „komplizierter“ Berechnungen ist oft einfach zu prüfen.
- Dann lohnt sich der Einsatz von assert für die Nachbedingung

```
// PRE: the discriminant p*p/4 - q is nonnegative
// POST: returns larger root of the polynomial x^2 + p x + q
double root(double p, double q)
{
    assert(p*p/4 >= q); // precondition
    double x1 = - p/2 + sqrt(p*p/4 - q);
    assert(equals(x1*x1+p*x1+q,0)); // postcondition
    return x1;
}
```

Assertions für den $ggT(x, y)$

Überprüfe, ob das Programm auf dem richtigen Weg ist ...

```
// Input x and y
std::cout << "x =? ";
std::cin >> x;
std::cout << "y =? ";
std::cin >> y;
```

Eingabe der Argumente für
die Berechnung

```
// Check validity of inputs
```

```
assert(x > 0 && y > 0);
```

← Vorbedingung für die weitere Berechnung

```
... // Compute gcd(x,y), store result in variable a
```

Assertions für den $ggT(x, y)$

... und hinterfrage das Offensichtliche! ...

...

`assert(x > 0 && y > 0);`  Vorbedingung für die weitere Berechnung

`... // Compute gcd(x,y), store result in variable a`

```
assert (a >= 1);  
assert (x % a == 0 && y % a == 0);  
for (int i = a+1; i <= x && i <= y; ++i)  
    assert(!(x % i == 0 && y % i == 0));
```

Verschiedene
Eigenschaften
des ggT
überprüfen

Fail-Fast mit Assertions

- Reale Software: viele C++-Dateien, komplexer Kontrollfluss
- Fehler machen sich erst spät(er) bemerkbar → Fehlersuche erschwert
- Assertions: Fehler frühzeitig bemerken



Assertions – Empfehlung

- Assertions sind ein wichtiges Werkzeug zur Fehlervermeidung während der Programmentwicklung, wenn man sie *konsequent* und *oft* einsetzt.
- Der Laufzeit-Overhead ist vernachlässigbar. Darüber hinaus kann der Compiler angewiesen werden, die Assertions „hinauszukompilieren“, also zu ignorieren.

Assertions abschalten

```
#define NDEBUG // To ignore assertions  
#include<cassert>
```

```
...
```

```
assert(x > 0 && y > 0); // Ignored
```

```
... // Compute gcd(x,y), store result in variable a
```

```
assert(a >= 1); // Ignored
```

```
...
```

Stepwise Refinement

Einfache Programmiertechnik zum Lösen komplexer Probleme

Niklaus Wirth. Program development by
stepwise refinement. Commun. ACM 14,
4, 1971

Education

P. Wegner
Editor

Program Development by Stepwise Refinement

Niklaus Wirth
Eidgenössische Technische Hochschule
Zürich, Switzerland

The creative activity of programming—to be distinguished from coding—is usually taught by examples serving to exhibit certain techniques. It is here considered as a sequence of design decisions concerning the decomposition of tasks into subtasks and of data into data structures. The process of successive refinement of specifications is illustrated by a short but nontrivial example, from which a number of conclusions are drawn regarding the art and the instruction of programming.

Key Words and Phrases: education in programming, programming techniques, stepwise program construction
CR Categories: 1.50, 4.0

1. Introduction

Programming is usually taught by examples. Experience shows that the success of a programming course critically depends on the choice of these examples. Unfortunately, they are too often selected with the prime intent to demonstrate what a computer can do. Instead, a main criterion for selection should be their suitability to exhibit certain widely applicable techniques. Furthermore, examples of programs are commonly presented as finished “products” followed by explanations of their purpose and their linguistic details. But active programming consists of the design of new programs, rather than contemplation of old programs. As a consequence of these teaching methods, the student obtains the impression that programming consists mainly of mastering a language (with all the peculiarities and intricacies so abundant in modern PL’s) and relying on one’s intuition to somehow transform ideas into finished programs. Clearly, programming courses should teach methods of design and construction, and the selected examples should be such that a gradual development can be nicely demonstrated.

This paper deals with a single example chosen with

these two purposes in mind. Some well-known techniques are briefly demonstrated and motivated (strategy of preselection, stepwise construction of trial solutions, introduction of auxiliary data, recursion), and the program is gradually developed in a sequence of refinement steps.

In each step, one or several instructions of the given program are decomposed into more detailed instructions. This successive decomposition or refinement of specifications terminates when all instructions are expressed in terms of an underlying computer or programming language, and must therefore be guided by the facilities available on that computer or language. The result of the execution of a program is expressed in terms of data, and it may be necessary to introduce further data for communication between the obtained subtasks or instructions. As tasks are refined, so the data may have to be refined, decomposed, or structured, and it is natural to refine program and data specifications in parallel.

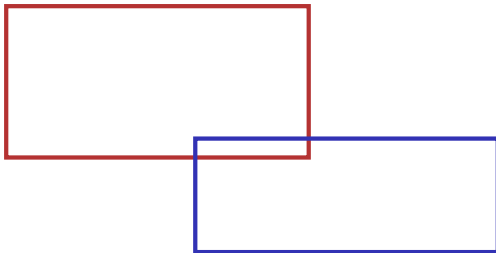
Every refinement step implies some design decisions. It is important that these decisions be made explicit, and that the programmer be aware of the underlying criteria and of the existence of alternative solutions. The possible solutions to a given problem emerge as the leaves of a tree, each node representing a point of deliberation and decision. Subtrees may be considered as families of solutions with certain common characteristics and structures. The notion of such a tree may be particularly helpful in the situation of changing purpose and environment to which a program may sometime have to be adapted.

A guideline in the process of stepwise refinement should be the principle to decompose decisions as much as possible, to untangle aspects which are only seemingly interdependent, and to defer those decisions which concern details of representation as long as possible. This

- Problem wird schrittweise gelöst. Man beginnt mit einer groben Lösung auf sehr hohem Abstraktionsniveau (nur Kommentare und fiktive Funktionen).
- In jedem Schritt werden Kommentare durch Programmtext ersetzt und Funktionen implementiert unterteilt (demselben Prinzip folgend).
- Die Verfeinerung bezieht sich auch auf die Entwicklung der Datenrepräsentation (mehr dazu später).
- Wird die Verfeinerung so weit wie möglich durch Funktionen realisiert, entstehen Teillösungen, die auch bei anderen Problemen eingesetzt werden können.
- Stepwise Refinement fördert (aber ersetzt nicht) das strukturelle Verständnis des Problems.

Beispielproblem

Finde heraus, ob zwei Rechtecke überlappen!



Grobe Lösung

(Include-Direktiven ausgelassen)

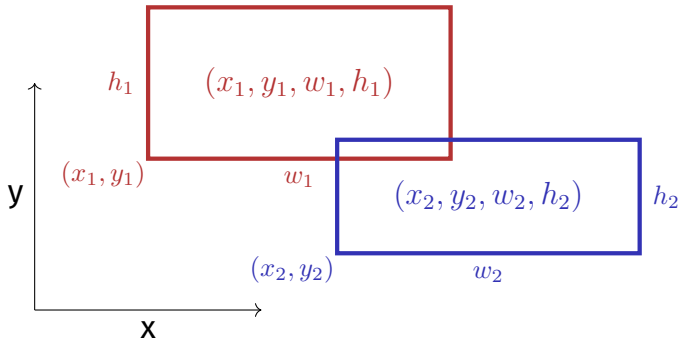
```
int main()
{
    // Eingabe Rechtecke

    // Überlappung?

    // Ausgabe der Loesung

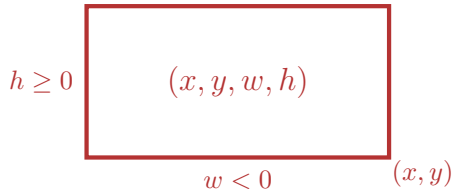
    return 0;
}
```

Verfeinerung 1: Eingabe Rechtecke



Verfeinerung 1: Eingabe Rechtecke

Breite w und/oder Höhe h dürfen negativ sein!



Verfeinerung 1: Eingabe Rechtecke

```
int main()
{
    std::cout << "Enter two rectangles [x y w h each] \n";
    int x1, y1, w1, h1;
    std::cin >> x1 >> y1 >> w1 >> h1;
    int x2, y2, w2, h2;
    std::cin >> x2 >> y2 >> w2 >> h2;

    // Ueberlappung?

    // Ausgabe der Loesung

    return 0;
}
```

Verfeinerung 2: Überlappung? und Ausgabe

```
int main()
```

```
{
```

Eingabe Rechtecke ✓

```
bool overlap = rectangles_overlap(x1,y1,w1,h1,x2,y2,w2,h2);
```

```
if (overlap)
```

```
    std::cout << "overlap!\n";
```

```
else
```

```
    std::cout << "no overlap!\n";
```

```
return 0;
```

```
}
```

Verfeinerung 3: Überlappungsfunktion...

```
bool rectangles_overlap(int x1, int y1, int w1, int h1,  
                        int x2, int y2, int w2, int h2)  
{  
    return false; // todo  
}
```

```
int main() {
```

Eingabe Rechtecke ✓

Überlappung? ✓

Ausgabe der Lösung ✓

```
    return 0;
```

```
}
```

Verfeinerung 3: Überlappungsfunktion...

```
bool rectangles_overlap(int x1, int y1, int w1, int h1,  
                        int x2, int y2, int w2, int h2)  
{  
    return false; // todo  
}
```

Funktion main ✓

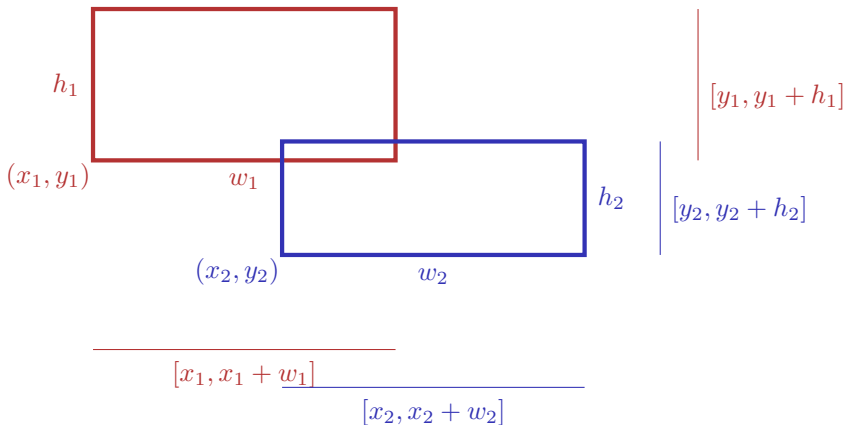
Verfeinerung 3:

...mit PRE und POST!

```
// PRE: (x1, y1, w1, h1), (x2, y2, w2, h2) are rectangles,  
//       where w1, h1, w2, h2 may be negative.  
// POST: returns true if (x1, y1, w1, h1) and  
//       (x2, y2, w2, h2) overlap  
bool rectangles_overlap(int x1, int y1, int w1, int h1,  
                        int x2, int y2, int w2, int h2)  
{  
    return false; // todo  
}
```

Verfeinerung 4: Intervallüberlappung

Zwei Rechtecke überlappen genau dann, wenn ihre x - **und** y -Intervalle überlappen.



Verfeinerung 4: Intervallüberlappung

```
// PRE: (x1, y1, w1, h1), (x2, y2, w2, h2) are rectangles, where
//       w1, h1, w2, h2 may be negative.
// POST: returns true if (x1, y1, w1, h1), (x2, y2, w2, h2) overlap
bool rectangles_overlap(int x1, int y1, int w1, int h1,
                       int x2, int y2, int w2, int h2)
{
    return intervals_overlap(x1, x1 + w1, x2, x2 + w2)
        && intervals_overlap(y1, y1 + h1, y2, y2 + h2); ✓
}
```


Verfeinerung 4: Intervallüberlappung

```
// PRE: [a1, b1], [a2, b2] are (generalized) intervals,  
//       with [a,b] := [b,a] if a>b  
// POST: returns true if [a1, b1],[a2, b2] overlap  
bool intervals_overlap(int a1, int b1, int a2, int b2)  
{  
    return false; // todo  
}
```

Funktion rectangles_overlap ✓

Funktion main ✓

Verfeinerung 5: Min und Max

```
// PRE: [a1, b1], [a2, b2] are (generalized) intervals,  
//       with [a,b] := [b,a] if a>b  
// POST: returns true if [a1, b1],[a2, b2] overlap  
bool intervals_overlap(int a1, int b1, int a2, int b2)  
{  
    return max(a1, b1) >= min(a2, b2)  
        && min(a1, b1) <= max(a2, b2); ✓  
}
```

Verfeinerung 5: Min und Max

// POST: the maximum of x and y is returned

```
int max(int x, int y){  
    if (x>y) return x; else return y;  
}
```

gibt es schon in der Standardbibliothek

// POST: the minimum of x and y is returned

```
int min(int x, int y){  
    if (x<y) return x; else return y;  
}
```

Funktion intervals_ioverlap ✓

Funktion rectangles_overlap ✓

Funktion main ✓

Nochmal zurück zu Intervallen

```
// PRE: [a1, b1], [a2, h2] are (generalized) intervals,  
//       with [a,b] := [b,a] if a>b  
// POST: returns true if [a1, b1],[a2, b2] overlap  
bool intervals_overlap(int a1, int b1, int a2, int b2)  
{  
    return std::max(a1, b1) >= std::min(a2, b2)  
        && std::min(a1, b1) <= std::max(a2, b2); ✓  
}
```

Das haben wir schrittweise erreicht!

```
#include <iostream>
#include <algorithm>

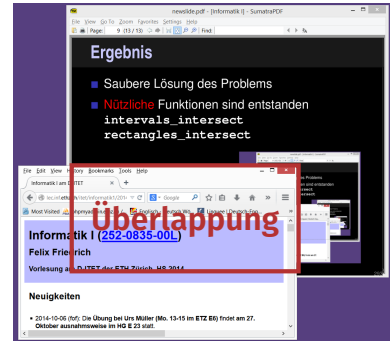
// PRE: [a1, b1], [a2, h2] are (generalized) intervals,
//      with [a,b] := [b,a] if a>b
// POST: returns true if [a1, b1],[a2, b2] overlap
bool intervals_overlap(int a1, int b1, int a2, int b2)
{
    return std::max(a1, b1) >= std::min(a2, b2)
        && std::min(a1, b1) <= std::max(a2, b2);
}

// PRE: (x1, y1, w1, h1), (x2, y2, w2, h2) are rectangles, where
//      w1, h1, w2, h2 may be negative.
// POST: returns true if (x1, y1, w1, h1),(x2, y2, w2, h2) overlap
bool rectangles_overlap(int x1, int y1, int w1, int h1,
                        int x2, int y2, int w2, int h2)
{
    return intervals_overlap(x1, x1 + w1, x2, x2 + w2)
        && intervals_overlap(y1, y1 + h1, y2, y2 + h2);
}
```

```
int main ()
{
    std::cout << "Enter two rectangles [x y w h] each\n";
    int x1, y1, w1, h1;
    std::cin >> x1 >> y1 >> w1 >> h1;
    int x2, y2, w2, h2;
    std::cin >> x2 >> y2 >> w2 >> h2;
    bool clash = rectangles_overlap(x1,y1,w1,h1,x2,y2,w2,h2);
    if (clash)
        std::cout << "overlap!\n";
    else
        std::cout << "no overlap!\n";
    return 0;
}
```

Ergebnis

- Saubere Lösung des Problems
- **Nützliche** Funktionen sind entstanden
intervals_overlap
rectangles_overlap



Wo darf man eine Funktion benutzen?

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    std::cout << f(1); // Fehler: f undeklariert
```

```
    return 0;
```

```
}
```

Gültigkeit f

```
int f(int i) // Gültigkeitsbereich von f ab hier
{
    return i;
}
```



Gültigkeitsbereich einer Funktion

- ist der Teil des Programmes, in dem die Funktion aufgerufen werden kann
- ist definiert als die Vereinigung der Gültigkeitsbereiche aller ihrer Deklarationen (es kann mehrere geben)

Deklaration einer Funktion: wie Definition aber ohne {...}.

```
double pow(double b, int e);
```


So geht's also nicht...

```
#include <iostream>
```

```
int main()
{
    std::cout << f(1); // Fehler: f undeklariert
    return 0;
}
```

Gültigkeit f

```
int f(int i) // Gültigkeitsbereich von f ab hier
{
    return i;
}
```



...aber so!

```
#include <iostream>

int f(int i); // Gultigkeitsbereich von f ab hier

int main()
{
    std::cout << f(1);
    return 0;
}

int f(int i)
{
    return i;
}
```

Forward Declarations, wozu?

Funktionen, die sich gegenseitig aufrufen:

```
int g(...); // forward declaration

int f(...) // f ab hier gültig
{
    g(...) // ok
}

int g(...)
{
    f(...) // ok
}
```

Wiederverwendbarkeit

- Funktionen wie `rectangles_overlap` und `pow` sind in vielen Programmen nützlich.
- „Lösung:“ Funktion einfach ins Hauptprogramm hineinkopieren, wenn wir sie brauchen!
- Hauptnachteil: wenn wir die Funktionsdefinition ändern wollen, müssen wir alle Programme ändern, in denen sie vorkommt.

Level 1: Auslagern der Funktion

```
// PRE: e >= 0 || b != 0.0
// POST: return value is b^e
double pow(double b, int e)
{
    double result = 1.0;
    if (e < 0) { // b^e = (1/b)^(-e)
        b = 1.0/b;
        e = -e;
    }
    for (int i = 0; i < e; ++i)
        result *= b;
    return result;
}
```

Level 1: Inkludieren der Funktion

```
// Prog: callpow2.cpp  
// Call a function for computing powers.
```

```
#include <iostream>
```

```
#include "mymath.cpp" ← Datei im Arbeitsverzeichnis
```

```
int main()  
{  
    std::cout << pow( 2.0, -2) << "\n";  
    std::cout << pow( 1.5, 2) << "\n";  
    std::cout << pow( 5.0, 1) << "\n";  
    std::cout << pow(-2.0, 9) << "\n";  
  
    return 0;  
}
```

Nachteil des Inkludierens

- `#include` kopiert die Datei (`mymath.cpp`) in das Hauptprogramm (`callpow2.cpp`).
- Der Compiler muss die Funktionsdefinition für jedes Programm neu übersetzen.
- Das kann bei sehr vielen und grossen Funktionen sehr lange dauern.

Level 2: Getrennte Übersetzung

von `mymath.cpp` unabhängig vom Hauptprogramm:

```
double pow(double b,  
           int e)  
{  
    ...  
}
```

`mymath.cpp`

`g++ -c mymath.cpp`

001110101100101010
000101110101000111
000101110101000111
Funktion pow
111100001101010001
111111101000111010
010101101011010001
100101111100101010

`mymath.o`

Level 2: Getrennte Übersetzung

Deklaration aller benötigten Symbole in sog. Header Datei.

```
// PRE: e >= 0 || b != 0.0  
// POST: return value is b^e  
double pow(double b, int e);
```

mymath.h

Level 2: Getrennte Übersetzung

des Hauptprogramms unabhängig von `mymath.cpp`, wenn eine *Deklaration* aus `mymath` inkludiert wird.

```
#include <iostream>
#include "mymath.h"
int main()
{
    std::cout << pow(2,-2) << "\n";
    return 0;
}
```

callpow3.cpp



```
001110101100101010
000101110101000111
000101110101000111
Funktion main
111100001101010001
010101101011010001
101010110101010101
rufe pow auf!
111111101000111010
```

callpow3.o

Der Linker vereint...

```
001110101100101010
000101110101000111
000101110101000111
111100001101010001
111111101000111010
010101101011010001
100101111100101010
```

mymath.o

+

```
001110101100101010
000101110101000111
000101110101000111
111100001101010001
010101101011010001
100101111100101010
111111101000111010
```

callpow3.o

... was zusammengehört

```
001110101100101010
000101110101000111
000101 Funktion pow
111100001101010001
111111101000111010
010101101011010001
100101111100101010
```

mymath.o

+

```
001110101100101010
000101110101000111
000101 Funktion main
111100001101010001
010101101011010001
100101111100101010
111111101000111010
rufe pow auf!
```

callpow3.o

=

```
001110101100101010
000101110101000111
000101 Funktion pow
111100001101010001
111111101000111010
010101101011010001
100101111100101010
001110101100101010
000101110101000111
000101 Funktion main
111100001101010001
010101101011010001
100101111100101010
111111101000111010
rufe addr auf!
```

Ausführbare Datei callpow3

- Alle Quellcodes sind verfügbar.
- Nur das erlaubt die Weiterentwicklung durch Benutzer und engagierte „Hacker“.
- Selbst im kommerziellen Bereich ist Open-Source-Software auf dem Vormarsch.
- Lizenzen erzwingen die Nennung der Quellen und die offene Weiterentwicklung. Beispiel: GPL (GNU General Public License).
- Bekannte Open-Source-Softwares: Linux (Betriebssystem), Firefox (Browser), Thunderbird (Email-Programm)

Verfügbarkeit von Quellcode?

Beobachtung

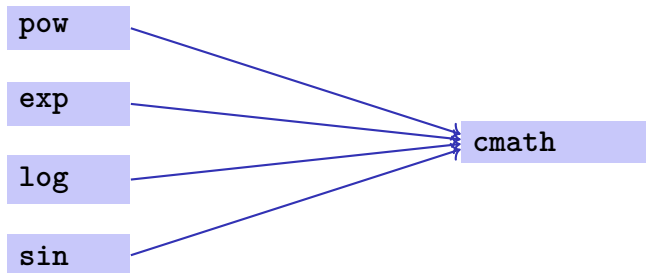
`mymath.cpp` (Quellcode) wird nach dem Erzeugen von `mymath.o` (Object Code) nicht mehr gebraucht.

Viele Anbieter von Funktionsbibliotheken liefern dem Benutzer keinen Quellcode.

Header-Dateien sind dann die *einzigsten* lesbaren Informationen.

Bibliotheken

■ Logische Gruppierung ähnlicher Funktionen



Namensräume...

```
// cmath
namespace std {

    double pow(double b, int e);

    ....
    double exp(double x);
    ...
}

// informatik
namespace informatik {
    double pow(double b, int e);
}
```


...vermeiden Namenskonflikte

```
#include <cmath>
#include "mymath.h"

int main()
{
    double x = std::pow(2.0, -2); // <cmath>
    double y = informatik::pow(2.0, -2); // mymath.h
}
```

In C++ ist das Konzept der separaten Kompilation *unabhängig* vom Konzept der Namensräume.

In manchen anderen Sprachen, z.B. Modula / Oberon (zum Teil auch bei Java) definiert die Kompilationseinheit gerade einen Namensraum.

Funktionen aus der Standardbibliothek

- vermeiden die Neuerfindung des Rades (wie bei `std::pow`);
- führen auf einfache Weise zu interessanten und effizienten Programmen;
- garantieren einen Qualitäts-Standard, der mit selbstgeschriebenen Funktionen kaum erreicht werden kann.
- Übersicht auf <https://cppreference.com>

Beispiel: Primzahltest mit sqrt

$n \geq 2$ ist Primzahl genau dann, wenn kein d in $\{2, \dots, n-1\}$ ein Teiler von n ist.

```
unsigned int d;  
for (d=2; n % d != 0; ++d);
```

Primzahltest mit sqrt

$n \geq 2$ ist Primzahl genau dann, wenn kein d in $\{2, \dots, \lfloor \sqrt{n} \rfloor\}$ ein Teiler von n ist.

```
unsigned int bound = std::sqrt(n);  
unsigned int d;  
for (d = 2; d <= bound && n % d != 0; ++d);
```

- Das funktioniert, weil `std::sqrt` auf die nächste darstellbare double-Zahl rundet (IEEE Standard 754).

Primzahltest mit sqrt

```
// Test if a given natural number is prime.
#include <iostream>
#include <cassert>
#include <cmath>

int main ()
{
    // Input
    unsigned int n;
    std::cout << "Test if n>1 is prime for n =? ";
    std::cin >> n;
    assert (n > 1);
```

Primzahltest mit sqrt Fortsetzung

```
// Computation: test possible divisors d up to sqrt(n)
unsigned int bound = std::sqrt(n);
unsigned int d;
for (d = 2; d <= bound && n % d != 0; ++d);

// Output
if (d <= bound)
    // d is a divisor of n in {2,...,[sqrt(n)]}
    std::cout << n << " = " << d << " * " << n / d << ".\n";
else
    // no proper divisor found
    std::cout << n << " is prime.\n";

return 0;
}
```

Funktionen sollten mehr können!

Swap ?

```
void swap(int x, int y) {  
    int t = x;  
    x = y;  
    y = t;  
}  
  
int main(){  
    int a = 2;  
    int b = 1;  
    swap(a, b);  
    assert(a==1 && b==2); // fail! 😞  
}
```


Funktionen sollten mehr können!

Swap ?

// POST: values of x and y are exchanged

```
void swap(int& x, int& y) {
```

```
    int t = x;
```

```
    x = y;
```

```
    y = t;
```

```
}
```

```
int main(){
```

```
    int a = 2;
```

```
    int b = 1;
```

```
    swap(a, b);
```

```
    assert(a==1 && b==2); // ok!
```



```
}
```

Sneak Preview: Referenztypen

- Wir können Funktionen in die Lage versetzen, die Werte ihrer Aufrufargumente zu ändern!
- Kein neues Konzept auf der Funktionenseite, sondern eine neue Klasse von Typen



Referenztypen (z.B. `int&`)

11. Vektoren I

Vektoren, Sieb des Eratosthenes, Speicherlayout, Iteration

Vektoren: Motivation

- Wir können jetzt über Zahlen iterieren

```
for (int i=0; i<n; ++i) {...}
```

- Oft muss man aber über *Daten* iterieren (Beispiel: Finde ein Kino in Zürich, das heute „C++ Runner 2049“ zeigt)
- Vektoren dienen zum Speichern *gleichartiger* Daten (Beispiel: Spielpläne aller Zürcher Kinos)

Vektoren: erste Anwendung

Das Sieb des Erathostenes

- berechnet alle Primzahlen $< n$
- Methode: Ausstreichen der Nicht-Primzahlen

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	--------------	---	--------------	---	--------------	--------------	---------------	----	---------------	----	---------------	---------------	---------------	----	---------------	----	---------------	---------------	---------------	----

Am Ende des Streichungsprozesses bleiben nur die Primzahlen übrig.

- Frage: wie streichen wir Zahlen aus?
- Antwort: mit einem *Vektor*.

Sieb des Eratosthenes mit Vektoren

```
#include <iostream>
#include <vector> // standard containers with vector functionality
int main() {
    // input
    std::cout << "Compute prime numbers in {2,...,n-1} for n =? ";
    unsigned int n; std::cin >> n;

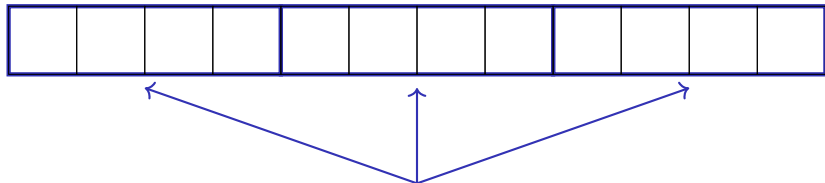
    // definition and initialization: provides us with Booleans
    // crossed_out[0],..., crossed_out[n-1], initialized to false
    std::vector<bool> crossed_out = std::vector<bool>(n, false);

    // computation and output
    std::cout << "Prime numbers in {2,...," << n-1 << "}: \n";
    for (unsigned int i = 2; i < n; ++i)
        if (!crossed_out[i]) { // i is prime
            std::cout << i << " ";
            // cross out all proper multiples of i
            for (unsigned int m = 2*i; m < n; m += i) crossed_out[m] = true;
        }
    std::cout << "\n";
    return 0;
}
```

Speicherlayout eines Vektors

Ein Vektor belegt einen *zusammenhängenden* Speicherbereich

Beispiel: Ein Vektor mit 3 Elementen vom Type T



Speichersegmente für jeweils einen Wert vom Typ T
(T belegt z.B. 4 Bytes)

Wahlfreier Zugriff (Random Access)

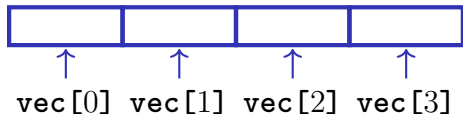
Gegeben

- Vektor `vec` mit `T`-Elementen
- `int`-Ausdruck `exp` mit Wert $i \geq 0$

Dann ist der Ausdruck

`vec [ exp ]`

- ein `L`-Wert vom Type `T`
- der sich auf das i -te Element von `vec` bezieht (Zählung ab 0!)



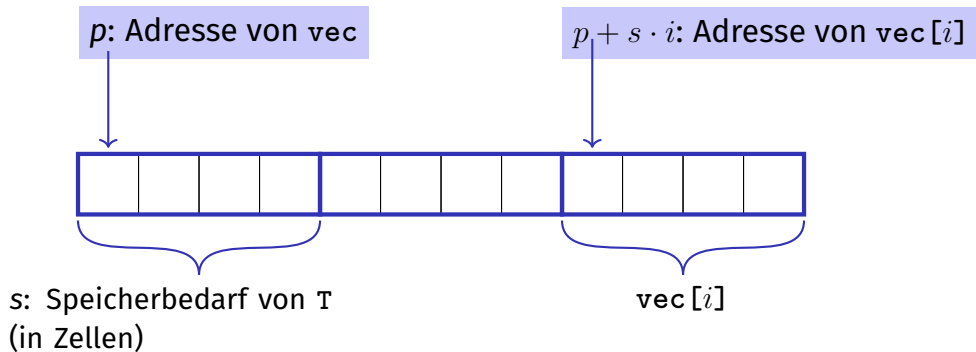
Wahlfreier Zugriff (Random Access)

`vec [ exp ]`

- Der Wert i von `exp` heisst *Index*
- `[]` ist der *Index-Operator* (auch *Subskript-Operator*)

Wahlfreier Zugriff (Random Access)

Wahlfreier Zugriff ist sehr effizient:



Vektorinitialisierung

- `vector<int> fivezeros = vector<int>(5);`
Die fünf Elemente von `fivezeros` werden mit 0 initialisiert
- `vector<int> fivetwos = vector<int>(5, 2);`
Die fünf Elemente von `fivetwos` werden mit 2 initialisiert
- `vector<int> countdown = {5, 4, 3, 2, 1};`
Der Vektor wird direkt mit einer Liste an Werten initialisiert
- `vector<int> empty;`
Deklariert *und initialisiert* einen leeren Vektor

Namensraumpräfix `std::` aus Platzgründen weggelassen

Achtung

Der Zugriff auf Elemente ausserhalb der gültigen Grenzen eines Vektors führt zu *undefiniertem Verhalten*

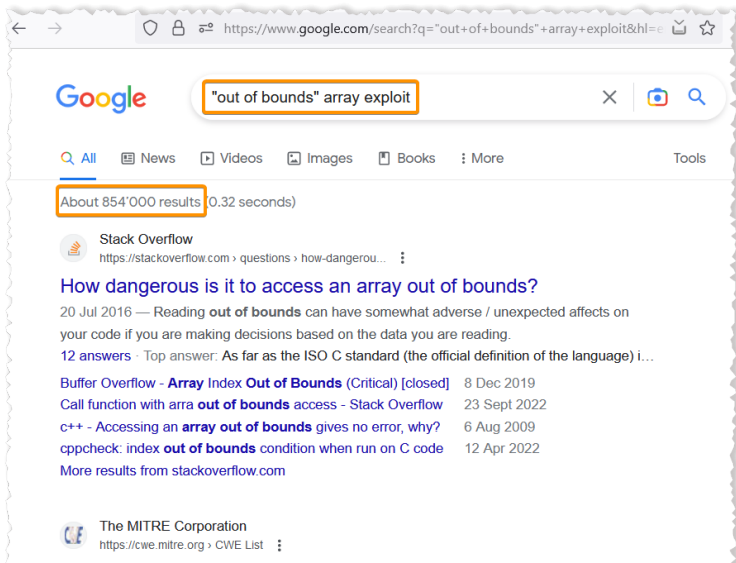
```
std::vector<int> vec = std::vector<int>(10);  
for (unsigned int i = 0; i <= 10; ++i)  
    vec[i] = 30; // undefiniertes Verhalten: Zugriff auf vec[10]
```

Achtung

Prüfung der Indexgrenzen

Bei Verwendung des Indexoperators auf einem Vektor ist es die alleinige Verantwortung des Programmierers, die Gültigkeit aller Elementzugriffe zu prüfen.

Konsequenzen illegaler Index-Zugriffe



Konsequenzen illegaler Index-Zugriffe

The screenshot shows the Rapid7 Vulnerability & Exploit Database interface. The search bar contains the text "buffer overflow". The results are displayed as a list of vulnerabilities, each with a title, publication date, severity, and an "EXPLORE" button. The results are filtered to show items 01 to 20 of a total of 21,562.

Results 01 - 20 of 21,562 in total

Vulnerability Title	Published	Severity	Action
Google Chrome Vulnerability: CVE-2023-1222 Heap buffer overflow in Web Audio API	March 08, 2023	4	EXPLORE
Google Chrome Vulnerability: CVE-2023-1217 Stack buffer overflow in Crash reporting	March 08, 2023	4	EXPLORE
Microsoft Edge Chromium : CVE-2023-1220 Heap buffer overflow in UMA	March 07, 2023	4	EXPLORE
SUSE : CVE-2023-1220: SUSE Linux Security Advisory	March 07, 2023	4	EXPLORE
FreeBSD : VID-D357F6BB-0AF4-4AC9-B096-EEEC183AD829 (CVE-2023-1220): chromium -- multiple vulnerabilities	March 07, 2023	4	EXPLORE
Debian : CVE-2023-1220: chromium -- security update			EXPLORE

Vektoren bieten viel Funktionalität

Hier ein paar Beispielfunktionen, weitere folgen später in der Vorlesung.

```
std::vector<int> v = std::vector<int>(10);  
std::cout << v.at(10);  
    // Zugriff mit Index-Check → Laufzeitfehler  
    // Ideal für Hausaufgaben
```

```
v.push_back(-1); // -1 is appended (added at end)  
std::cout << v.size(); // outputs 11  
std::cout << v.at(10); // outputs -1
```


12. Referenztypen

Referenztypen: Definition und Initialisierung, Pass by Value, Pass by Reference, temporäre Objekte, Const-Referenzen

Swap!

// POST: values of x and y have been exchanged

```
void swap(int& x, int& y) {  
    int t = x;  
    x = y;  
    y = t;  
}
```

```
int main() {  
    int a = 2;  
    int b = 1;  
    swap(a, b);  
    assert(a == 1 && b == 2); // ok! 😊  
}
```

Referenztypen

- Wir können Funktionen in die Lage versetzen, die Werte ihrer Aufrufargumente zu ändern
- Kein neues Konzept auf der Funktionenseite, sondern eine neue Klasse von Typen: *Referenztypen*

Referenztypen: Definition



- $T\&$ hat den gleichen Wertebereich und gleiche Funktionalität wie T ...
- ...aber Initialisierung und Zuweisung funktionieren anders

Anakin Skywalker alias Darth Vader

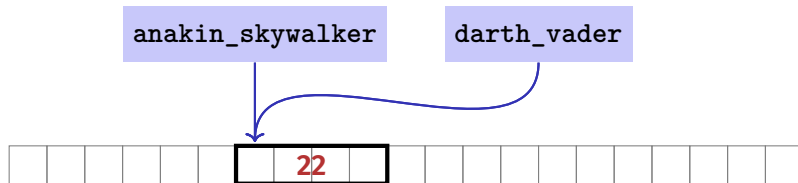


Anakin Skywalker alias Darth Vader

```
int anakin_skywalker = 9;  
int& darth_vader = anakin_skywalker; // Alias  
darth_vader = 22;
```

Zuweisung an den L-Wert hinter dem Alias

```
std::cout << anakin_skywalker; // 22
```



Referenztypen: Initialisierung & Zuweisung

```
int& darth_vader = anakin_skywalker;  
darth_vader = 22; // Effekt: anakin_skywalker = 22
```

- Eine Variable mit **Referenztyp** (eine *Referenz*) muss mit einem **L-Wert** initialisiert werden
- Die Variable wird dabei ein *Alias* des **L-Werts** (ein anderer Name für das referenzierte Objekt)
- Zuweisung an die Referenz erfolgt an das Objekt *hinter* dem Alias

Referenztypen: Realisierung

Intern wird ein Wert vom Typ $T\&$ durch die Adresse eines Objekts vom Typ T repräsentiert.

```
int& j; // Fehler: j muss Alias von irgendetwas sein
```

```
int& k = 5; // Fehler: Literal 5 hat keine Adresse
```

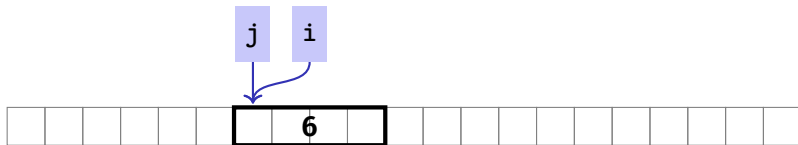

Pass by Reference

Referenztypen erlauben Funktionen, die Werte ihrer Aufrufargumente zu ändern

```
void increment(int& i) {  
    ++i;  
}
```

Initialisierung der formalen Argumente: i
wird Alias des Aufrufarguments j

```
...  
int j = 5;  
increment(j);  
std::cout << j; // 6
```



Pass by Reference

Formales Argument *hat* Referenztyp:

⇒ *Pass by Reference*

Formales Argument wird (intern) mit der *Adresse* des Aufrufarguments (L-Wert) initialisiert und wird damit zu einem *Alias*.

Pass by Value

Formales Argument *hat keinen* Referenztyp:

⇒ *Pass by Value*

Formales Argument wird mit dem *Wert* des Aufrufarguments (R-Wert) initialisiert und wird damit zu einer *Kopie*.

Return by Reference (Beispiel 1)

```
int increment(int& i) {  
    return ++i;  
}
```

- Wir möchten wiederholte Anwendung: `increment(increment(x))`
Compiler Error
- Funktion muss L-Wert zurückgeben, gibt aber R-Wert zurück

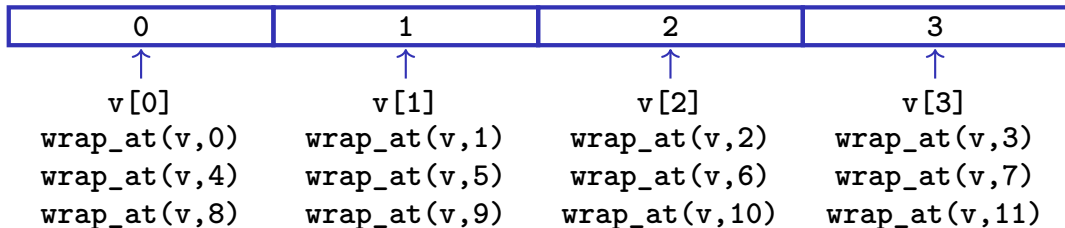
```
int& inc(int& i) {  
    return ++i;  
}
```

- Auch der Rückgabetyt ist Referenztyp: *Return by Reference*
- Funktionsaufruf *selbst* ist nun ein L-Wert
- Aufruf `inc(x)` hat exakt die Semantik des Prä-Inkrement `++x`

Return by Reference (Beispiel 2)

```
std::vector<int> v = {0, 1, 2, 3};
```

Ziel: Indexieren auf Vektor v mit Wrap-Around beim Index $i \geq 4$.



$\text{wrap_at}(v, i)$ greift auf Vektor v bei Index $i \% v.size()$ zu

wrap_at-Funktion: Erster Versuch

```
int wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

```
std::vector<int> v = {0, 1, 2, 3};  
std::cout << wrap_at(v, 2); // outputs 2  
std::cout << wrap_at(v, 6); // outputs 2
```

```
wrap_at(v, 6) = 6; // compiler error
```

Compilerfehler: lvalue required as left operand of assignment

⇒ wrap_at gibt einen R-Wert zurück, nicht einen L-Wert

wrap_at-Funktion

```
int& wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

```
std::vector<int> v = {0, 1, 2, 3};  
std::cout << wrap_at(v, 2); // outputs 2  
std::cout << wrap_at(v, 6); // outputs 2
```

```
wrap_at(v, 6) = 6;  
std::cout << wrap_at(v, 2); // outputs 6
```

Return by Reference: Rückgabe von L-Wert

Temporäre Objekte

Was ist hier falsch?

```
int& foo(int i) {  
    return i;   
}
```

Rückgabewert vom Typ `int&` wird
Alias des formalen Arguments
(lokale Variable `i`), dessen Speicher-
dauer aber nach Auswertung des
Funktionsaufrufes endet

```
int k = 3;  
int& j = foo(k); // j ist Alias einer "Leiche"  
std::cout << j; // undefined behavior
```


Die Referenz-Richtlinie

Referenz-Richtlinie

Wenn man eine Referenz erzeugt, muss das Objekt, auf das sie verweist, mindestens so lange „leben“ wie die Referenz selbst.

Return by Reference – Warum?

- Nutzen von *Pass by Reference*:

- Kopieren von Daten beim Aufruf vermeiden
- Effekt auf Argumente/Aufruferseite gewünscht:
`swap(...)`

- Nutzen von *Return by Reference*
wenn Rückgabewert L-Wert sein soll

- Vektoren:
`vec[i] = x`
`wrap_at(vec, i) = x`
- mehr Anwendungen später im Kapitel zu dynamischen Datenstrukturen

Durchschnitt: Erste Versuche

- Funktion, die den Durchschnitt eines Vektors berechnet

```
double average(std::vector<double>& values){  
    double sum = 0;  
    for (unsigned int i = 0; i < values.size(); ++i)  
        sum += values[i];  
    return sum / values.size();  
}
```

pass by reference, da Kopieren von Vektor vermieden werden soll!

- Aufruf

```
const std::vector<double> temperatures =  
{0.9,1.3,5.1,9.1,13.2,17.1,18.8,18.6,14.7,10.6,5.2,1.8};  
std::cout << average(temperatures); // compiler error
```

const-Vektor, da die Werte nicht verändert werden (sollen)

Durchschnitt: Lösung mit Pass by Const Reference

```
double average(const std::vector<double>& values){  
    double sum = 0;  
    for (unsigned int i = 0; i < values.size(); ++i)  
        sum += values[i];  
    return sum / values.size();  
}
```

```
const std::vector<double> temperatures =  
{0.9, 1.3, 5.1, 9.1, 13.2, 17.1, 18.8, 18.6, 14.7, 10.6, 5.2, 1.8};  
std::cout << average(temperatures); // outputs 9.7
```

```
std::vector<double> numbers = {2.0, 3.0, 4.0};  
std::cout << average(numbers); // outputs 3.0
```

```
void compute(std::vector<double>& v){ ... }
```

```
const std::vector<int> vec = std::vector<double>(5, 0.0);  
compute(vec); // Compiler error: const-qualification discarded
```

- Ohne Kenntnis der Implementation von `compute` können wir nicht ausschliessen, dass diese den Wert von `v` verändert
- Beim Funktionsaufruf würde Parameter `v` ein Alias des Arguments `vec`, dessen Wert deshalb möglicherweise geändert würde
- Der Compiler stellt mit der Fehlermeldung sicher, dass ein konstanter L-Wert nicht über einen nicht-konstanten Alias zugänglich gemacht wird

Const und Aliasing

```
void compute1(const std::vector<double>& x){ ... } // x ist const
void compute2(std::vector<double>& y){ ... } // y veränderlich
```

```
const std::vector<double> v1 = std::vector<double>(5, 0.0);
std::vector<double> v2 = std::vector<double>(5, 0.0);

compute1(v1); // OK: x wird Lese-Alias von v1
compute1(v2); // OK: x wird Lese-Alias von v2
compute2(v1); // NOT OK: y wird Schreib-Alias von const v1
compute2(v2); // OK: y wird Schreib-Alias von v2
```

- Ein nicht-konstanter L-Wert darf über Lese- und Schreib-Alias zugänglich gemacht werden
- Existenz von Alias ändert die const-ness des (original) L-Werts nicht

Const-Referenzen

- haben Typ `const T &`
- Typ kann verstanden werden als „`(const T) &`“

```
const T& r = lvalue;
```

`r` wird mit der Adresse von `lvalue` initialisiert (effizient)

- können auch mit R-Werten initialisiert werden (Compiler erzeugt temporäres Objekt ausreichender Lebensdauer)

```
const T& r = rvalue;
```

`r` wird mit der Adresse eines temporären Objektes vom Wert des `rvalue` initialisiert (pragmatisch)

Wann `const T&` verwenden?

```
void f_1(T& arg);
```

```
void f_2(const T& arg);
```

- Argumenttypen sind Referenzen; Aufrufargumente werden daher nicht kopiert, was effizient ist
- Aber nur `f_2` „verspricht“ Argument nicht zu verändern

Regel

Funktionsargumenttypen, die von nicht fundamentalem Typ sind als `const T&` (*pass by read-only reference*) deklarieren, falls möglich: effizient *und* sicher.

Lohnt sich i.d.R. nicht für fundamentale Typen (`int`, `double`, ...).

13. Zeichen und Texte

Zeichen, ASCII, UTF-8, Caesar-Code, Texte als Strings, String-Operationen

Zeichen und Texte

- Texte haben wir schon gesehen:

```
std::cout << "Prime numbers in {2,...,999}:\n";
```

String-Literal

- Können wir auch „richtig“ mit Texten arbeiten? Ja!

Zeichen: Wert des fundamentalen Typs `char`

Text: `std::string` $\approx$ Vektor von `char` Elementen

Der Typ char („character“)

Repräsentiert druckbare Zeichen (z.B. 'a') und *Steuerzeichen* (z.B. '\n')

```
char c = 'a';
```

Deklariert und initialisiert
Variable c vom Typ char
mit Wert 'a'

Literal vom Typ char

Der Typ `char` („character“)

Ist formal ein ganzzahliger Typ

- Werte konvertierbar nach `int` / `unsigned int`
- Alle arithmetischen Operatoren verfügbar (Nutzen zweifelhaft: was ist `'a'` / `'b'` ?)
- Werte belegen meistens 8 Bit

Wertebereich:

$\{-128, \dots, 127\}$ oder $\{0, \dots, 255\}$

Der ASCII-Code

- Definiert konkrete Konversionsregeln `char` $\longrightarrow$ (unsigned) `int`

Zeichen $\longrightarrow \{0, \dots, 127\}$

'A', 'B', ... , 'Z' $\longrightarrow$ 65, 66, ..., 90

'a', 'b', ... , 'z' $\longrightarrow$ 97, 98, ..., 122

'0', '1', ... , '9' $\longrightarrow$ 48, 49, ..., 57

- Wird von allen gängigen Computersystemen unterstützt
- Erlaubt Arithmetik über Zeichen

```
for (char c = 'a'; c <= 'z'; ++c)
    std::cout << c; // abcdefghijklmnopqrstuvwxyz
```

Erweiterung von ASCII: Unicode, UTF-8

- Internationalisierung von Software $\Rightarrow$ grosse Zeichensätze nötig. Heute daher üblich:
 - Zeichensatz *Unicode*: 150 Schriftsysteme, ca. 137'000 Zeichen
 - Codierungsstandard *UTF-8*: Abbildung Zeichen $\leftrightarrow$ Zahlen
- UTF-8 ist eine *Multibyte-Codierung*: Häufig genutzte Zeichen (z.B. lat. Alphabet) belegen nur ein Byte, andere Zeichen bis zu vier
- Länge einer Zeichen-Bytefolge wird dabei durch Bitmuster codiert

Nutzbare Bits	Bitmuster
7	0xxxxxxx
11	110xxxxx 10xxxxxx
16	1110xxxx 10xxxxxx 10xxxxxx
21	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Einige Unicode-Zeichen in UTF-8

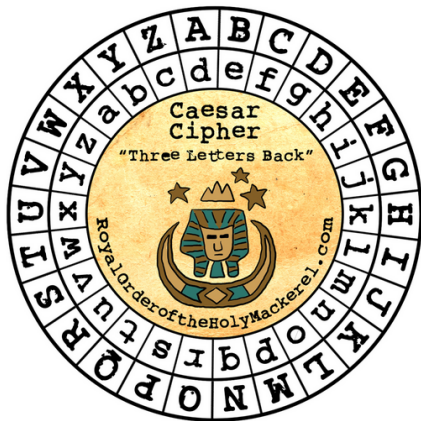
Symbol	Codierung (jeweils 16 Bit)
	11101111 10101111 10111001
	11100010 10011000 10100000
	11100010 10011000 10000011
	11100010 10011000 10011001
A	01000001

P.S.: Suchen Sie mal nach apple "unicode of death" P.S.: Unicode & UTF-8 sind nicht klausurrelevant

Caesar-Code

Ersetze jedes druckbare Zeichen in einem Text durch seinen Vor-Vor-Vorgänger.

' '	(32)	→	' '	(124)
'!'	(33)	→	'}''	(125)
	...			
'D'	(68)	→	'A'	(65)
'E'	(69)	→	'B'	(66)
	...			
~	(126)	→	'{'	(123)



Caesar-Code:

shift-Funktion

```
// PRE:  divisor > 0
// POST: return the remainder of dividend / divisor
//        with 0 <= result < divisor
int mod(int dividend, int divisor);

// POST: if c is one of the 95 printable ASCII characters, c is
//        cyclically shifted s printable characters to the right
char shift(char c, int s) {
    if (c >= 32 && c <= 126) { // c is printable
        c = 32 + mod(c - 32 + s, 95);
    }

    return c;
}
```

"- 32" transforms interval [32, 126] to

[0, 94]

"mod(x, 95)" computes $x \bmod 95$ in [0, 94]

"32 +" transforms [0, 94] back to [32, 126]

Caesar-Code:

caesar-Funktion

```
// POST: Each character read from std::cin was shifted cyclically
//       by s characters and afterwards written to std::cout
void caesar(int s) {
    std::cin >> std::noskipws; // #include <ios>

    char next;
    while (std::cin >> next) {
        std::cout << shift(next, s);
    }
}
```

Konversion nach bool: liefert *false*
genau dann, wenn die Eingabe leer ist

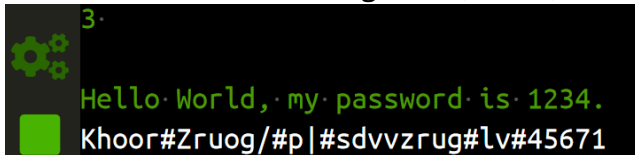
Verschiebung druckbarer Zeichen um s

Caesar-Code:

Hauptprogramm

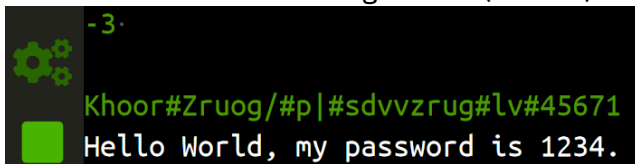
```
int main() {  
    int s;  
    std::cin >> s;  
  
    // Shift input by s  
    caesar(s);  
  
    return 0;  
}
```

Verschlüsseln: Verschiebung um n (hier: 3)



A terminal window with a dark background. The prompt is a green gear icon. The user enters '3'. The prompt is a green square icon. The user enters 'Hello World, my password is 1234.'. The output is 'Khoor#Zruog/#p|#sdvvzrug#lv#45671'.

Entschlüsseln: Verschiebung um $-n$ (hier: -3)



A terminal window with a dark background. The prompt is a green gear icon. The user enters '-3'. The prompt is a green square icon. The user enters 'Khoor#Zruog/#p|#sdvvzrug#lv#45671'. The output is 'Hello World, my password is 1234.'.

Texte

- Text „Sein oder nicht sein“ könnte als `vector<char>` repräsentiert werden
- Texte sind jedoch allgegenwärtig, daher existiert in der Standardbibliothek ein eigener Typ für sie: `std::string` (Zeichenkette)
- Benutzung benötigt `#include <string>`

Benutzung von `std::string`

- Deklaration und Initialisierung mittels Literal:

```
std::string text = "Essen ist fertig!"
```

- Mit variabler Länge initialisieren:

```
std::string text = std::string(n, 'a')
```

text wird mit n 'a's gefüllt

- Texte vergleichen:

```
if (text1 == text2) ...
```

true wenn zeichenweise gleich

Benutzung von `std::string`

■ Grösse auslesen:

```
for (unsigned int i = 0; i < text.size(); ++i) ...
```

Grösse ungleich Textlänge für Multibytekodierungen, z.B. UTF-8

■ Einzelne Zeichen lesen:

```
if (text[0] == 'a') ... // or text.at(0)
```

`text[0]` prüft Indexgrenzen nicht, `text.at(0)` hingegen schon

■ Einzelne Zeichen schreiben:

```
text[0] = 'b'; // or text.at(0)
```

Benutzung von `std::string`

- Strings konkatenieren (zusammensetzen):

```
text = ":-";  
text += ")";  
assert(text == ":-)");
```

- Viele weitere Operationen, bei Interesse siehe <https://en.cppreference.com/w/cpp/string>

14. Vektoren II

Mehrdimensionale Vektoren/Vektoren von Vektoren

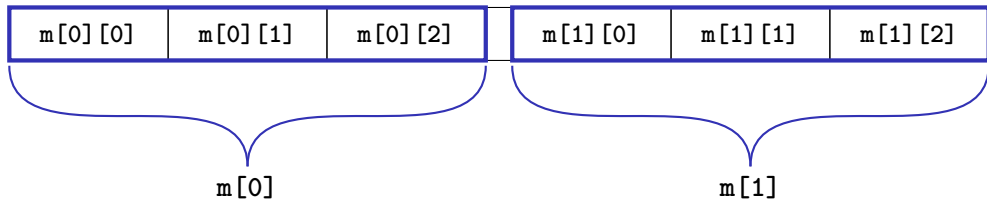
Mehrdimensionale Vektoren

- Zum Speichern von mehrdimensionalen Strukturen wie Tabellen, Matrizen, ...
- ...können *Vektoren von Vektoren* verwendet werden:

```
std::vector<std::vector<int>>> m; // An empty matrix
```

Mehrdimensionale Vektoren

Im Speicher: flach



Im Kopf: Matrix

Spalten			
	0	1	2
0	m[0][0]	m[0][1]	m[0][2]
1	m[1][0]	m[1][1]	m[1][2]

Zeilen

Mehrdimensionale Vektoren: Initialisierung

Mittels Initialisierungslisten:

```
// the 4-by-4 identity matrix
std::vector<std::vector<std::int>> matrix = {
    {1, 0, 0, 0},
    {0, 1, 0, 0},
    {0, 0, 1, 0},
    {0, 0, 0, 1}
};

for (unsigned int i = 0; i < 4; ++i) assert(matrix[i][i] == 1);
```

Mehrdimensionale Vektoren: Initialisierung

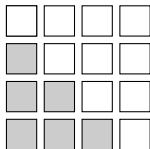
Auf bestimmte Grösse füllen:

```
unsigned int rows = ...;  
unsigned int cols = ...;  
std::vector<std::vector<int>> m;  
m = std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

`m` (Typ `std::vector<std::vector<int>>`) ist ein Vektor der Länge `rows`, dessen Elemente (Typ `std::vector<int>`) Vektoren der Länge `cols` sind, deren Elemente (Typ `int`) alles Einsen sind

(Es gibt noch viele weitere Wege, Vektoren zu initialisieren)

Matrix: Summe des unteren Dreiecks



```
int lower_triangle_sum(const std::vector<std::vector<int>>& matrix) {  
    int sum = 0;  
    for (unsigned int row = 0; row < matrix.size(); ++row) {  
        for (unsigned int column = 0; column < row; ++column) {  
            sum += matrix[row][column];  
        }  
    }  
    return sum;  
}
```

Mehrdimensionale Vektoren

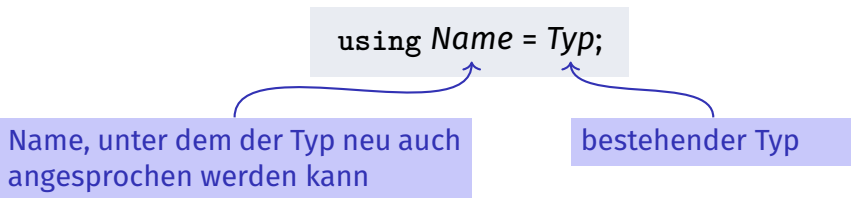
nicht-quadratische und sogar nicht-rechteckige 2-dimensionale Vektoren sind möglich:

```
std::vector<std::vector<std::string>> cantons = {  
    {"ZH", "BE", "SG", "GE", "LU", "SO", "NE", "ZG"},  
    {"FR", "VD", "VS", "JU"},  
    {"AI", "OW", "NW", "AR", "BS", "BL"}  
};
```

```
assert(cantons[1][2] == "VS");  
assert(cantons[1].size() == 4);
```

Mehrdimensionale Vektoren und Typ-Aliasse

- Auch möglich: Vektoren von Vektoren von Vektoren von ...:
`std::vector<std::vector<std::vector<...>>>`
- Typnamen können offensichtlich laaaaaaaang werden
- Dann hilft die Deklaration eines *Typ-Alias*:



Typ-Alias: Beispiel

```
#include <iostream>
#include <vector>
using imatrix = std::vector<std::vector<int>>>;

// POST: Matrix 'm' was output to standard output
void print(const imatrix& m);

int main() {
    imatrix m = ...;
    print(m);
}
```

Erinnerung: `const`-Referenz für Effizienz (keine Kopie) und Sicherheit (unveränderlich)

Keyword auto

```
std::vector<std::vector<int>> all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

stattdessen auch möglich:

```
auto all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

keyword auto erlaubt kompaktere Schreibweise:
Platzhalter für den Typ der deklarierten Variable
⇒ Compiler leitet Typ vom Initialisierungswert ab

```
auto x = 2 + 3; // type of x is int  
std::cout << x[2]; // compiler error
```

15. Rekursion I

Mathematische Rekursion, Terminierung, der Aufrufstapel, Beispiele

Mathematische Rekursion

- Viele mathematische Funktionen sind sehr natürlich *rekursiv* definierbar
- Das heisst, die Funktion erscheint in ihrer eigenen Definition

$$n! = \begin{cases} 1, & \text{falls } n \leq 1 \\ n \cdot (n-1)!, & \text{andernfalls} \end{cases}$$

Rekursion in C++: Genauso!

$$n! = \begin{cases} 1, & \text{falls } n \leq 1 \\ n \cdot (n-1)!, & \text{andernfalls} \end{cases}$$

```
// POST: return value is n!  
unsigned int fac(unsigned int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fac(n-1);  
}
```

Unendliche Rekursion

- ist so schlecht wie eine Endlosschleife ...
- ...nur noch schlechter: „verbrennt“ Zeit *und* Speicher

```
void f() {  
    f() // f() → f() → ... → stack overflow  
}
```

Rekursive Funktionen: Terminierung

Wie bei Schleifen brauchen wir garantierten Fortschritt Richtung einer Abbruchbedingung ($\approx$ Basisfall)

Beispiel `fac(n)`:

- Rekursion endet falls $n \leq 1$
- Rekursiver Aufruf mit neuem Argument $< n$
- Abbruchbedingung wird daher garantiert erreicht

```
unsigned int fac(  
    unsigned int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fac(n-1);  
}
```

Rekursive Funktionen: Auswertung

```
int fac(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fac(n-1);  
}
```

```
...  
std::cout << fac(4);
```

$\text{fac}(4) \rightsquigarrow \text{int } n = 4$

$\hookrightarrow \text{fac}(n - 1) \rightsquigarrow \text{int } n = 3$

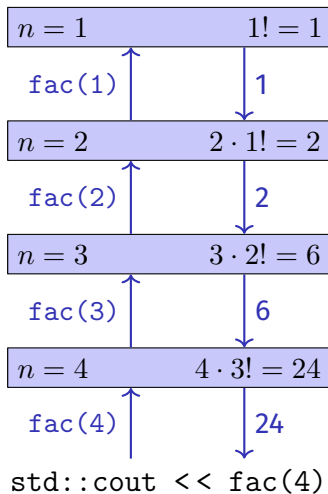
$\vdots$

Jeder Aufruf von fac arbeitet mit seinem eigenen n

Der Aufrufstapel

Bei jedem Funktionsaufruf:

- Wert des Aufrufarguments kommt auf einen Stapel
- Es wird immer mit dem obersten Wert gearbeitet
- Am Ende des Aufrufs wird der oberste Wert wieder vom Stapel gelöscht



Euklidischer Algorithmus

- findet den grössten gemeinsamen Teiler $\gcd(a, b)$ zweier natürlicher Zahlen a und b
- basiert auf folgender mathematischen Rekursion:

$$\gcd(a, b) = \begin{cases} a, & \text{falls } b = 0 \\ \gcd(b, a \bmod b), & \text{andernfalls} \end{cases}$$

Euklidischer Algorithmus in C++

$$\text{gcd}(a, b) = \begin{cases} a, & \text{falls } b = 0 \\ \text{gcd}(b, a \bmod b), & \text{andernfalls} \end{cases}$$

```
unsigned int gcd(unsigned int a, unsigned int b) {  
    if (b == 0)  
        return a;  
    else  
        return gcd(b, a % b);  
}
```

Terminierung: $a \bmod b < b$, also wird b in jedem rekursiven Aufruf kleiner

Bitstrings generieren

alle Bitstrings der Länge n generieren

Beispiel für $n = 6$:

000000

000001

000010

000011

...

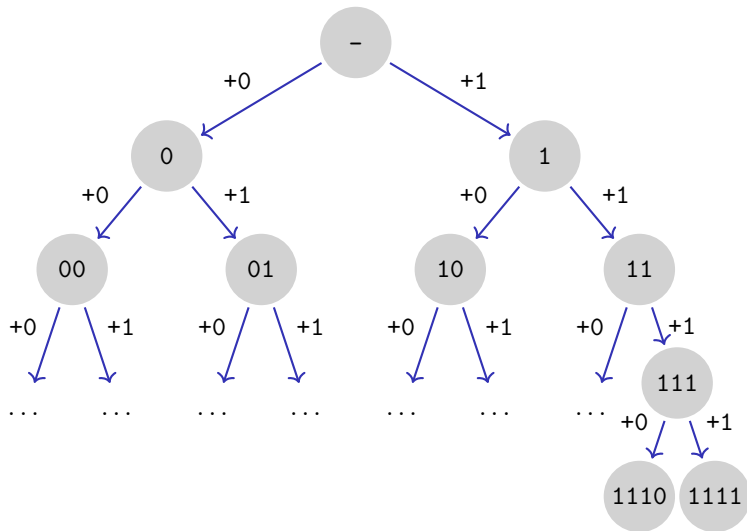
111100

111101

111110

111111

Bitstrings generieren: Idee



Bitstrings generieren: 1. Version

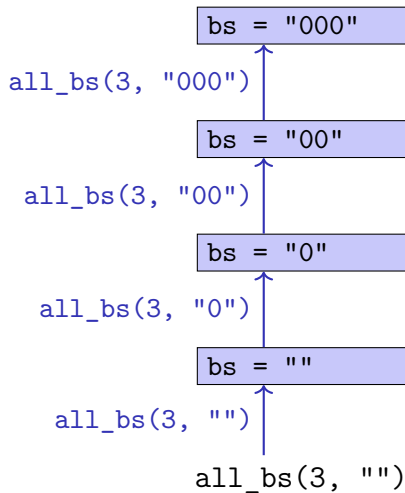
```
void all_bitstrings(unsigned int n, std::string bitstring) {  
    if (bitstring.size() == n) {  
        std::cout << bitstring << "\n";  
    } else {  
        all_bitstrings(n, bitstring + "0");  
        all_bitstrings(n, bitstring + "1");  
    }  
}
```

Aufruf mit `all_bitstrings(6, "")`;

Bitstrings: Aufrufstapel und Speicher

```
void all_bs(unsigned int n, std::string bs) {  
    if (bs.size() == n) {  
        std::cout << bs << "\n";  
    } else {  
        all_bs(n, bs + "0");  
        all_bs(n, bs + "1");  
    }  
}
```

"000"	3	"00"	3	"0"	3	" "	3
bs	n	bs	n	bs	n	bs	n



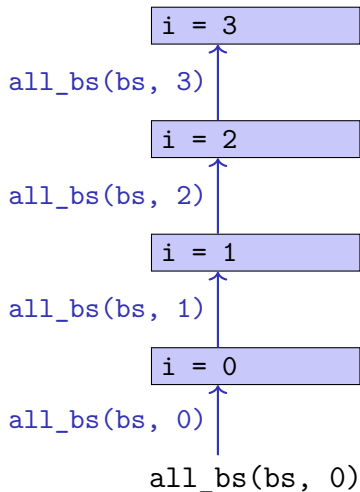
Bitstrings generieren: 2. Version

```
void all_bitstrings(std::string& bitstring, unsigned int i) {  
    if (i == bitstring.size()) {  
        std::cout << bitstring << "\n";  
    } else {  
        bitstring[i] = '0';  
        all_bitstrings(bitstring, i+1);  
        bitstring[i] = '1';  
        all_bitstrings(bitstring, i+1);  
    }  
}  
  
...  
std::string bitstring = std::string(3, '*');  
all_bitstrings(bitstring, 0);
```

Bitstrings: Aufrufstapel und Speicher

```
void bs(std::string& bs, unsigned int i) {  
    if (i == bs.size()) {  
        std::cout << bs << "\n";  
    } else {  
        bs[i] = '0';  
        all_bs(bs, i+1);  
        bs[i] = '1';  
        all_bs(bs, i+1);  
    }  
}  
...  
std::string bs = std::string(3, '*');  
all_bs(bs, 0);
```

		3	2	1	0	"000"
		i	i	i	i	bs



Die Macht der Rekursion

Einige Probleme scheinen ohne Rekursion kaum lösbar zu sein. Mit Rekursion werden sie plötzlich deutlich einfacher lösbar.

- n -Damen-Problem
- Die Türme von Hanoi
- Sudoku-Löser
- Parsen von Ausdrücken
- Umgekehrte Aus- oder Eingabe
- Suchen in Bäumen
- Divide-And-Conquer (z.B. Sortieren)

16. Rekursion II

Decrease and Conquer vs. Divide and Conquer, Rekursion vs. Iteration

Vektorsumme

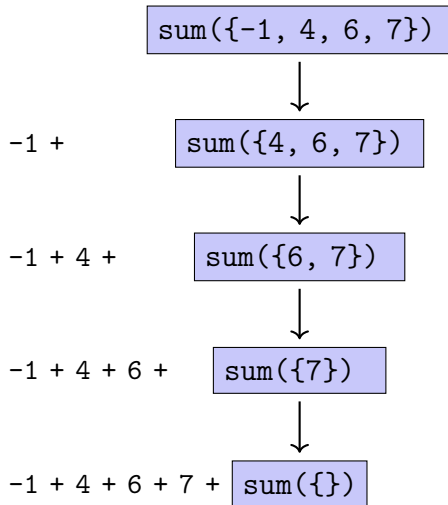
altbekanntes Problem:

Summe aller Einträge eines Vektors `std::vector<int> v`

einfache iterative Lösung:

```
int sum(const std::vector<int>& v) {  
    int s = 0;  
    for (unsigned int i = 0; i < v.size(); ++i) {  
        s += v.at(i);  
    }  
    return s;  
}
```

Vektorsumme: Variante 1

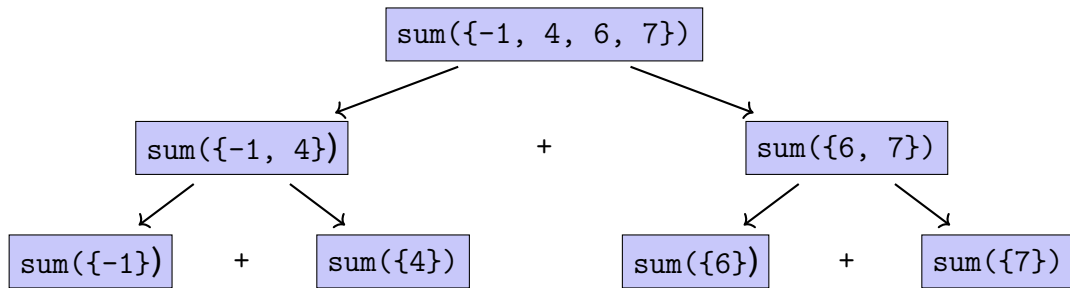


Vektorsumme: Variante 1

```
int sum(const std::vector<int>& v, unsigned int from) {  
    if (from >= v.size())  
        return 0;  
    else  
        return v.at(from) + sum(v, from + 1);  
}
```

Aufruf mit z.B. `sum({-1, 4, 6, 7}, 0);`

Vektorsumme: Variante 2

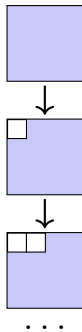


Vektorsumme: Variante 2

```
using uint = unsigned int;
int sum(const std::vector<int>& v, uint from, uint to) {
    if (from > to)
        return 0;
    else if (from == to)
        return v.at(from);
    else {
        uint middle = (from + to) / 2;
        return sum(v, from, middle) + sum(v, middle + 1, to);
    }
}
```

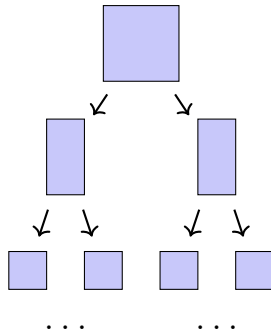
Aufruf mit z.B. `sum({-1, 4, 6, 7}, 0, 3);`

Rekursive Strategien



Decrease and Conquer

- ein Element entfernen,
ein rekursiver Aufruf auf Rest
- begrenzt parallelisierbar
- grosse Rekursionstiefe



Divide and Conquer

- halbieren der Elemente,
ein rekursiver Aufruf pro Hälfte
- oft gut parallelisierbar
- geringere Rekursionstiefe

Rekursion und Iteration

Rekursion kann *immer* simuliert werden durch

- Iteration (Schleifen)
- expliziten „Aufrufstapel“ (z.B. mittels eines Vektors)

Oft sind rekursive Formulierungen einfacher, aber meistens auch weniger effizient.

- Overhead durch Funktionsaufrufe
- Stack Overflow bei zu grosser Rekursionstiefe
(undefined behavior, typischerweise Segmentation Fault)
- potentielle Mehrfachberechnung von Zwischenresultaten

Fibonacci-Zahlen

$$F_n := \begin{cases} 0, & \text{falls } n = 0 \\ 1, & \text{falls } n = 1 \\ F_{n-1} + F_{n-2}, & \text{falls } n > 1 \end{cases}$$

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89 ...

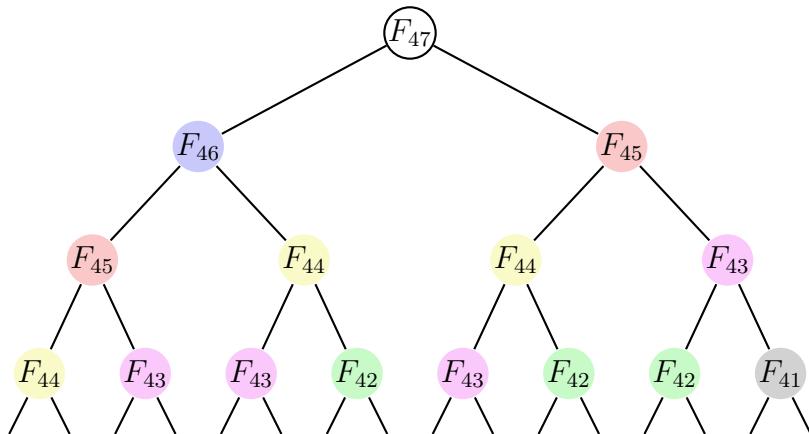
Fibonacci-Zahlen in C++

Laufzeit

`fib(50)` dauert „ewig“, denn es berechnet
 F_{48} 2-mal, F_{47} 3-mal, F_{46} 5-mal, F_{45} 8-mal, F_{44} 13-mal,
 F_{43} 21-mal ... F_1 ca. 10^9 mal (!)

```
unsigned int fib(unsigned int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
    return fib(n-1) + fib(n-2); // n > 1  
}
```

Fibonacci visualisiert



Mehrfachberechnung selber Werte = Mehrfachlösung selber Teilprobleme

Schnelle (iterative) Fibonacci-Zahlen

Idee:

- Berechne jede Fibonacci-Zahl nur einmal, in der Reihenfolge $F_0, F_1, F_2, \dots, F_n$
- Speichere jeweils die zwei letzten berechneten Fibonacci-Zahlen (Variablen `fib1` und `fib2`)
- Berechne die nächste Zahl als Summe von `fib1` und `fib2`

Kann rekursiv und iterativ implementiert werden, letzteres ist einfacher/direkter

Schnelle Fibonacci-Zahlen in C++

```
unsigned int fibonacci(unsigned int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
  
    unsigned int fib2 = 0; // value two before  
    unsigned int fib1 = 1; // value one before  
    unsigned int fib;      // current value  
  
    for (unsigned int i = 2; i <= n; ++i) {  
        fib = fib2 + fib1; // i-th fibonacci number  
        fib2 = fib1;  
        fib1 = fib;  
    }  
    return fib;  
}
```

sehr schnell auch bei fib(50)

$(F_{i-2}, F_{i-1}) \longrightarrow (F_{i-1}, F_i)$

fib2

fib1

Rekursion

- Einige Probleme scheinen ohne Rekursion kaum lösbar zu sein. Mit Rekursion werden sie plötzlich deutlich einfacher lösbar.
- Rekursive Lösungen sind oftmals einfacher und eleganter...
- ...aber iterative Lösungen meist effizienter.

Überladen von Funktionen

```
// iterative solution
```

```
int sum(const std::vector<int>& v) {  
    ...  
}
```

```
// recursive solution: decrease and conquer
```

```
int sum(const std::vector<int>&v, uint from) {  
    ...  
}
```

```
// recursive solution: divide and conquer
```

```
int sum(const std::vector<int>&v, uint from, uint to) {  
    ...  
}
```


- Funktionen sind durch Ihren Namen im Gültigkeitsbereich ansprechbar
- Es ist sogar möglich, mehrere Funktionen des gleichen Namens zu definieren und zu deklarieren
- Die „richtige“ Version wird aufgrund der *Signatur* der Funktion ausgewählt

Funktionsüberladung

- Eine Funktion ist bestimmt durch Namen, Typen, Anzahl und Reihenfolge der Argumente

```
double sq(double x) { ... }           // f1
int sq(int x) { ... }                 // f2
int pow(int b, int e) { ... }         // f3
int pow(int e) { return pow (2,e); } // f4
```

- Der Compiler wählt bei einem Funktionsaufruf automatisch die Funktion, welche „am besten passt“ (wir vertiefen das nicht)

```
std::cout << sq(3);    // Compiler wählt f2
std::cout << sq(1.414); // Compiler wählt f1
std::cout << pow(2);   // Compiler wählt f4
std::cout << pow(3,3); // Compiler wählt f3
```

17. Custom Data Types I

Rationale Zahlen, Struct-Definition, Operatorüberladung

Rechnen mit rationalen Zahlen

- Rationale Zahlen ($\mathbb{Q}$) sind von der Form $\frac{n}{d}$ mit n und d in $\mathbb{Z}$
- C++ hat keinen „eingebauten“ Typ für rationale Zahlen

Ziel

Wir bauen uns selbst einen C++-Typ für rationale Zahlen!



Vision

So könnte (wird) es aussehen

```
// input
std::cout << "Rational number r =? ";
rational r;
std::cin >> r;
std::cout << "Rational number s =? ";
rational s;
std::cin >> s;

// computation and output
std::cout << "Sum is " << r + s << ".\n";
```

Ein erstes Struct

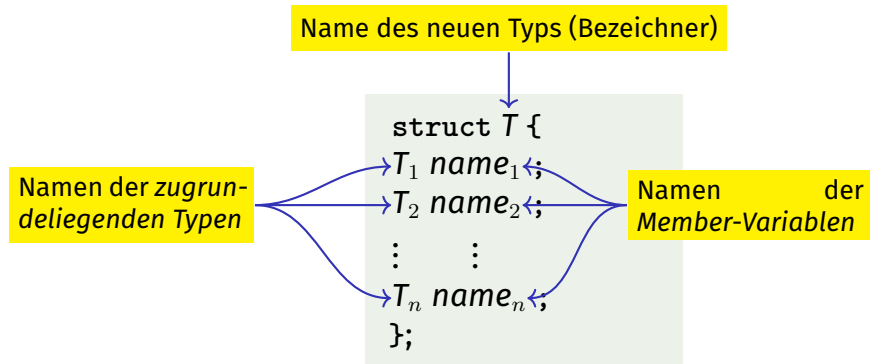
```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Invariante: spezifiziert gültige Wertkombinationen (informell).

Member-Variable (**d**enominator)

- struct definiert einen neuen Typ
- Formaler Wertebereich: *kartesisches Produkt* der Wertebereiche existierender Typen
- Echter Wertebereich: $\text{rational} \subsetneq \text{int} \times \text{int}$.

Struct-Definitionen



Wertebereich von T : $T_1 \times T_2 \times \dots \times T_n$

Struct-Definitionen: Beispiele

```
struct rational_vector_3 {  
    rational x;  
    rational y;  
    rational z;  
};
```

Zugrundeliegende Typen können fundamentale aber auch *benutzerdefinierte* Typen sein.

Struct-Definitionen: Beispiele

```
struct extended_int {  
    // represents value if is_positive==true  
    // and -value otherwise  
    unsigned int value;  
    bool is_positive;  
};
```

Die zugrundeliegenden Typen können natürlich auch *verschieden* sein.

Eingabe

```
// Input r
rational r;
std::cout << "Rational number r is: \n";
std::cout << " numerator =? ";
std::cin >> r.n;
std::cout << " denominator =? ";
std::cin >> r.d;

// Input s the same way
rational s;
...
```

Addition

```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};  
  
rational add(rational a, rational b){  
    rational result;  
    result.n = a.n * b.d + a.d * b.n;  
    result.d = a.d * b.d;  
    return result;  
}
```

$$\frac{r_n}{r_d} := \frac{a_n}{a_d} + \frac{b_n}{b_d} = \frac{a_n \cdot b_d + a_d \cdot b_n}{a_d \cdot b_d}$$

Structs: Member-Zugriff



Ausdruck vom Struct-Typ T .

Name einer Member-Variablen des Typs T .

$expr.name_k$

Ausdruck vom Typ T_k ; Wert ist der Wert des durch $name_k$ bezeichneten Objekts.

Member-Zugriff-Operator $.$

Structs: Initialisierung und Zuweisung

`rational s;` ← Member-Variablen uninitialisiert (wird sich bald ändern)

`rational t = {1, 5};` ← Memberweise Initialisierung:
t.n = 1, t.d = 5

`rational u = t;` ← Memberweise Kopie

`t = u;` ← Memberweise Kopie

`rational v = add(u,t);` ← Memberweise Kopie

Default-Initialisierung:

```
rational t;
```

- Member-Variablen von `t` werden default-initialisiert
- für Member-Variablen fundamentaler Typen passiert dabei nichts (Wert undefiniert)

Initialisierung:

```
rational t = {5, 1};
```

- Member-Variablen von `t` werden mit den Werten der Liste, entsprechend der Deklarationsreihenfolge, initialisiert.

Zuweisung:

```
rational s;  
...  
rational t = s;
```

- Den Member-Variablen von `t` werden die Werte der Member-Variablen von `s` zugewiesen.

Structs: Initialisierung und Zuweisung



```
t.n    = add (r, s) .n  
t.d    = add (r, s) .d ;
```

Initialisierung:

```
rational t = add(r, s);
```

- t wird mit dem Wert von add(r, s) initialisiert

Zuweisung:

```
rational t;  
t = add(r, s);
```

- `t` wird default-initialisiert
- Der Wert von `add (r, s)` wird `t` zugewiesen

Ein erstes Struct: Funktionalität

Ein struct definiert einen *Typ*, keine *Variable*!

```
// new type rational
```

```
struct rational {
```

```
    int n; ←
```

```
    int d; // INV: d != 0
```

```
};
```

Bedeutung: jedes Objekt des neuen Typs ist durch zwei Objekte vom Typ `int` repräsentiert, die die Namen `n` und `d` tragen.

```
// POST: return value is the sum of a and b
```

```
rational add (const rational a, const rational b)
```

```
{
```

```
    rational result;
```

```
    result.n = a.n * b.d + a.d * b.n;
```

```
    result.d = a.d * b.d;
```

```
    return result;
```

```
}
```

Member-Zugriff auf die `int`-Objekte von `a`.

Structs als Funktionsargumente

```
void increment(rational dest, const rational src)
{
    dest = add(dest, src); // veraendert nur lokale Kopie
}
```

Call by Value !

```
rational a;
rational b;
a.d = 1; a.n = 2;
b = a;
increment(b, a); // kein Effekt!
std::cout << b.n << "/" << b.d; // 1 / 2
```

Structs als Funktionsargumente

```
void increment(rational& dest, const rational src)
{
    dest = add(dest, src);
}
```

Call by Reference

```
rational a;
rational b;
a.d = 1; a.n = 2;
b = a;
increment(b, a);
std::cout << b.n << "/" << b.d; // 2 / 2
```

Vision in Reichweite ...

```
// computation
const rational t = add(r, s);

// output
std::cout << "Sum is " << t.n << "/" << t.d << ".\n";
```

Benutzerdefinierte Operatoren

Statt

```
rational t = add(r, s);
```

würden wir lieber

```
rational t = r + s;
```

schreiben.

Das geht mit *Überladen von Operatoren*.

Operator-Überladung (Operator Overloading)

- Operatoren sind spezielle Funktionen und können auch überladen werden
- Name des Operators *op*:
`operatorop`
- Wir wissen schon, dass z.B. `operator+` für verschiedene Typen existiert

rational addieren, bisher

```
// POST: return value is the sum of a and b
rational add(rational a, rational b)
{
    rational result;
    result.n = a.n * b.d + a.d * b.n;
    result.d = a.d * b.d;
    return result;
}

...
const rational t = add(r, s);
```

rational addieren, neu

```
// POST: return value is the sum of a and b
rational operator+(rational a, rational b)
{
    rational result;
    result.n = a.n * b.d + a.d * b.n;
    result.d = a.d * b.d;
    return result;
}
...
const rational t = r + s;
```



Infix-Notation

Andere binäre Operatoren für rationale Zahlen

```
// POST: return value is difference of a and b  
rational operator-(rational a, rational b);
```

```
// POST: return value is the product of a and b  
rational operator*(rational a, rational b);
```

```
// POST: return value is the quotient of a and b  
// PRE: b != 0  
rational operator/(rational a, rational b);
```

Unäres Minus

Hat gleiches Symbol wie binäres Minus, aber nur ein Argument:

```
// POST: return value is -a
rational operator-(rational a)
{
    a.n = -a.n;
    return a;
}
```

Arithmetische Zuweisungen

Wir wollen z.B. schreiben

```
rational r;  
r.n = 1; r.d = 2;           // 1/2
```

```
rational s;  
s.n = 1; s.d = 3;          // 1/3
```

```
r += s;  
std::cout << r.n << "/" << r.d;    // 5/6
```

Operator+= Erster Versuch



```
rational operator+=(rational a, rational b)
{
    a.n = a.n * b.d + a.d * b.n;
    a.d *= b.d;
    return a;
}
```

Das funktioniert nicht! Warum?

- Der Ausdruck `r += s` hat zwar den gewünschten Wert, weil die Aufrufargumente R-Werte sind (call by value!) jedoch **nicht den gewünschten Effekt** der Veränderung von `r`.
- Das Resultat von `r += s` stellt zudem entgegen der Konvention von C++ keinen L-Wert dar.

Operator +=

```
rational& operator+=(rational& a, rational b)
{
    a.n = a.n * b.d + a.d * b.n;
    a.d *= b.d;
    return a;
}
```

Das funktioniert!

- Der L-Wert a wird um den Wert von b erhöht und als L-Wert zurückgegeben.

`r += s;` hat nun den gewünschten Effekt.

Structs vergleichen?

Für jeden fundamentalen Typ (`int`, `double`, ...) gibt es die Vergleichsoperatoren `==` und `!=`, aber nicht für Structs! Warum?

- Memberweiser Vergleich ergibt im allgemeinen keinen Sinn,...

- ...denn dann wäre z.B. $\frac{2}{3} \neq \frac{4}{6}$

Vergleichsoperatoren

Sind für Structs nicht eingebaut, können aber definiert werden:

```
// POST: returns true iff a == b
bool operator==(rational a, rational b)
{
    return a.n * b.d == a.d * b.n;
}
```

$$\frac{2}{3} = \frac{4}{6} \quad \checkmark$$

Ein-/Ausgabeoperatoren

können auch überladen werden.

■ Bisher:

```
std::cout << "Sum is " << t.n << "/" << t.d << "\n";
```

■ Neu (gewünscht):

```
std::cout << "Sum is " << t << "\n";
```

Ein-/Ausgabeoperatoren

können auch überladen werden wie folgt:

```
// POST: r has been written to out
std::ostream& operator<<(std::ostream& out, rational r)
{
    return out << r.n << "/" << r.d;
}
```

schreibt `r` auf den Ausgabestrom
und gibt diesen als L-Wert zurück

`std::cout` ist vom Typ `std::ostream`

Eingabe

```
// PRE: in starts with a rational number of the form "n/d"  
// POST: r has been read from in  
std::istream& operator>>(std::istream& in, rational& r){  
    char c; // separating character '/'  
    return in >> r.n >> c >> r.d;  
}
```

liest r aus dem Eingabestrom
und gibt diesen als L-Wert zurück.

std::cin ist vom Typ std::istream

Ziel erreicht!

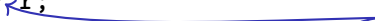
```
// input
```

```
std::cout << "Rational number r =? ";
```

```
rational r;
```

```
std::cin >> r;
```

operator >>



```
std::cout << "Rational number s =? ";
```

```
rational s;
```

```
std::cin >> s;
```

operator +



```
// computation and output
```

```
std::cout << "Sum is " << r + s << ".\\n";
```

operator<<



Invarianten garantieren?

```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};
```

...

```
rational r;  
r.n = 1;  
r.d = 1;
```

...

```
r.d--;
```

18. Custom Data Types II

Datenkapselung, Klassen, Memberfunktionen, Konstruktoren

Ein neuer Typ mit Funktionalität...

```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};  
  
// POST: return value is the sum of a and b  
rational operator+ (rational a, rational b)  
{  
    rational result;  
    result.n = a.n * b.d + a.d * b.n;  
    result.d = a.d * b.d;  
    return result;  
}  
...
```


...gehört in eine Bibliothek!

rational.h

- Definition des Structs `rational`
- Funktionsdeklarationen

rational.cpp

- Arithmetische Operatoren (`operator+`, `operator+=`, ...)
- Relationale Operatoren (`operator==`, `operator>`, ...)
- Ein-/Ausgabe (`operator >>`, `operator <<`, ...)

Invarianten und Repräsentation

- Wollen für eine Datenstruktur Invarianten garantieren, ohne den Benutzercode mit Asserts zu übersäen.

Beispiel 1: Für jedes `Rational r` gilt stets $r.d \neq 0$.

Beispiel 2: Jedes `Rational r` ist stets gekürzt.

- Liefern dem Benutzer das „was“ und nicht das „wie“.

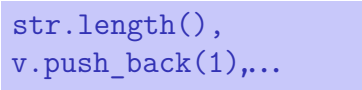
```
Vector v: v.size(), v[3], v.at(3)
```

- Die interne Darstellung sollte einfach anzupassen sein, ohne dass Benutzercode neu geschrieben werden muss.

`Complex`: Polarkoordinaten vs. kartesische Koordinaten

Idee der Datenkapselung (Information Hiding)

- Ein Typ ist durch *Wertebereich* und *Funktionalität* eindeutig definiert.
- Die *Repräsentation* soll *nicht sichtbar* sein.
- $\Rightarrow$ Es wird keine *Repräsentation*, sondern Funktionalität angeboten.

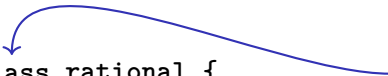


```
str.length(),  
v.push_back(1),...
```

Klassen

- sind das Konzept zur Datenkapselung in C++
- sind eine Variante von Structs
- gibt es in vielen *objektorientierten Programmiersprachen*

Datenkapselung: public / private



```
class rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Wird statt struct verwendet, wenn überhaupt etwas “versteckt” werden soll.

Einzigster Unterschied:

- struct: standardmässig wird nichts versteckt
- class : standardmässig wird alles versteckt

Datenkapselung: public / private

```
class rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Gute Nachricht: `r.d = 0` aus Versehen geht nicht mehr

Schlechte Nachricht: wir können gar nichts mehr machen ...

Anwendungscode:

```
rational r;  
r.n = 1;      // error: n is private  
r.d = 2;      // error: d is private  
int i = r.n;  // error: n is private
```

Memberfunktionen: Deklaration

```
class rational {  
public:  
    // POST: return value is the numerator of this instance  
    int numerator () const {  
        return n;  
    }  
    // POST: return value is the denominator of this instance  
    int denominator () const {  
        return d;  
    }  
private:  
    int n;  
    int d; // INV: d != 0  
};
```

öffentlicher Bereich

Memberfunktion

Memberfunktionen haben Zugriff auf private Daten

Gültigkeitsbereich von Mem-
bern in einer Klasse ist die
ganze Klasse, unabhängig von
der Deklarationsreihenfolge

Memberfunktionen: Aufruf

```
// Definition des Typs
class rational {
    ...
};
...
// Variable des Typs
rational r;
int n = r.numerator();    // Zaehler
int d = r.denominator();  // Nenner
```

Member-Zugriff



const und Memberfunktionen

```
class rational {  
public:  
    int numerator () const  
    { return n; }  
    void set_numerator (int N)  
    { n = N;}  
    ...  
}
```

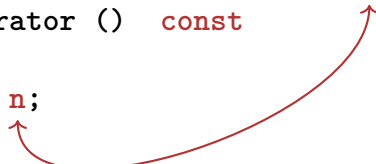
```
rational x;  
x.set_numerator(10); // ok;  
const rational y = x;  
int n = y.numerator(); // ok;  
y.set_numerator(10); // error;
```

Das `const` an einer Memberfunktion liefert das Versprechen, dass eine Instanz nicht über diese Funktion verändert wird.

`const` Objekte dürfen nur `const` Memberfunktionen aufrufen!

Memberfunktionen: Definition

```
// POST: returns numerator of this instance
int numerator () const
{
    return n;
}
```



- Eine Memberfunktion wird für einen Ausdruck der Klasse aufgerufen. In der Funktion: **this** ist der Name dieses *impliziten Arguments*. **this** selbst ist ein Zeiger darauf.
- Das **const** bezieht sich auf die Instanz **this**, verspricht also, dass das implizite Argument nicht im Wert verändert wird.
- **n** ist Abkürzung in der Memberfunktion für **this->n** (genaue Erklärung von „->“ nächste Woche)

This rational vs. dieser Bruch

So wäre es in etwa ...

```
class rational {  
    int n;  
    ...  
public:  
    int numerator () const  
    {  
        return this->n;  
    }  
};  
  
rational r;  
...  
std::cout << r.numerator();
```

... ohne Memberfunktionen

```
struct bruch {  
    int n;  
    ...  
};  
  
int numerator (const bruch& dieser)  
{  
    return dieser.n;  
}  
  
bruch r;  
..  
std::cout << numerator(r);
```

- sind spezielle *Memberfunktionen* einer Klasse, die den Namen der Klasse tragen.
- können wie Funktionen überladen werden, also in der Klasse mehrfach, aber mit verschiedener *Signatur* vorkommen.
- werden bei der Variablendeklaration wie eine Funktion aufgerufen. Der Compiler sucht die „naheliegendste“ passende Funktion aus.
- wird kein passender Konstruktor gefunden, so gibt der Compiler eine *Fehlermeldung* aus.

Initialisierung? Konstruktoren!

```
class rational
{
public:
    rational (int num, int den)
        : n (num), d (den) ← Initialisierung der
                               Membervariablen
    {
        assert (den != 0); ← Funktionsrumpf.
    }
    ...
};
...
rational r = rational(2,3); // r = 2/3
```

- direkt

```
rational r (1,2); // initialisiert r mit 1/2
```

- indirekt (Kopie)

```
rational r = rational (1,2);
```

Der Default-Konstruktor

```
class rational
{
public:
    ...
    rational () 
        : n (0), d (1)
    {}
    ...
};
...
rational r;    // r = 0
// Abkürzung für rational r = rational();
```

⇒ Es gibt keine uninitialisierten Variablen vom Typ rational mehr!

- wird automatisch aufgerufen bei Deklarationen der Form `rational r;`
- ist der eindeutige Konstruktor mit leerer Argumentliste (falls existent)
- muss existieren, wenn `rational r;` kompilieren soll
- wenn in einem Struct keine Konstruktoren definiert wurden, wird der Default-Konstruktor automatisch erzeugt (wegen der Sprache C)

- Aus technischen und historischen Gründen gibt es in C++ verschiedene Formen der Objektinitialisierung, die sich in Syntax und Semantik unterscheiden. Die Initialisierung eines neuen Objekts t vom Typ T kann daher potenziell wie folgt erfolgen:
 - $T\ t(...);$
 - $T\ t\{...\};$
 - $T\ t = T(...);$
 - $T\ t = T\{...\};$
 - $T\ t = \{...\};$
- Wichtig: Bei allen Formen handelt es sich um Initialisierungen, nicht um Zuweisungen und $T\ t = ...;$ ist nicht äquivalent zu $T\ t;$ $t = ...;$
- Bei Interesse können Sie hier mehr erfahren:
 - <https://en.cppreference.com/w/cpp/language/initialization>
 - <https://isocpp.org/wiki/faq/cpp11-language#uniform-init>
 - <https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Res-list>
 - <https://quuxplusone.github.io/blog/2019/02/18/knightmare-of-initialization>

Objektinitialisierung in diesem Kurs: Hintergrund



- Ziel dieses Kurses ist nicht, Sie zu C++-Experten auszubilden, sondern Ihnen Programmiergrundlagen zu vermitteln, die allgemein relevant sind und sich – möglichst direkt – auf andere Sprachen übertragen lassen
- Wir verwenden daher drei Syntaxvarianten, jede in einer bestimmten Situation
- Dabei haben wir uns von folgenden, nicht immer gleichzeitig erreichbaren Zielen leiten lassen:
 1. Übertragbarkeit: Die Syntax sollte derer anderer populärer Sprachen, z.B. Python, Java, Go, JavaScript, möglichst ähneln
 2. Einheitlichkeit: Je weniger Syntaxen verwendet werden, desto geringer das Potenzial für Verwirrung
 3. Stabilität: Kleine Änderungen (z.B. Hinzufügen eines expliziten Konstruktors) sollte keine unerwarteten Probleme nachsichziehen (z.B. weil der Compiler nun bestimmte Konstruktoren nicht mehr erstellt)
 4. Zweckmässigkeit: Bestimmte Formen sind, in manchen Situationen, besonders intuitiv & praktisch
 5. Idiomatisch: Der resultierende Code sollte realistischer C++-Code sein

Objektinitialisierung in diesem Kurs: Konventionen H

1. Für **primitive Datentypen** (`int`, `bool`, `double`, ...) und Initialisierung mittels **Literalen** verwenden wir *Copy Initialisation*. Beispiele:

```
int i = 5;  
std::string s = "I am literally a string, folks!";
```

2. Für die **komponentenweise** Initialisierung von Containern (z.B. `vector`; demnächst mehr) und simplen Datentypen mit ausschliesslich öffentlichen Membervariablen (z.B. `struct rational` auf Folie 461) nutzen wir *List Initialisation* bzw. *Aggregate Initialisation*. Beispiele:

```
std::vector<char> vowels = {'A', 'E', 'I', 'O', 'U'};  
rational half = {1, 2};
```

Objektinitialisierung in diesem Kurs: Konventionen H

3. In den **verbleibenden Fällen**, z.B. für `struct rational` mit privaten Membervariablen auf Folie 508, nutzen wir *Copy Initialisation* mittels expliziter Konstruktoraufrufe in Funktionsaufrufsyntax. Beispiele:

```
rational half = rational(1,2);  
std::vector<int> empty = std::vector<int>(7, 0); // vector with seven zeroes  
  
// 'auto' fits in nicely, and avoids having to repeat types  
auto half = rational(1,2);  
auto empty = std::vector<int>(7, 0);
```

Objektinitialisierung in diesem Kurs: Konventionen H

4. **Ausnahme:** Initialisierung von Membervariablen in Konstruktordefinitionen (*member initialiser lists*). Beispiel:

```
class rational {  
    ...  
    rational(int n, int d): num(n), den(d) {...}  
}
```

Keine der hier erlaubten Syntaxformen – $t(\dots)$ und $t\{\dots\}$ – ist ideal und wir haben uns daher relativ beliebig für die runden Klammern entschieden.

Keine Panik!

- Sie müssen weder die Gründe für, noch die Unterschiede zwischen den einzelnen Initialisierungsformen kennen
- Auch die Namen der unterschiedlichen Initialisierungsformen müssen Sie nicht kennen
- Es reicht, wenn Sie Objektinitialisierung auf dem Niveau der Vorlesungsbeispiele und Übungsaufgaben verstehen
- Sie müssen sich nicht an unsere Konventionen halten und dürfen andere Formen der Objektinitialisierung nutzen

Alternative: Default-Konstruktor löschen

```
class rational
{
public:
    ...
    rational () = delete;
    ...
};
...
rational r;    // error: use of deleted function 'rational::rational()'

```

⇒ Es gibt keine uninitialisierten Variablen vom Typ rational mehr!

Initialisierung “rational = int”?

```
class rational
{
public:
    rational (int num)
        : n (num), d (1)
    {}    ← Leerer Funktionsrumpf
    ...
};
...
rational r = rational(2); // Explizite Initialisierung mit 2
rational s = 2; // Implizite Konversion
```


Benutzerdefinierte Konversionen

sind definiert durch Konstruktoren mit genau *einem* Argument

```
rational (int num) ← Benutzerdefinierte Konversion von int  
    : n (num), d (1) nach rational. Damit wird int zu einem  
    {} Typ, dessen Werte nach rational kon-  
        vertierbar sind.
```

```
rational r = 2; // implizite Konversion
```

Benutzerdefinierte Konversionen

Wie kann man implizite Konversion von `rational` nach `double` realisieren?

- Problem: `double` ist kein Struct (keine Klasse), wir können dem Typ keinen Konstruktor „verpassen“ (gilt auch für alle anderen Zieltypen, die nicht „uns“ gehören)
- Lösung: wir bringen unserem Typ `rational` die Konversion nach `double` bei (als Member-Funktion):

```
struct rational{  
    ...  
    operator double () ← impliziter Rückgabetyp double  
    {  
        return double (n)/d;  
    }  
};  
  
rational a(1,2);  
double b = a;  // implizite Konversion
```

Member-Definition: In-Class vs. Out-of-Class

```
class rational {  
    int n;  
    ...  
public:  
    int numerator () const  
    {  
        return n;  
    }  
    ....  
};
```

- Keine Trennung zwischen Deklaration und Definition (schlecht für Bibliotheken)

```
class rational {  
    int n;  
    ...  
public:  
    int numerator () const;  
    ...  
};  
  
int rational::numerator () const  
{  
    return n;  
}
```

- So geht's auch.

Bereichsauflösungsoperator ::

Der Operator `::` wird genutzt um die Zugehörigkeit eines Namen zu bestimmen. Dies funktioniert sowohl für Namensräume als auch für Klassen. Beispiel: `std::cout`

Praktisch relevant hier: bei Definition einer Member-Funktion ausserhalb der Klasse ist `::` notwendig. Beispiel:

```
class rational {  
    ...  
};  
...  
int rational::numerator () const {  
    return n;  
}
```

19. Container und Iteratoren

Container, Mengen, Iteratoren, const-Iteratoren, Algorithmen

Vektoren sind Container

- Abstrakt gesehen ist ein Vektor
 1. Eine (geordnete) Ansammlung von Elementen
 2. Plus Operationen auf dieser Ansammlung
- In C++ heissen `vector<T>` und ähnliche „Ansammlungs“-Datenstrukturen *Container*
- In manchen Sprachen, z.B. Java, *Collections* genannt

Container-Eigenschaften

- Jeder Container hat bestimmte *charakteristische Eigenschaften*
- Ein Vektor `std::vector<T>` z.B. die folgenden:
 - Effizienter, index-basierter Zugriff (`v[i]`)
 - Effiziente Speichernutzung: Nur die Elemente selbst belegen Platz (plus Elementezähler)
 - Einfügen/Entfernen an beliebigem Index ist potenziell ineffizient
 - Suchen eines bestimmten Elements ist potenziell ineffizient
 - Kann Elemente mehrfach enthalten
 - Elemente sind in Einfügereihenfolge enthalten (geordnet aber unsortiert)

Container in C++

- Fast jede Anwendung erfordert die Verwaltung und Manipulation von beliebig vielen Datensätzen
- Aber mit unterschiedlichen Anforderungen (z.B. Elemente nur hinten anhängen, fast nie entfernen, oft suchen, ...)
- Deswegen enthält die Standardbibliothek von C++ diverse Container mit unterschiedlichen Eigenschaften, siehe <https://en.cppreference.com/w/cpp/container>
- Viele weitere sind über Bibliotheken Dritter verfügbar, z.B. <https://www.boost.org/doc/libs/release/libs/container/>, <https://github.com/abseil/abseil-cpp>

Beispiel-Container: `std::list<T>`

- wie ein Vektor, eine geordnete Liste von Elementen
- Eigenschaften:
 - effizientes Einfügen/Entfernen an beliebiger Stelle
 - kein Random Access
 - ineffizientere Speichernutzung als Vektor

```
std::list<int> l = {1, 2, 3};  
  
// add 4 to the back of the list  
l.push_back(4); // l = {1, 2, 3, 4}  
  
// add 0 to the front of the list  
l.push_front(0); // l = {0, 1, 2, 3, 4}
```

Beispiel-Container: `std::unordered_set<T>`

- Eine *mathematische Menge* ist eine ungeordnete, duplikatfreie Zusammenfassung von Elementen:

$$\{1, 2, 1\} = \{1, 2\} = \{2, 1\}$$

- In C++: `std::unordered_set<T>`
- Eigenschaften:
 - Kann kein Element doppelt enthalten
 - Elemente haben keine bestimmte Reihenfolge
 - Kein indexbasierter Zugriff (`s[i]` nicht definiert)
 - Effiziente „Element enthalten?“-Prüfung
 - Effizientes Einfügen und Löschen von Elementen
- Randbemerkung: Implementiert als Hash-Tabelle

Anwendungsbeispiel `std::unordered_set<T>`

Problem:

- Gegeben eine Sequenz an Paaren (*Name*, *Prozente*) von Code-Expert-Submissions ...

```
// Input: file submissions.txt
Friedrich 90
Sasse 70
Schwerhoff 10
Lehner 20
Schwerhoff 11
Sasse 60
```

- ... bestimme die Abgebenden, die mindestens 50% erzielt haben

```
// Output
Friedrich Sasse
```

Anwendungsbeispiel `std::unordered_set<T>`

```
std::ifstream in("submissions.txt");  
std::unordered_set<std::string> names;  
  
std::string name;  
unsigned int score;  
  
while (in >> name >> score) {  
    if (50 <= score)  
        names.insert(name);  
}  
  
std::cout << "Unique submitters: "  
          << names << '\n';
```

Öffne submissions.txt

Namen-Menge, initial leer

Paar (Name, Punkte)

Nächstes Paar einlesen
Namen merken falls Punkte
ausreichen

Gemerkte Namen ausgeben

Beispiel-Container: `std::set<T>`

- Fast gleich wie `std::unordered_set<T>`, aber die Elemente sind *geordnet*

$$\{1, 2, 1\} = \{1, 2\} \neq \{2, 1\}$$

- Elemente suchen, einfügen und löschen weiterhin effizient (besser als bei `std::vector<T>`), aber weniger effizient als bei `std::unordered_set<T>`
- Denn das Beibehalten der Ordnung zieht etwas Aufwand nach sich
- Randbemerkung: Implementiert als Rot-Schwarz-Baum

Anwendungsbeispiel `std::set<T>`

```
std::ifstream in("submissions.txt");  
std::set<std::string> names; ← set statt unordered_set ...
```

```
std::string name;  
unsigned int score;
```

```
while (in >> name >> score) {  
    if (50 <= score)  
        names.insert(name);  
}
```

```
std::cout << "Unique submitters: "  
           << names << '\n'; ← ... und die Ausgabe erfolgt  
                               alphabetisch sortiert
```

Iteratoren

- viele Operationen (z.B. Ausgabe eines Containers) erfordern Iteration über Container
- $\Rightarrow$ Jeder C++-Container implementiert seinen eigenen *Iterator*
- Gegeben ein Container `c`:
 - `it = c.begin()`: Iterator aufs erste Element
 - `it = c.end()`: Iterator *hinters* letzte Element
 - `*it`: Zugriff aufs aktuelle Element (Dereferenzoperator)
 - `++it`: Iterator um ein Element verschieben
- Iteratoren sind quasi gepimpte Zeiger (siehe nächste Woche!)

Beispiel: Iteration über `std::vector`

```
std::vector<int> v = {1, 2, 3};
```

```
for (std::vector<int>::iterator it = v.begin(),
```

```
    it != v.end();
```

```
    ++it)
```

```
    *it = -*it;
```

```
}
```

it ist ein zu `std::vector<int>` passender Iterator

it zeigt initial aufs erste Element

Abbruch falls it Ende erreicht hat

it elementweise vorwärtssetzen

Aktuelles Element negieren ($e \rightarrow -e$)

Beispiel: Iteration über `std::vector`

Zur Erinnerung: Type-Aliasse können genutzt werden um oft verwendete Typnamen abzukürzen

```
using ivit = std::vector<int>::iterator; // int-vector iterator

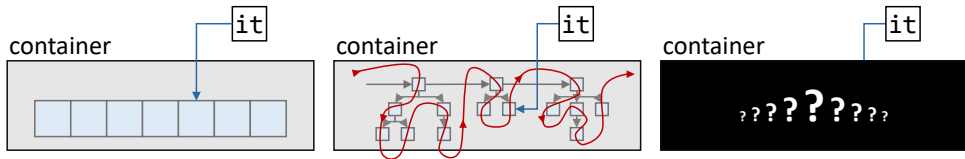
for (ivit it = v.begin();
    ...
```

alternativ können wir `auto` verwenden

```
for (auto it = v.begin();
    ...
```

Iteratoren

- Iteratoren ermöglichen Zugriff auf verschiedene Container auf *uniforme* Weise: `*it`, `++it`, etc.
- Nutzer bleiben unabhängig von der Container-Implementierung
- Iterator weiss, wie man die Elemente „seines“ Containers abläuft
- Nutzer müssen und sollen interne Details nicht kennen
- $\Rightarrow$ Containerimplementierung kann geändert werden, ohne dass Nutzer Code ändern müssen



- Viele weitere nützliche Funktionen können mittels Container-Iteration implementiert werden:
- `contains(c, e)`: wahr gdw. Container `c` Element `e` enthält
- `min/max(c)`: Gibt das grösste/kleinste Element zurück
- `sort(c)`: Sortiert die Elemente von `c`
- `replace(c, e1, e2)`: Ersetzt alle `e1` in `c` mit `e2`
- `sample(c, n)`: Wählt zufällig `n` Elemente aus `c` aus
- ...

Negieren als Funktion

Wie zuvor: Übergabe eines *Arbeitsbereichs* (-intervalls)

```
void neg(std::vector<int>::iterator begin,  
        std::vector<int>::iterator end) {
```

← Elemente im Intervall
[begin, end) negieren

```
    for (std::vector<int>::iterator it = begin;  
         it != end;  
         ++it) {  
  
        *it = -*it;  
    }  
}
```

Negieren als Funktion

Wie zuvor: Übergabe eines *Arbeitsbereichs* (-intervalls)

```
void neg(std::vector<int>::iterator begin;  
        std::vector<int>::iterator end);
```

```
// in main():  
std::vector<int> v = {1, 2, 3};  
neg(v.begin(), v.begin() + (v.size() / 2)); ← Erste Hälfte negieren
```

Algorithmen-Bibliothek in C++

- Die C++-Standardbibliothek enthält viele nützliche Algorithmen (Funktionen), die auf durch Iteratoren bestimmten Intervallen [*Anfang*, *Ende*) arbeiten
- Zum Beispiel `find`, `fill` and `sort`; siehe auch <https://en.cppreference.com/w/cpp/algorithm>
- Dank Iteratoren können diese ≥ 100 (!) Algorithmen auf beliebigen* Containern ausgeführt werden: Den 17 (!) Standardcontainern von C++, unsere eigenen Containers, etc.
- Gäbe es diesen uniformen Zugriff auf Containerelemente nicht, müsste *sehr* viel Code dupliziert werden

Nicht jeder Algorithmus kann auf jedem Container aufgerufen werden. Z.B. kann ein `std::unordered_set` nicht sortiert werden.

Bereichsbasierte for-Schleife

- Sequenzielle Iteration über einen Container (hier: `std::list<int>`):

```
std::list<int> l = {0, 0, 0};  
for (list<int>::iterator it = l.begin(); it != l.end(); ++it)  
    std::cout << *it; // prints 000
```

- Kann alternativ auch wie folgt geschrieben werden:

```
for (int e : l) std::cout << e; // prints 000
```

Wird vom Compiler zu Iterator-basierter Schleife übersetzt.

- Modifizierender Zugriff ist auch möglich:

```
for (int& e : l) e += 3; // modifies  
for (int e : l) std::cout << e; // prints 333
```

Const-Iteratoren

- Neben `iterator` sollte jeder Container auch einen *Const-Iterator* `const_iterator` bereitstellen
- Const-Iteratoren gestatten nur Lesezugriff auf den darunterliegenden Container
- Zum Beispiel für `llvec`:

```
llvec::const_iterator llvec::cbegin() const;  
llvec::const_iterator llvec::cend() const;  
  
const int& llvec::const_iterator::operator*() const;  
...
```

- Daher nicht möglich (Compilerfehler): `*(v.cbegin()) = 0`

Const-Iterator *kann* verwendet werden um nur Lesen zu erlauben:

```
llvec v = ...;  
for (llvec::const_iterator it = v.cbegin(); ...)  
    std::cout << *it;
```

Hier könnte auch der nicht-const iterator verwendet werden

Const-Iterator *muss* verwendet werden falls Vektor selbst const ist:

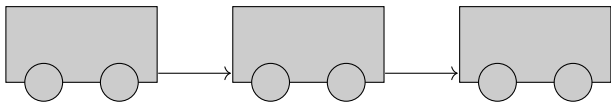
```
const llvec v = ...;  
for (llvec::const_iterator it = v.cbegin(); ...)  
    std::cout << *it;
```

Hier kann nicht der `iterator` verwendet werden (Compilerfehler)

20. Dynamischer Speicher und Zeiger

Adressen und Zeiger, Const-Zeiger, **new**-Ausdruck

Wagons



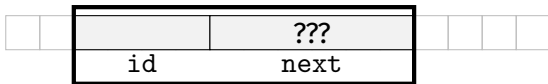
Ziel: beliebig viele Wagons aneinanderhängen

Idee: Objekte vom Typ `wagon`, die sich den nächsten Wagon speichern

Wagons: 1. Versuch

```
struct wagon {  
    unsigned int id;  
    wagon next;  
};
```

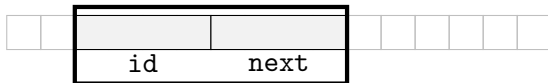
field 'next' has incomplete type 'wagon'



Struct kann keine Membervariable des gleichen Typs haben (unendliche Rekursion)

Wagons: 2. Versuch

```
struct wagon {  
    unsigned int id;  
    wagon& next;  
    wagon(unsigned int id) : id(id) {}  
};
```



```
wagon w = wagon(1);
```

uninitialized reference member in 'struct wagon&'

Referenzen müssen bei Deklaration initialisiert werden

Zeiger (Pointer)

Zeiger (Pointer) sind wie Referenzen, die auch ins Nichts zeigen dürfen!

wagon&: Referenz auf ein Wagon

wagon*: Pointer auf ein Wagon

Zeiger (Pointer) dürfen nach ihrer Erstellung verändert werden.

Zeiger-Typen

T* Zeiger-Typ für zugrunde liegenden Typ T

Ein Ausdruck vom Typ T* heisst *Zeiger (auf T)*

```
int* p = ...; // Zeiger auf einen int  
std::string* q = ...; // Zeiger auf einen std::string
```

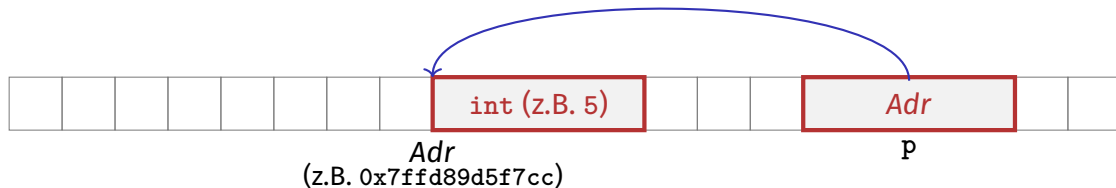

Zeiger-Typen

T* Zeiger-Typ für zugrunde liegenden Typ T

Wo ein T* draufsteht, muss auch ein T drin sein

```
int* p = ...;  
std::string* q = p; // compiler error!
```

Zeiger-Typen

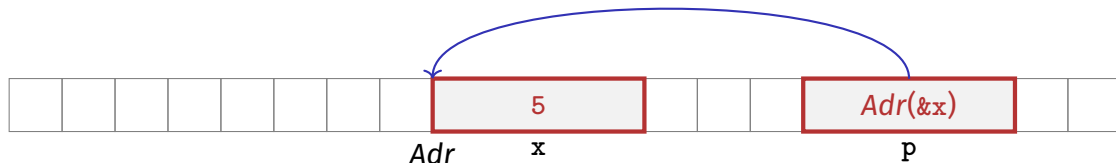


```
int* p = ...;  
std::cout << p; // z.B. 0x7ffd89d5f7cc
```

Wert eines Zeigers auf T ist die Adresse eines Objektes vom Typ T

Adressoperator

```
int x = 5; // x initialisiert mit 5  
int* p = &x; // p initialisiert mit der Adresse von x
```



Adressoperator

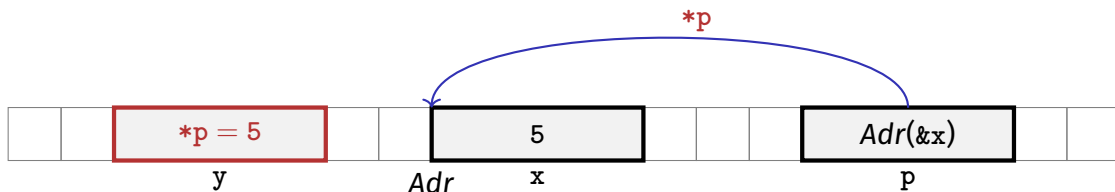
&expr

expr: L-Wert vom Typ *T*

- **Wert** des Ausdrucks *&expr*: Die *Adresse* des Objekts (L-Wertes) *expr*
- **Typ** des Ausdrucks: Ein Zeiger *T** (vom Typ *T*)

Dereferenzoperator

```
int x = 5;  
int* p = &x; // p = Adresse von x  
int y = *p;  // y = 5
```



Dereferenzoperator

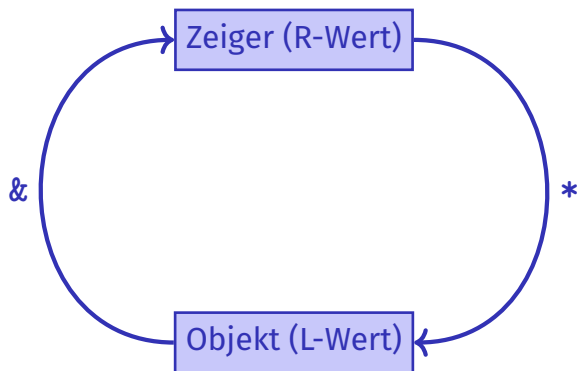
Dereferenzoperator *

**expr*

expr: R-Wert vom Typ T^*

- **Wert** des Ausdrucks: Der Wert des Objekts an der durch *expr* bestimmten Adresse
- **Typ** des Ausdrucks: T

Adress- und Dereferenzoperator



Eselsbrücke

Die Deklaration

```
T* p;    // p ist vom Typ „Zeiger auf T“
```

kann gelesen werden als

```
T *p;    // *p ist vom Typ T
```



Obwohl das legal ist,
schreiben wir es nicht so!

Null-Zeiger

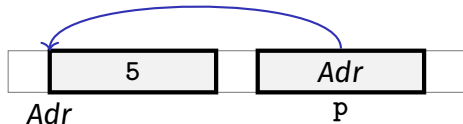
- Spezieller Zeigerwert, der angibt, dass (noch) auf kein Objekt gezeigt wird
- Repräsentiert durch das Literal `nullptr` (konvertierbar nach `T*`)

```
int* p = nullptr;
```

- Kann nicht dereferenziert werden (undefiniertes Verhalten!)
⇒ in Praxis oft Laufzeitfehler
- Dient der Vermeidung undefinierten Verhaltens

```
int* p; // Zugriff auf p ist undefiniertes Verhalten  
int* q = nullptr; // q hingegen zeigt explizit ins Nichts
```


Zeiger-Visualisierungen



- Bisher: Technische präzise Visualisierung *mit Indirektion*
- In p steckt eine Adresse, erst $*p$ gibt uns den Wert/das Objekt, auf den p konzeptionell zeigt

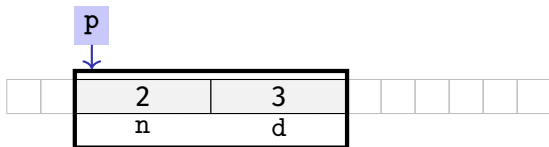


- Neu: Vereinfachte Visualisierung *ohne Indirektion*
- Adresse ist i.d.R. nur „Mittel zum Zweck“; relevant ist, worauf p konzeptionell zeigt

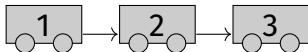
Zeiger auf Klasse

```
struct rational
{
    int n, d;
    rational(int num, int den) : n(num), d(den) {}
};
```

```
rational r = rational(2, 3);
rational* p = &r;
std::cout << (*p).n << (*p).d; // std::cout << p->n << p->d;
```



Wagons: Fortschritt



```
struct wagon {  
    unsigned int id;  
    wagon* next;  
    wagon(unsigned int id) : id(id), next(nullptr) {}  
};
```

```
wagon w1 = wagon(1);  
wagon w2 = wagon(2);  
wagon w3 = wagon(3);  
w1.next = &w2;  
w2.next = &w3;  
std::cout << w1.next->next->id; // outputs 3
```

Funktionen mit/ohne Effekt

- Zeiger können, wie auch Referenzen, für Funktionen mit Effekt verwendet werden.

```
void set_next(wagon* w1, wagon* w2) {  
    w1->next = w2;  
}
```

- Aber viele Funktionen verändern die Daten nicht, sie lesen sie nur
⇒ Verwendung von const

```
void print(const wagon* w) {  
    std::cout << "Wagon " << w->id;  
}
```

Positionierung von Const

Zur Determinierung der Zugehörigkeit des `const` Modifiers:

`const T` ist äquivalent zu `T const` (und kann auch so geschrieben werden):

```
const int zero = ...   ⇔   int const zero = ...  
const int& nil = ...   ⇔   int const& nil = ...
```

Beide Schreibweisen werden in der Praxis genutzt

Const und Zeiger

Lies Deklaration von rechts nach links

<code>int const p1;</code>	p1 ist konstante Ganzzahl
<code>int const* p2;</code>	p2 ist Zeiger auf konstante Ganzzahl (read-only Pointer auf Ganzzahl)
<code>int* const p3;</code>	p3 ist konstanter Zeiger auf Ganzzahl (nicht-veränderbarer Pointer auf eine Ganzzahl)
<code>int const* const p4;</code>	p4 ist konstanter Zeiger auf konstante Ganzzahl (nicht-veränderbarer read-only Pointer auf eine Ganzzahl)

const ist nicht absolut

- Der Wert an einer Adresse kann sich ändern, auch wenn ein Zeiger zu const (read-only pointer) diese Adresse speichert/stores this address.

Beispiel

```
int a;  
const int* p1 = &a;  
int*      p2 = &a;  
*p1 = 1;    // Fehler: *p1 ist const  
*p2 = 1;    // ok, obwohl sich damit auch *p1 ändert
```

- const ist ein Versprechen lediglich aus Sicht des Zeigers (read-only pointer), keine absolute Garantie.

Beliebig viele Wagons

mit n Variablen vom Typ `wagon` können wir nur n Wagons haben...

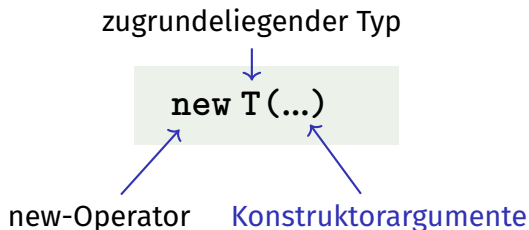
Wie können wir zur Laufzeit mehr Wagons erstellen?

⇒ `new`-Ausdruck!

```
new wagon(4);
```

erstellt zur Laufzeit ein neues Objekt vom Typ `wagon` mit id 4

new-Ausdruck



- **Effekt:** Speicher für ein neues Objekt vom Typ `T` wird alloziert ...
- ...und mit Hilfe des passenden Konstruktors initialisiert
- **Wert:** Adresse des neuen `T`-Objekts, **Typ:** Zeiger `T*`
- Objekt „lebt“ bis es explizit gelöscht wird
 - Mittels `new` allozierter Speicher wird *nicht* automatisch **dealloziert** (= freigegeben), sobald der Pointer auf ihn out of scope geht
 - Zu jedem `new` gehört ein `delete`, das den Speicher explizit freigibt
→ nächste Woche

Beliebig viele Wagons: (partielle) Lösung

```
unsigned int n; // number of wagons
std::cin >> n;
assert(n >= 1);

wagon first = wagon(1);
wagon* last = &first;
for (unsigned int i = 2; i <= n; ++i) {
    wagon* new_wagon = new wagon(i);
    last->next = new_wagon;
    last = new_wagon;
}

std::cout << first.next->id; // outputs 2
std::cout << first.next->next->id; // outputs 3
```

21. Dynamische Datenstrukturen I

Verkettete Liste `std::list<int>` (vereinfacht)

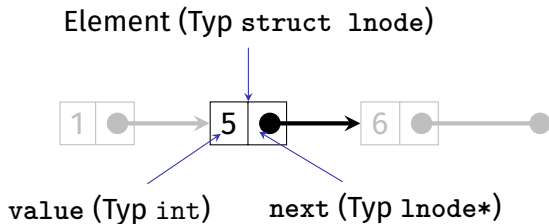
Anderes Speicherlayout: Verkettete Liste

- Kein zusammenhängender Speicherbereich und kein wahlfreier Zugriff
- Jedes Element zeigt auf seinen Nachfolger
- Einfügen und Löschen beliebiger Elemente ist einfach



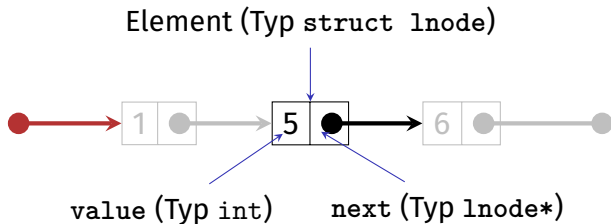
⇒ `std::list<int>` kann als verkettete Liste realisiert werden

Verkettete Liste: Zoom



```
struct lnode {  
    int value;  
    lnode* next;  
  
    lnode(int v, lnode* n): value(v), next(n) {} // Constructor  
};
```

Liste = Zeiger aufs erste Element



```
class our_list {  
    lnode* head;  
public:  
    our_list(unsigned int size);  
    unsigned int size() const;  
    ...  
};
```

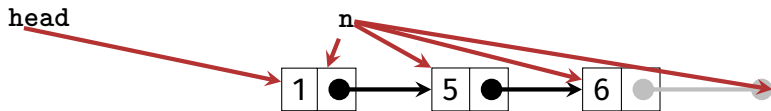
Funktion `our_list::print()`

```
struct lnode {  
    int value;  
    lnode* next;  
    ...  
};
```

```
void our_list::print() const {  
    for (lnode* n = head;  Zeiger auf erstes Element  
        n != nullptr;  Abbruch falls Ende erreicht  
        n = n->next)  Zeiger elementweise voranschieben  
    {  
         Aktuelles Element ausgeben  value << " | ";  
    }  
}
```

Funktion `our_list::print()`

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
    {  
        std::cout << n->value << ' '; // 1 5 6  
    }  
}
```



Funktion `our_list::operator[]`

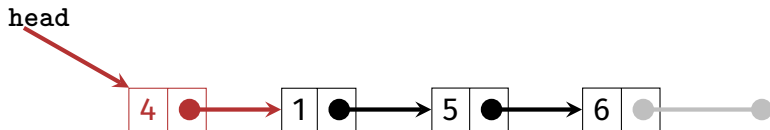
Zugriff auf i -tes Element ähnlich implementiert wie `print()`:

```
int& our_list::operator[](unsigned int i) {  
    lnode* n = head;  Zeiger auf erstes Element  
  
    for (int j = 0; j < i; ++j)  Bis zum  $i$ -ten voranschreiten  
        n = n->next;  
  
    return n->value;   $i$ -tes Element zurückgeben  
}
```

Funktion `our_list::push_front()`

Vorteil `our_list`: Elemente am Anfang anfügen ist sehr einfach:

```
void our_list::push_front(int e) {  
    head =  
        new lnode(e, head);  
}
```



Achtung: Wäre der neue `lnode` nicht *dynamisch* alloziert, dann würde er am Ende von `push_front` sofort wieder gelöscht (= Speicher dealloziert)

Funktion `our_list::our_list()`

Konstruktor kann mittels `push_front()` implementiert werden:

```
our_list::our_list(unsigned int size) {  
    head = nullptr;  head zeigt zunächst ins Nichts  
  
    for (unsigned int i = 0; i < size; ++i)  size mal 0 vorne  
        push_front(0);  anfügen  
}
```

Anwendungsbeispiel:

```
our_list v = our_list(3);  
v.print(std::cout); // 0 0 0
```

Funktion `our_list::push_back()` v1

Einfach, aber ineffizient: Verkettete Liste bis ans Ende traversieren und neues Element anhängen.

```
void our_list::push_back(int e) {  
    lnode* n = head;    
  
    for (; n->next != nullptr; n = n->next);   
  
    n->next =  
        new lnode(e, nullptr)   
}  

```

Beim ersten Element be ... und bis zum letzten Element gehen

Neues Element an bisher letztes anhängen

Was passiert, wenn die Liste leer ist? 🤔

Funktion `our_list::push_back()` v1

Was passiert, wenn die Liste leer ist? → Dann ist `head == nullptr` ...

```
void our_list::push_back(int e) {  
    lnode* n = head;  
  
    for (; n->next != nullptr; n = n->next);  
    ...  
}
```



... und wir dereferenzieren `nullptr` → Undefined Behavior!

Funktion `our_list::push_back()` v2

Finale Version, ohne vorheriges Undefined Behavior:

```
void our_list::push_back(int e) {  
    if (head == nullptr) { // Case 1: empty list  
        head = new lnode(e, nullptr);  
    } else { // Case 2: non-empty list (code as before)  
        lnode* n = head;  
        for (; n->next != nullptr; n = n->next);  
  
        n->next = new lnode(e, nullptr);  
    }  
}
```

Funktion `our_list::push_back()` v2



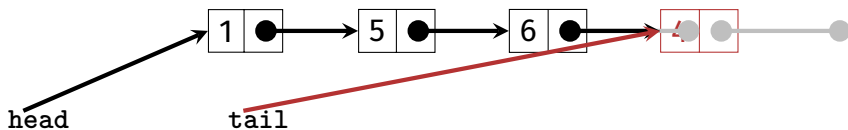
Eine alternative Version, die mit Doppelzeigern arbeitet. Kürzer, aber auch weniger explizit und daher schwerer zu verstehen. Können Sie verstehen, müssen Sie aber nicht.

```
void our_list::push_back(int e) {  
    lnode** target = &(head);  
    for (; *target != nullptr; target = &((*target)->next));  
  
    *target = new lnode(e, nullptr);  
}
```

Funktion `our_list::push_back()`

■ Effizienter, aber auch etwas komplexer:

1. Zweiter Zeiger, der auf das letzte Element zeigt: `tail`
2. Mittels dieses Zeigers kann direkt am Ende angehängt werden



- **Aber:** Verschiedene Grenzfälle, z.B. Vektor noch leer, müssen beachtet werden

Funktion `our_list::size()`

Einfach, aber ineffizient: Grösse durch Abzählen *berechnen*

```
unsigned int our_list::size() const {  
    unsigned int c = 0;   
  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
        ++c;   
  
    return c;   
}
```

The diagram illustrates the execution flow of the `size()` function. It starts with initializing the counter `c` to 0, then enters a loop to traverse the list nodes, incrementing the counter for each node. Finally, it returns the value of the counter.

Funktion `our_list::size()`

Effizienter, aber etwas komplexer: Grösse als Membervariable *nachhalten*

1. Membervariable `unsigned int count` zur Klasse `our_list` hinzufügen
2. `count` muss nun bei *jeder* Operation, die die Grösse des Vektors verändert (z.B. `push_front`), aktualisiert werden

Ein Iterator für `our_list`

Wir brauchen:

1. Einen `our_list`-spezifischen Iterator mit mindestens folgender Funktionalität:
 - Zugriff aktuelles Element: `operator*`
 - Iterator vorwärtssetzen: `operator++`
 - Ende-Erreicht-Prüfung: `operator!=` (oder `operator==`)
2. Memberfunktionen `begin()` und `end()` für `our_list` um einen Iterator auf den Anfang bzw. hinter das Ende zu erhalten

Iterator our_list::iterator (Schritt 1/2)

```
class our_list {  
    ...  
public:  
    class iterator {  
        ...  
    };  
  
    ...  
}
```

- Der Iterator gehört zu unserer Liste, daher ist `iterator` eine öffentliche *innere Klasse* von `our_list`
- Instanzen unseres Iterators sind vom Typ `our_list::iterator`

Iterator our_list::iterator (Schritt 1/2)

```
class iterator {  
    lnode* node;  Zeiger auf aktuelles Vektor-Element  
  
public:  
    iterator(lnode* n);  Erzeuge Iterator auf bestimmtes Element  
    iterator& operator++();  Iterator ein Element vorwärtssetzen  
    int& operator*() const;  Zugriff auf aktuelles Element  
    bool operator!=(const iterator& other) const;   
};  
 Vergleich mit anderem Iterator
```

Iterator our_list::iterator (Schritt 1/2)

// Constructor

```
our_list::iterator::iterator(lnode* n): node(n) ← {}
```

Iterator initial auf `n` zeigen lassen

// Pre-increment

```
our_list::iterator& our_list::iterator::operator++() {  
    assert(node != nullptr);
```

```
    node = node->next; ← Iterator ein Element vorwärtssetzen
```

```
    return *this; ← Referenz auf verschobenen Iterator zurückgeben  
}
```

Iterator our_list::iterator (Schritt 1/2)

```
// Element access
int& our_list::iterator::operator*() const {
    return node->value;  Zugriff auf aktuelles Element
}

// Comparison: when are two iterators not equal?
bool our_list::iterator::operator!=(
    const our_list::iterator& other) const
{
    return node != other.node; 
}

this Iterator verschieden von other falls sie
auf unterschiedliche Elemente zeigen
```

Ein Iterator für `our_list` (Wiederholung)

Wir brauchen:

1. Einen `our_list`-spezifischen Iterator mit mindestens folgender Funktionalität:
 - Zugriff aktuelles Element: `operator*`
 - Iterator vorwärtssetzen: `operator++`
 - Ende-Erreicht-Prüfung: `operator!=` (oder `operator==`)
2. Memberfunktionen `begin()` und `end()` für `our_list` um einen Iterator auf den Anfang bzw. hinter das Ende zu erhalten



Iterator our_list::iterator (Schritt 2/2)

```
class our_list {  
    ...  
public:  
    class iterator {...};  
  
    iterator begin();  
    iterator end();  
  
    ...  
}
```

our_list braucht Memberfunktionen um Iteratoren *auf den Anfang* bzw. *hinter das Ende* des Vektors herausgeben zu können

Iterator our_list::iterator (Schritt 2/2)

```
our_list::iterator our_list::begin() {  
    return our_list::iterator(head);  
}
```

Iterator auf erstes Vektorelement

```
our_list::iterator our_list::end() {  
    return our_list::iterator(nullptr);  
}
```

Iterator hinter letztes Vektorelement

Random Access mit `our_list::operator[]`?

Zugriff auf i -tes Element

```
int& our_list::operator[](unsigned int i) {  
    lnode* n = head;  
  
    for (int j = 0; j < i; ++j)  
        n = n->next;  
  
    return n->value;  
}
```

muss durch die i ersten Elemente iterieren...

Wie erhalten wir Random Access?

22. Dynamische Datenstrukturen II und Speicherverwaltung

Problem

Letzte Woche: Dynamische Datenstruktur `our_list`

Haben dynamischen Speicher angelegt, aber nicht wieder freigegeben.
Insbesondere: keine Funktionen zum Entfernen von Elementen aus `our_list`.

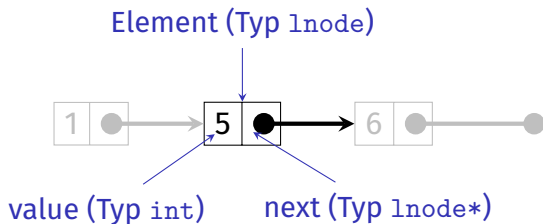
Heute: Korrektes Speichermanagement mit `new` und `delete` mit Hilfe der (einfacheren) dynamischen Datenstruktur `stack`.



Ziel: Stapelklasse mit Speichermanagement

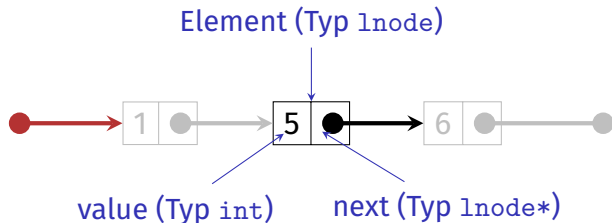
```
class stack {  
public:  
    // post: Element auf den Stapel legen  
    void push(int value);  
    // pre: Stack nicht leer  
    // post: Entfernt oberstes Element vom Stapel  
    void pop();  
    // pre: Stack nicht leer  
    // post: Gibt Wert des obersten Elementes zurück  
    int top() const;  
    // post: gibt zurück, ob Stack leer ist  
    bool empty() const;  
    ...  
};
```

Erinnerung: Verkettete Liste



```
struct lnode {  
    int value;  
    lnode* next;  
    // constructor  
    lnode(int v, lnode* n) : value(v), next(n) {}  
};
```

Stapel = Zeiger aufs oberste Element



```
class stack {  
public:  
    void push(int value);  
    ...  
private:  
    lnode* topn;  
};
```

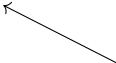

Leerer Stapel

```
class stack {  
public:  
    stack() : topn(nullptr) {} // default constructor  
  
    void push(int value);  
    void pop();  
    int top() const;  
    bool empty() const;  
  
private:  
    lnode* topn;  
}
```

empty(), top()

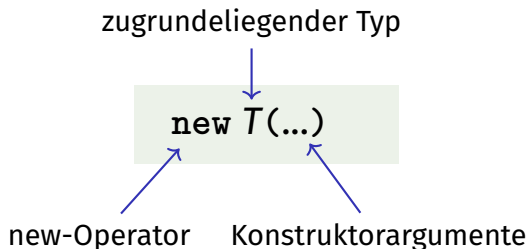
```
// POST: Returns true if stack is empty, false otherwise
bool stack::empty() const {
    return topn == nullptr;
}

// PRE: Stack must be non-empty
// POST: Returns the stack's top value
int stack::top() const {
    assert(!empty());
    return topn->value;
}
```



Erinnerung: Abkürzung für (*topn).value

Erinnerung: der `new`-Ausdruck



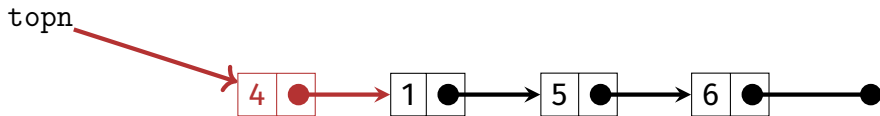
- **Effekt**: Speicher für ein neues Objekt vom Typ T wird alloziert ...
- ...und mit Hilfe des passenden Konstruktors initialisiert
- **Wert**: Adresse des neuen T -Objekt, **Typ**: Zeiger T^* !

Der new-Ausdruck:

push(4)

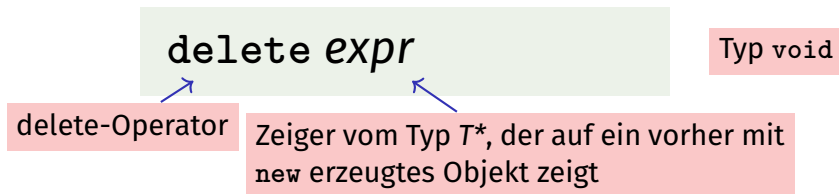
- **Effekt:** Neues Objekt vom Typ T wird im Speicher angelegt ...
- ...und mit Hilfe des passenden Konstruktors initialisiert.
- **Wert:** Adresse des neuen Objekts

```
void stack::push(int value) {  
    topn = new lnode(value, topn);  
}
```



Der delete-Ausdruck

Objekte, die mit `new` erzeugt worden sind, haben dynamische Speicherdauer: sie „leben“, bis sie *explizit gelöscht* werden.



- **Effekt:** Objekt wird dekonstruiert (Erklärung folgt)
... und Speicher wird freigegeben.

Achtung: `delete` setzt den Zeiger *nicht* auf `nullptr`

Wer geboren wird, muss sterben...

Richtlinie „Dynamischer Speicher“

Zu jedem `new` gibt es ein passendes `delete`!

Nichtbeachtung führt zu *Speicherlecks*:

- Ungenutzte Objekte, die den Speicher blockieren...
- ...bis er irgendwann voll ist (**heap overflow**)

Aufpassen mit `new` und `delete`!

```
rational* t = new rational; ← Speicher für t wird angelegt  
rational* s = t; ← Auch andere Zeiger können auf das Objekt zeigen..  
delete s; ← ... und zur Freigabe verwendet werden.  
int nominator = (*t).denominator(); ← Fehler: Speicher freigegeben!
```

↑

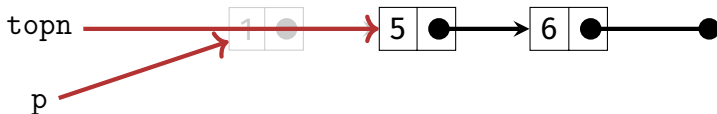
Dereferenzieren eines „dangling pointers“

- Zeiger auf freigegebene Objekte: hängende Zeiger (*dangling pointers*)
- Mehrfache Freigabe eines Objektes mit `delete` ist ein ähnlicher schwerer Fehler.

Weiter mit dem Stapel:

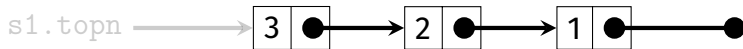
pop()

```
void stack::pop() {  
    assert(!empty());  
    lnode* p = topn;  
    topn = topn->next;  
    delete p;  
}
```



Zombie-Elemente

```
{  
    stack s1; // lokale Variable  
    s1.push(1);  
    s1.push(2);  
    s1.push(3);  
    std::cout << s1; // 3 2 1  
}
```



- `s1` nach dem Codeblock nicht mehr zugreifbar (Stapel ist „gestorben“) ...
- ...aber die drei *Elemente* (1node-Instanzen) des Stapels leben weiter
→ Speicherleck!
- Sie sollten zusammen mit `s1` dealloziert werden

Der Destruktor

- Der Destruktor einer Klasse T ist die eindeutige Memberfunktion mit Deklaration

$\sim T ();$

- Wird automatisch aufgerufen, wenn die Speicherdauer eines Klassenobjekts vom Typ T endet – z.B. bei Aufruf von `delete` auf einem Objekt vom Typ T^* oder wenn der Gültigkeitsbereich eines Objektes vom Typ T endet.
- Falls kein Destruktor deklariert ist, so wird er automatisch erzeugt und ruft die Destruktoren für die Membervariablen auf (Zeiger `topn`, kein Effekt – Grund für Zombie-Elemente)

Mit dem Destruktor klappt's!

```
// POST: the dynamic memory of *this is deleted
stack::~~stack() {
    while (topn != nullptr) {
        lnode* t = topn;
        topn = t->next;
        delete t;
    }
}
```

- löscht automatisch alle Stapелеlemente, wenn der Stapel ungültig wird
- Unsere Stapel-Klasse scheint jetzt die Richtlinie „Dynamischer Speicher“ zu befolgen (?)

Stapel fertig?

Offenbar noch nicht...

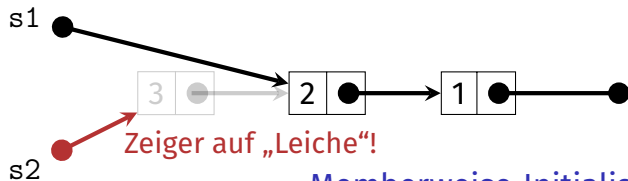
```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n"; // 3 2 1
```

```
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1
```

```
s1.pop();  
std::cout << s1 << "\n"; // 2 1
```

```
s2.pop(); // Oops, Programmabsturz!
```

Was ist hier schiefgegangen?



Memberweise Initialisierung: kopiert nur
den topn-Zeiger

```
...  
stack s2 = s1;   
std::cout << s2 << "\n"; // 3 2 1  
  
s1.pop();  
std::cout << s1 << "\n"; // 2 1  
  
s2.pop(); // Oops, Programmabsturz!
```

Das eigentliche Problem

Schon das geht schief:

```
{  
    stack s1;  
    s1.push(1);  
    stack s2 = s1;  
}
```

Beim Verlassen des Gültigkeitsbereiches werden beide Stacks aufgeräumt (dekonstruiert). Aber beide Stacks versuchen dieselben Daten zu löschen, denn sie haben Zugriff auf denselben Zeiger.

Lösungsmöglichkeiten

1. **Wir erstellen eine echte Kopie aller Daten (deep copy).**

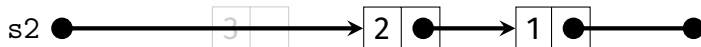
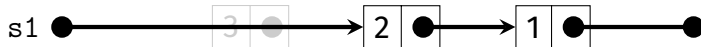
oder

2. Wir erlauben keine "Kopien".

oder

3. Wir lösen obiges Problem der gemeinsam genutzten Datenstruktur (hier nicht weiter diskutiert).

1. Wir erstellen eine echte Kopie!



```
...  
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1  
  
s1.pop();  
std::cout << s1 << "\n"; // 2 1  
  
s2.pop(); // ok
```


Der Copy-Konstruktor

- Der Copy-Konstruktor einer Klasse T ist der eindeutige Konstruktor mit Deklaration

$T(\text{const } T\& x);$

- wird automatisch aufgerufen, wenn Werte vom Typ T mit Werten vom Typ T *initialisiert* werden

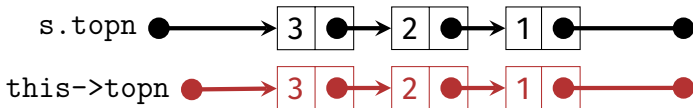
$T\ x = t;$ (t vom Typ T)

$T\ x(t);$

- Falls kein Copy-Konstruktor deklariert ist, so wird er automatisch erzeugt (und initialisiert memberweise – Grund für obiges Problem)

Mit dem Copy-Konstruktor klappt's!

```
// POST: *this is initialized with a copy of s
stack::stack(const stack& s) : topn (nullptr) {
    if (s.topn == nullptr) return;
    topn = new lnode(s.topn->value, nullptr);
    lnode* prev = topn;
    for (lnode* n = s.topn->next; n != nullptr; n = n->next) {
        lnode* copy = new lnode(n->value, nullptr);
        prev->next = copy;
        prev = copy;
    }
}
```



prev

NB: rekursives Kopieren

```
lnode* copy(node* that){  
    if (that == nullptr) return nullptr;  
    return new lnode(that->value, copy(that->next));  
}
```

Elegant, oder? Warum haben wir das nicht gleich so gemacht?

Grund: verkettete Listen können sehr lang werden. Dann könnte `copy` zum Stapelüberlauf⁵ führen. Aufrufstapel ist nämlich meist kleiner als Heapspeicher.

⁵nicht von dem Stapel, den wir gerade implementieren, sondern vom Aufrufstapel der Rekursion

Initialisierung $\neq$ Zuweisung!

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n"; // 3 2 1
```

```
stack s2;  
s2 = s1; // Zuweisung
```

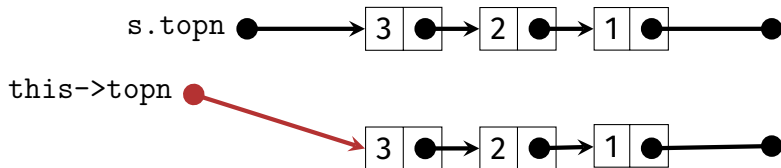
```
s1.pop();  
std::cout << s1 << "\n"; // 2 1  
s2.pop(); // Oops, Programmabsturz!
```

Der Zuweisungsoperator

- Überladen von `operator=` als Memberfunktion
- Wie Copy-Konstruktor ohne Initialisierer, aber zusätzlich
 - Prüfen auf Selbstzuweisung (`s1 = s1`), die keinen Effekt haben sollten
 - Freigeben des Speichers des bisherigen (jetzt alten) Inhalts
- Falls kein Zuweisungsoperator deklariert ist, so wird er automatisch erzeugt (und weist memberweise zu – Grund für obiges Problem)

Mit dem Zuweisungsoperator klappt's!

```
// POST: *this (left operand) becomes a copy of s (right operand)
stack& stack::operator=(const stack& s){
    if (this != &s) { // keine Selbstzuweisung
        stack copy = s; // Kopierkonstruktor
        std::swap(topn, copy.topn); // copy hat nun den Müll!
    } // copy wird aufgeräumt -> Dekonstruktion
    return *this; // Rückgabe als L-Wert (Konvention)
}
```



Copy & Swap-Idiom: Coooooler Trick! 😊

Fertig

```
class stack {  
public:  
    stack(); // constructor  
    ~stack(); // destructor  
    stack(const stack& s); // copy constructor  
    stack& operator=(const stack& s); // assignment operator  
  
    void push(int value);  
    void pop();  
    int top() const;  
    bool empty() const;  
private:  
    lnode* topn;  
};
```

Dreierregel (Rule of Three)

Typ, der dynamischen Speicher verwaltet (z.B. unsere Klasse für Stapel), braucht:

- Konstruktoren
 - Destruktor
 - Copy-Konstruktor
 - Zuweisungsoperator
- } Dreierregel: definiert eine Klasse eines davon, so muss sie auch die anderen zwei definieren!

Lösungsmöglichkeiten

1. Wir erstellen eine echte Kopie aller Daten (deep copy).

oder

2. **Wir erlauben keine "Kopien".**

oder

3. Wir lösen obiges Problem der gemeinsam genutzten Datenstruktur (hier nicht weiter diskutiert).

2. Stapel ohne Kopien!

```
class stack {  
public:  
    stack(); // constructor  
    ~stack(); // destructor  
    stack(const stack& s)=delete; // no copy constructor  
    stack& operator=(const stack& s)=delete; // no assignment operator  
    ...  
};
```

```
stack s1;  
stack s2;  
s2 = s1 // compiler error: use of deleted function
```

```
stack s1;  
stack s3 = s1; // compiler error: use of deleted function
```

Lösungsmöglichkeiten

1. Wir erstellen eine echte Kopie aller Daten (deep copy).

oder

2. Wir erlauben keine "Kopien".

oder

3. **Wir lösen obiges Problem der gemeinsam genutzten Datenstruktur (hier nicht weiter diskutiert).**

Smart Pointers

Smart Pointers sind Pointers mit zusätzlicher Funktionalität, die das Memory Management für uns übernehmen.

`std::shared_ptr<T>`

zählt die Anzahl Zeiger, die auf ein Objekt verweisen. Löscht Objekt nur wenn diese Anzahl auf 0 zurückfällt.

`std::unique_ptr<T>`

Verbietet, dass mehrere Zeiger auf ein Objekt zeigen können.

Shared Pointers: Beispiel

■ Shared Pointer erstellen

```
// create shared pointer with std::make_shared<T>
// std::make_shared<T> takes same arguments as constructor of T
#include <memory>
std::shared_ptr<rational> sp1 = std::make_shared<rational>(3, 4);
```

■ Handling des Shared Pointers wie mit normalem Pointer

```
// dereferencing
rational r = *sp1;
// assignment
std::shared_ptr<rational> sp2 = sp1;
// member access
std::cout << sp2->n << " / " << sp2->d; // 3 / 4
```

Shared Pointers: Beispiel

- Es wird mitgezählt, wie viele Shared Pointer auf das Objekt zeigen

```
std::shared_ptr<rational> sp1 = std::make_shared<rational>(3, 4);  
std::shared_ptr<rational> sp2 = sp1;  
std::cout << sp1.use_count(); // 2
```

- Wenn ein Shared Pointer out of scope geht, wird Counter dekrementiert

```
{  
    std::shared_ptr<rational> sp3 = sp2;  
    std::cout << sp1.use_count(); // 3  
} // sp3 goes out of scope here; counter is decreased  
sp1 = nullptr; // explicitly remove pointer sp1 to object  
std::cout << sp2.use_count(); // 1
```

- wenn Counter 0 wird, wird Objekt destruiert und Speicher freigegeben

Anwendungsbeispiel: Werkstatt



Arbeiter arbeiten in Werkstatt.
Sie benutzen regelmässig gewisse Werkzeuge und *teilen* sich diese.
Ungebrauchte Werkzeuge sollen automatisch zurückgegeben werden

Werkstatt: 1. Versuch

```
struct tool {  
    std::string name;  
    tool(std::string name) : name(name) {}  
    ~tool() {std::cout << name << " gets returned!\n";}  
};
```

```
struct worker {  
    std::map<std::string, tool> tools; // all tools a worker is using  
    void use(tool t); // acquire  
    tool offer(std::string tool_name); // share  
    void release(std::string tool_name); // release  
};
```


Werkstatt: 1. Versuch

```
void worker::use(tool t) {  
    tools[t.name] = t;  
}  
tool worker::offer(std::string tool_name) {  
    return tools[tool_name];  
}  
void worker::release(std::string tool_name) {  
    tools.erase(tool_name);  
}  
  
tool borrow(std::string tool_name) {  
    return tool(tool_name);  
}
```

Werkstatt: 1. Versuch

```
worker w1;  
worker w2;  
w1.use(borrow("hammer"));  
w2.use(w1.offer("hammer"));  
w2.release("hammer");
```

hammer gets returned

hammer gets returned

hammer gets returned

hammer gets returned

Jeder Worker hat seine eigene Kopie des Werkzeugs: kein Teilen!
(Noch schlimmer, wir haben hier 4 Kopien des Hammers!)

Werkstatt: Versuch mit Zeiger

```
struct tool {  
    std::string name;  
    tool(std::string name) : name(name) {}  
    ~tool() {std::cout << name << " gets returned!\n";}  
};
```

```
using tool_ptr = tool*;
```

```
struct worker {  
    std::map<std::string, tool_ptr> tools; // all tools a worker is using  
    void use(tool_ptr t); // acquire  
    tool_ptr offer(std::string tool_name); // share  
    void release(std::string tool_name); // release  
};
```

Werkstatt: Versuch mit Zeiger

```
void worker::use(tool_ptr t) {  
    tools[t->name] = t;  
}  
tool_ptr worker::offer(std::string tool_name) {  
    return tools[tool_name];  
}  
void worker::release(std::string tool_name) {  
    tools.erase(tool_name);  
}  
  
tool_ptr borrow(std::string tool_name) {  
    return new tool(tool_name);  
}
```

Werkstatt: Versuch mit Zeiger

```
worker w1;  
worker w2;  
w1.use(borrow("hammer"));  
w2.use(w1.offer("hammer"));  
w2.release("hammer");
```

Wer baut Speicher für Werkzeuge ab, wenn sie nicht mehr in Gebrauch sind?

Memory Leak!

Werkstatt: Lösung mit Shared Pointers

```
struct tool {  
    std::string name;  
    tool(std::string name) : name(name) {}  
    ~tool() {std::cout << name << " gets returned!\n";}  
};
```

```
using tool_ptr = std::shared_ptr<tool>;
```

```
struct worker {  
    std::map<std::string, tool_ptr> tools; // all tools a worker is using  
    void use(tool_ptr t); // acquire  
    tool_ptr offer(std::string tool_name); // share  
    void release(std::string tool_name); // release  
};
```

Werkstatt: Lösung mit Shared Pointers

```
void worker::use(tool_ptr t) {  
    tools[t->name] = t;  
}  
tool_ptr worker::offer(std::string tool_name) {  
    return tools[tool_name];  
}  
void worker::release(std::string tool_name) {  
    tools.erase(tool_name);  
}  
  
tool_ptr borrow(std::string tool_name) {  
    return std::make_shared<tool>(tool_name);  
}
```

Werkstatt: Lösung mit Shared Pointers

```
worker w1;  
worker w2;  
w1.use(borrow("hammer"));  
w2.use(w1.offer("hammer"));  
w2.release("hammer");
```

hammer gets returned!

Sobald der letzte Shared Pointer auf ein Werkzeug out of scope geht, wird der Speicher (genau einmal!) abgebaut.

Unique Pointers: Beispiel

■ Unique Pointer erstellen

```
std::unique_ptr<rational> up = std::make_unique<rational>(3, 4);
```

■ Handling des Unique Pointers wie mit normalem Pointer

```
rational r = *up;  
std::cout << up->n << " / " << up->d; // 3 / 4
```

■ Zuweisung nicht erlaubt...

```
std::unique_ptr<rational> x = up; // compiler error
```

■ ...nur in Kombination mit Übertragen des Besitzes

```
std::unique_ptr<rational> x = std::move(up) // transfer ownership
```

Unique Pointer ist alleiniger Besitzer des Objekts, auf das er zeigt;
Objekt wird zerstört, wenn der Besitzer out of scope geht.

Anwendungsbeispiel: Hotdesk-Büro



Arbeiter beziehen bei Ankunft Gegenstände wie Schreibtisch und Computer.
Sie teilen sie nicht.
Unbenutzte sollen automatisch freigegeben werden.

Erste Versuche

```
struct item {  
    std::string name;  
    item(std::string name) : name(name) {};  
    ~item() {std::cout << name << " gets returned!\n";}   
};  
  
struct worker {  
    std::map<std::string, item*> items; // all items a worker is using  
    void use(item* t); // acquire  
    item* release(std::string item_name); // release  
};
```

- ohne Pointer $\Rightarrow$ Kein Teilen der Gegenstände: Kopien werden erstellt!
- mit Raw Pointer $\Rightarrow$ Gegenstände werden nie zurückgegeben

Versuch mit Shared Pointers

```
struct item {  
    std::string name;  
    item(std::string name) : name(name) {};  
    ~item() {std::cout << name << " gets returned!\n";}  
};
```

```
using item_ptr = std::shared_ptr<item>;
```

```
struct worker {  
    std::map<std::string, item_ptr> items; // all items a worker is using  
    void use(item_ptr t); // acquire  
    item_ptr release(std::string item_name); // release  
};
```

Versuch mit Shared Pointers

```
void worker::use(item_ptr t){  
    items[t->name] = t;  
}
```

```
item_ptr worker::release(std::string item_name) {  
    item_ptr t = items[item_name];  
    items.erase(item_name);  
    return t;  
}
```

```
item_ptr claim(std::string item_name) {  
    return std::make_shared<item>(item_name);  
}
```

Versuch mit Shared Pointers

```
worker w1;  
worker w2;  
auto desk = claim("desk");  
w1.use(desk);  
w2.use(desk);
```

desk gets returned!

Schreibtisch wird zwar korrekt freigegeben, aber mehrere Arbeiter können sich einen Schreibtisch teilen.

Hotdesk-Büro: Lösung mit Unique Pointers

```
struct item {  
    std::string name;  
    item(std::string name) : name(name) {};  
    ~item() {std::cout << name << " gets returned!\n";}  
};
```

```
using item_ptr = std::unique_ptr<item>;
```

```
struct worker {  
    std::map<std::string, item_ptr> items; // all items a worker is using  
    void use(item_ptr t); // acquire  
    item_ptr release(std::string item_name); // release  
};
```

Hotdesk-Büro: Lösung mit Unique Pointers

```
void worker::use(item_ptr t){  
    items[t->name] = std::move(t);  
}
```

```
item_ptr worker::release(std::string item_name) {  
    item_ptr t = std::move(items[item_name]);  
    items.erase(item_name);  
    return t;  
}
```

```
item_ptr claim(std::string item_name) {  
    return std::make_unique<item>(item_name);  
}
```


Hotdesk-Büro: Lösung mit Unique Pointers

```
worker w1;  
worker w2;  
w1.use(claim("desk"));  
w2.use(w1.release("desk"));
```

desk gets returned!

Sobald der Unique Pointer auf ein Objekt (der alleinige Besitzer eines Objekts) out of scope geht, wird der Speicher (genau einmal!) abgebaut.

Speicherverwaltung unabhängig von C++

Garbage Collector (GC): in anderen Programmiersprachen übernimmt der GC das Entfernen alter Daten

- Problem 1: nicht voraussagbare Latenz wenn gestartet
- Problem 2: Speicher nicht sofort frei, erst nach folgendem GC Lauf

Datenstrukturen mit geteilten Objekten brauchen Verwaltung

- Beispiel: ein Knoten wird in zwei stacks benutzt – beim Ändern des Knotenwert verändern sich beide Stacks

Andere Sprachen haben also auch Speicherverwaltung, verstecken diese nur (soweit möglich). Bessere Kontrolle in C++!

23. Dynamische Datenstrukturen III

Wiederholung: `vector<T>`

- Kann mit beliebiger Grösse `n` initialisiert werden
- Unterstützt diverse Operationen:

```
e = v[i];           // Get element
v[i] = e;           // Set element
l = v.size();        // Get size
v.push_back(e);      // Append element
...
```

- Ein Vektor ist eine *dynamische Datenstruktur*, deren Grösse sich zur Laufzeit ändern kann

Unser eigener Vektor!

Wir implementieren unseren eigenen Vektor: `vec<int>`

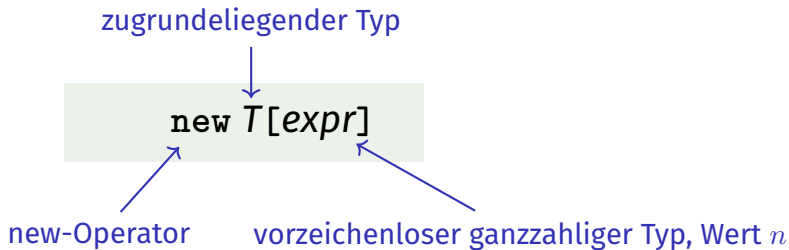
Vektoren im Speicher

Bereits bekannt: Ein Vektor belegt einen *zusammenhängenden* Speicherbereich



Frage: Wie *alloziert* (reserviert) man einen Speicherblock *beliebiger* Grösse zur Laufzeit, also *dynamisch*?

Der new-Ausdruck für Arrays

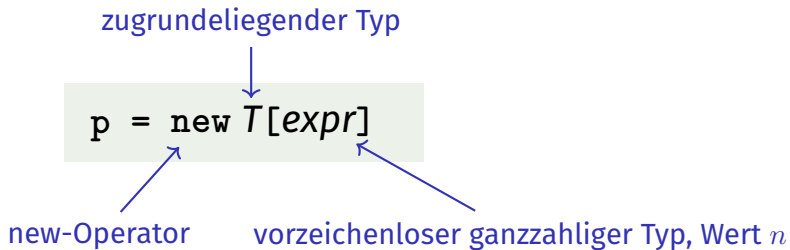


- **Effekt:** Neuer zusammenhängender Bereich im Speicher für n Elemente vom Typ T wird alloziert



- Dieser Speicherbereich wird **Array** (der Länge n) genannt

Der new-Ausdruck für Arrays



- **Wert:** Die Startadresse des Speicherbereichs



- **Typ:** Ein Zeiger T^*

Zeiger-Arithmetik: Zeiger plus int

```
T* p = new T[n]; // p points to first array element
```

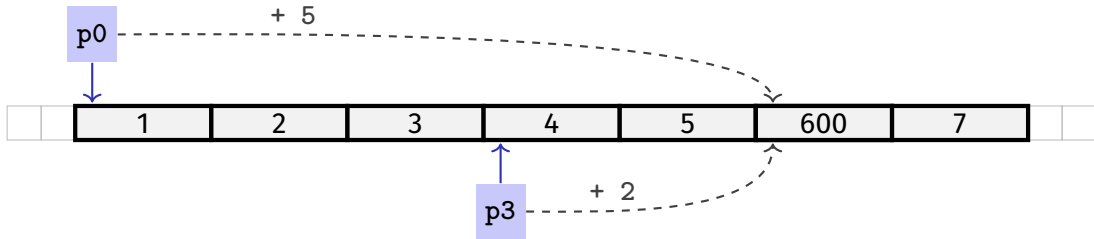


Frage: Wie zeigt man auf hintere Elemente? → mittels *Zeiger-Arithmetik*:

- p gibt die Adresse des *ersten* Array-Elements, $*p$ dessen Wert
- $*(p + i)$ gibt den Wert des *i-ten* Array-Elements, für $0 \leq i < n$
- $*p$ ist äquivalent zu $*(p + 0)$

Zeiger-Arithmetik: Zeiger plus int

```
int* p0 = new int[7]{1,2,3,4,5,6,7};    // p0 points to 1st element
int* p3 = p0 + 3;    // p3 points to 4th element
*(p3 + 2) = 600;    // set value of 6th element to 600
std::cout << *(p0 + 5);    // output 6th element's value (i.e., 600)
```

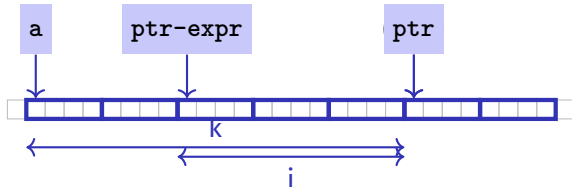


Zeiger-Arithmetik: Zeiger minus `int`

- Wenn ptr ein Zeiger auf das Element mit Index k in einem Array a der Länge n ist
- und der Wert von $expr$ eine ganze Zahl i ist, $0 \leq k - i \leq n$,
dann liefert der Ausdruck

$ptr - expr$

einen Zeiger zum Element von a mit Index $k - i$.



Zeigersubtraktion

- Wenn $p1$ und $p2$ auf Elemente desselben Arrays a mit Länge n zeigen
- und $0 \leq k_1, k_2 \leq n$ die Indizes der Elemente sind, auf die $p1$ und $p2$ zeigen, so gilt

$p1 - p2$ hat den Wert $k_1 - k_2$



Nur gültig, wenn $p1$ und $p2$ ins gleiche Array zeigen.

- Die Zeigerdifferenz beschreibt, wie weit die Elemente voneinander entfernt im Speicher liegen

Beschreibung	Op	Stelligkeit	Präzedenz	Assoziativität	Zuordnung
Subskript	[]	2	17	links	R-Werte → L-Wert
Dereferenzierung	*	1	16	rechts	R-Wert → L-Wert
Adresse	&	1	16	rechts	L-Wert → R-Wert

Präzedenzen und Assoziativitäten von +, -, ++ (etc.) wie in Kapitel 2

Zeigerwerte sind keine Ganzzahlen

- Adressen können als „Hausnummern des Speichers“, also als Zahlen interpretiert werden.
- Ganzzahl- und Zeigerarithmetik verhalten sich aber unterschiedlich.

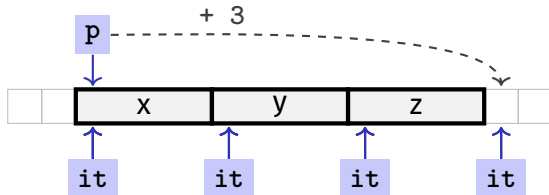
`ptr + 1` ist nicht die nächste Hausnummer, sondern die *s*-nächste, wobei *s* der Speicherbedarf eines Objekts des Typs ist, der `ptr` zugrundeliegt.

- Zeiger und Ganzzahlen sind nicht kompatibel:

```
int* ptr = 5; // Fehler: invalid conversion from int to int*  
int a = ptr;  // Fehler: invalid conversion from int* to int
```

Sequenzielle Zeiger-Iteration

```
char* p = new char[3]{'x', 'y', 'z'};
```



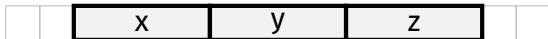
```
for (char* it = p;   
    it != p + 3;   
    ++it) {   
    std::cout << *it << ' ';   
}
```

Annotations for the code:

- `it = p`: it zeigt aufs erste Element
- `it != p + 3`: Abbruch falls Ende erreicht
- `++it`: Zeiger elementweise voranschieben
- `*it`: Aktuelles Element ausgeben: 'x'

Wahlfreier Zugriff auf Arrays

```
char* p = new char[3]{'x', 'y', 'z'};
```



- Der Ausdruck $*(p + i)$
- kann auch geschrieben werden als $p[i]$
- z.B. $p[1] == *(p + 1) == 'y'$

Wahlfreier Zugriff auf Arrays

Iteration über ein Array mittels Indizes und *wahlfreiem Zugriff*:

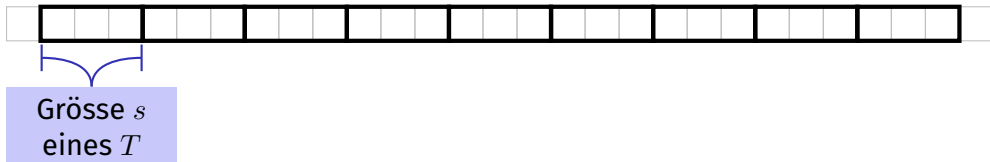
```
char* p = new char[3]{'x', 'y', 'z'};

for (int i = 0; i < 3; ++i)
    std::cout << p[i] << ' ';
```

Aber: Dies ist weniger *effizient* als der vorher gezeigte *sequenzielle* Zugriff mittels Zeiger-Iteration

Wahlfreier Zugriff auf Arrays

```
T* p = new T[n];
```



- Zugriff $p[i]$, also $*(p + i)$, „kostet“ Berechnung $p + i \cdot s$
- Iteration mittels *wahlfreiem Zugriff* ($p[0], p[1], \dots$) kostet eine Addition und eine Multiplikation pro Zugriff
- Iteration mittels *sequenziellem Zugriff* ($++p, ++p, \dots$) kostet nur eine Addition pro Zugriff
- Sequenzieller Zugriff ist daher für Iterationen zu bevorzugen

Ein Buch lesen

Wahlfreier Zugriff

- öffne Buch auf S.1
- klappe Buch zu
- öffne Buch auf S.2-3
- klappe Buch zu
- öffne Buch auf S.4-5
- klappe Buch zu
-

Sequenzieller Zugriff

- öffne Buch auf S.1
- blättere um
- blättere um
- blättere um
- blättere um
- blättere um
- ...

- `int* p = new int[expr]` erzeugt ein dynamisches Array der Grösse *expr*
- C++ hat noch *statische* Arrays von der Vorgängersprache C geerbt:
`int a[cexpr]`
- Statische Arrays haben unter anderem den Nachteil, dass ihre Grösse *cexpr* eine Konstante sein muss. D.h. *cexpr* kann z.B. 5 oder $4*3+2$ sein, aber kein von der Tastatur eingelesener Wert *n*.
- Eine statisches Array-Variable *a* kann wie ein Zeiger verwendet werden
- Faustregel: Vektoren sind besser als dynamische Arrays, welche besser als statische Arrays sind

Arrays in Funktionen

Konvention in C++: Übergabe eines Arrays (oder eines Array-Ausschnitts) mit zwei Zeigern



- `begin`: Zeiger auf das erste Element
- `end`: Zeiger *hinter* das letzte Element
- `[begin, end)` bezeichnet die Elemente des Array-Ausschnitts
- `[begin, end)` ist leer, wenn `begin == end`
- `[begin, end)` muss ein *gültiger Bereich* sein, d.h. ein echter (evtl. leerer) Array-Ausschnitt

Arrays in (mutierenden) Funktionen: fill

```
// PRE: [begin, end) ist ein gültiger Bereich  
// POST: Jedes Element in [begin, end) wurde auf value gesetzt  
void fill(int* begin, int* end, int value) {  
    for (int* p = begin; p != end; ++p)  
        *p = value;  
}
```

```
int* p = new int[5];  
fill(p, p+5, 1); // Array bei p wird zu {1, 1, 1, 1, 1}
```

Array-basierter Vektor

- Vektoren ... da war doch was 🤔
- Nun wissen wir, wie man Speicherblöcke beliebiger Grösse allozieren kann ...
- ...und können einen Vektor, auf einem solchen Speicherblock aufbauend, implementieren
- `our_vector` – ein Array-basierter Vektor für `int`-Elemente

Unser eigener Vektor!

Wir implementieren unseren eigenen Vektor: `vec<int>`

Array-basierter Vektor `our_vector`: Klassensignatur

```
class our_vector {  
    // Private (internal) state:  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
  
public: // Public interface:  
    our_vector(unsigned int size); // Constructor  
    unsigned int size() const; // Size of vector  
    int& operator[](int i); // Access an element  
    void print(std::ostream& sink) const; // Output elements  
};
```


Konstruktor `our_vector::our_vector()`

```
our_vector::our_vector(unsigned int size)
    : count(size) ← { Grösse speichern

    elements = new int[size]; ← Speicher allozieren
}
```

Achtung: Vektor wurde nicht initialisiert (mit einem Standardwert, z.B. 0)

Funktion `our_vector::size()`

```
int our_vector::size() const {  
    return count;  
}
```



Verändert den Vektor nicht

Grösse zurückgeben

Anwendungsbeispiel:

```
our_vector v = our_vector(7);  
assert(v.size() == 7); // ok
```

Funktion `our_vector::operator[]`

```
int& our_vector::operator[](int i) {  
    return elements[i];  
}
```



i-tes Element zurückgeben

Elementzugriff mit Indexüberprüfung:

```
int& our_vector::at(int i) {  
    assert(0 <= i && i < count);  
    return elements[i];  
}
```

Anwendungsbeispiel:

```
our_vector v = our_vector(7);  
v[6] = 0;  
std::cout << v[6]; // Outputs 0
```

Funktion `our_vector::operator[]` braucht's doppelt

```
int& our_vector::operator[](int i) {return elements[i];}  
const int& our_vector::operator[](int i) const {return elements[i];}
```

- Die erste Memberfunktion ist *nicht const* und gibt eine *nicht-const*-Referenz zurück

```
our_vector v = ...; // A non-const vector  
std::cout << v.get[0]; // Reading elements is allowed  
v.get[0] = 123; // Modifying elements is allowed
```

- Sie wird auf nicht-const-Vektoren aufgerufen

Funktion `our_vector::operator[]` braucht's

doppelt

```
int& our_vec::operator[](int i) {return elements[i];}  
const int& our_vec::operator[](int i) const {return elements[i];}
```

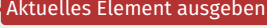
- Die zweite Memberfunktion *ist const* und gibt eine *const*-Referenz zurück

```
const our_vec v = ...; // A const vector  
std::cout << v.get[0]; // Reading elements is allowed  
v.get[0] = 123; // Compiler error: modifications are not allowed
```

- Sie wird auf const-Vektoren aufgerufen

Funktion `our_vector::print()`

Elemente mittels sequenziellem Zugriff ausgeben:

```
void our_vector::print(std::ostream& sink) const {  
    for (int* p = elements;     
        p != elements + count;   
        ++p)    
    {  
        sink << *p << ' ' ;    
    }  
}
```

Funktion `our_vector::print()`

Abschluss: Ausgabeoperator überladen:

```
std::ostream& operator<<(std::ostream& sink,  
                        const our_vector& vec) {  
    vec.print(sink);  
    return sink;  
}
```

Beobachtungen:

- Konstante Referenz auf `vec`, da unverändert
- Aber nicht auf `sink`: Elemente ausgeben gleich Veränderung
- `sink` zurückgeben, um verkettete Ausgaben zu ermöglichen, z.B.
`std::cout << v << '\n';`

Ein Iterator für `our_vector`

Klasse mit

- `operator*`
- `operator++`
- `operator!=`

Im `our_vector` haben wir bereits einen Pointer (`elements`) vom Typ `int*` mit allen Funktionalitäten, die ein Iterator benötigt (Pointer sind Iteratoren).

Wir brauchen nur noch Memberfunktionen `begin()` und `end()` im `our_vector`, um einen Iterator auf den Anfang bzw. hinter das Ende zu erhalten

Iterator für our_vector

```
class our_vector {  
    int* elements;  
    unsigned int count;  
public:  
    using iterator = int*; // our_vector::iterator stands for int*  
    ...  
    iterator begin() {  
        return elements;   
    }  
    iterator end() {  
        return elements + count;   
    }  
    ...  
}
```

our_vector Iterator Verwendung

Funktionalität, die auf Iteratoren beruht, kann auf unserem Vektor verwendet werden

```
our_vector v;  
...  
for(int& i: v){ // iterator used implicitly  
    i += 3;  
}  
int max = *std::max_element(v.begin(), v.end());  
...
```

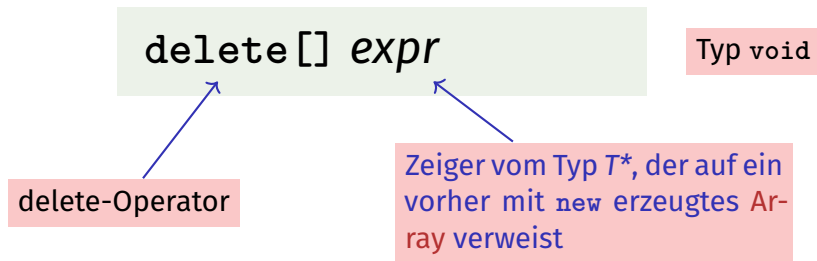
Was passiert hier?

```
class our_vector {  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
public:  
    our_vector(unsigned int size); // Constructor  
    unsigned int size() const; // Size of vector  
    int& operator[](int i); // Access an element  
    void print(std::ostream& sink) const; // Output elements  
};
```

```
for (unsigned int i = 0; i < 1000000; ++i) {  
    our_vector v = our_vector(1000000);  
}
```

Vektoren gehen out of scope und werden abgebaut (inklusive der Zeiger auf die Elemente), aber Elemente (jeweils eine Million viele) leben weiter... 683

Der delete-Ausdruck für Arrays



■ **Effekt:** Array wird gelöscht, Speicher wird wieder freigegeben

Achtung: `delete[]` setzt den Zeiger *nicht* auf `nullptr`

Vektor: delete[] im Destruktor

- zum new[] im Konstruktor...

```
our_vector::our_vector(unsigned int size) : count(size) {  
    elements = new int[size];  
}
```

- ...braucht es ein delete[] im Destruktor

```
our_vector::~~our_vector(){  
    delete[] elements;  
}
```

Was passiert hier?

```
{  
    our_vector v = our_vector(10);  
    our_vector w = v;  
}
```

⇒ Elemente des Vektors werden zweimal abgebaut...

```
{  
    our_vector v = our_vector(10);  
    our_vector w;  
    w = v;  
}
```

⇒ hier ebenso...

Dreierregel (Rule of Three)...

```
class our_vector {  
    // Private (internal) state:  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
public: // Public interface:  
    our_vector(unsigned int size); // Constructor  
    our_vector(const our_vector& v); // Copy constructor  
    ~our_vector(); // Destructor  
    operator=(const our_vector& v); // Assignment operator  
    unsigned int size() const; // Size of vector  
    int& operator[](int i); // Access an element  
    void print(std::ostream& sink) const; // Output elems.  
}
```

Kopierkonstruktor und Zuweisungsoperator

```
our_vector::our_vector(const our_vector& v): count(v.count) {  
    elements = new int[count];  
    for (unsigned int i = 0; i < count; ++i) {  
        elements[i] = v.elements[i];  
    }  
}
```

```
our_vector& our_vector::operator=(const our_vector& v) {  
    if(this != &v) { // no self assignment  
        auto copy = v;  
        std::swap(elements, copy.elements);  
        std::swap(count, copy.count);  
    }  
    return *this;  
}
```


Alternative: Smart Pointer

- Member: `std::unique_ptr<int[]>` statt `int* elements`

```
class our_vector {  
    std::unique_ptr<int[]> elements;  
    ...  
};
```

- Konstruktor: `std::make_unique<int[]>(size)` statt `new int[size]`

```
our_vector::our_vector(unsigned int size) : count(size) {  
    elements = std::make_unique<int[]>(size);  
}
```

- kein `delete[]` nötig im Destruktor (automatisch erzeugter Destruktor genügt)

Alternative: Smart Pointer

- Kopier-Konstruktor: `std::make_unique` statt `new[]`

```
our_vector::our_vector(const our_vector& v): count(v.count) {  
    elements = std::make_unique<int[]>(count);  
    ...  
}
```

- Zuweisungsoperator mit Copy & Swap-Idiom wie gehabt
- Raw Pointer extrahieren für Iterator mit `elements.get()`

```
our_vector::iterator our_vector::begin() {  
    return our_vector::iterator(elements.get());  
}  
our_vector::iterator our_vector::end() {  
    return our_vector::iterator(elements.get() + count);  
}
```

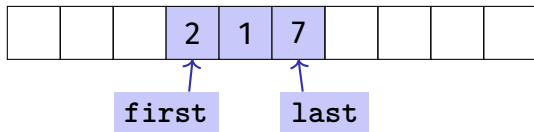
Elemente hinzufügen und löschen?

```
class our_vector {  
    ...  
    void push_front(int e)      // Prepend e to vector  
    void push_back(int e)      // Append e to vector  
    void remove(unsigned int i) // Cut out ith element  
    ...  
}
```

Gemeinsamkeit: Diese Operationen müssen die *Grösse* des Vektors verändern

Arrays vergrössern/verkleinern

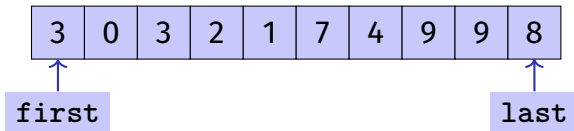
Ein allozierter Speicherblock (z.B. `new int [3]`) kann nicht nachträglich vergrössert/verkleinert werden



Möglichkeit:

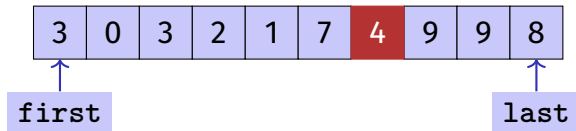
- Mehr Speicher als initial nötig allozieren
- Befüllen aus der Mitte heraus, mittels Zeigern auf erstes und letztes Element

Arrays vergrössern/verkleinern

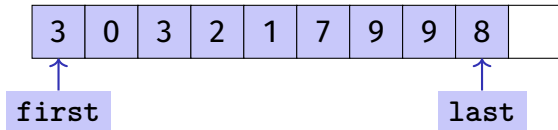


- Aber irgendwann sind alle Plätze belegt
- Dann nötig: Grösseren Speicherblock allozieren und Daten umkopieren

Arrays vergrössern/verkleinern



Elemente löschen erfordert verschieben (via kopieren) aller vorhergehenden oder nachfolgenden Elemente



Ähnlich: Einfügen an beliebiger Position

Effizienz: Arrays vs. Verkettete Listen

- Speicher: Unser `our_vector` belegt ungefähr n ints (Vektorgrösse n), unser `our_list` ungefähr $3n$ ints (ein Zeiger belegt i.d.R. 8 Byte)
- Laufzeit (mit `our_vector = std::vector`, `our_list = std::list`):

prepending (insert at front) [100,000x]:

- ▶ `our_vector`: 310 ms
- ▶ `our_list`: 8 ms

appending (insert at back) [100,000x]:

- ▶ `our_vector`: 1 ms
- ▶ `our_list`: 9 ms

removing first [100,000x]:

- ▶ `our_vector`: 329 ms
- ▶ `our_list`: 4 ms

removing last [100,000x]:

- ▶ `our_vector`: 0 ms
- ▶ `our_list`: 4 ms

removing randomly [10,000x]:

- ▶ `our_vector`: 1 ms
- ▶ `our_list`: 93 ms

inserting randomly [10,000x]:

- ▶ `our_vector`: 6 ms
- ▶ `our_list`: 334 ms

batch inserting & removing randomly (600 elements) [6,000x]:

- ▶ `our_vector`: 803 ms
- ▶ `our_list`: 282 ms

fully iterate sequentially (5000 elements) [5,000x]:

- ▶ `our_vector`: 279 ms
- ▶ `our_list`: 410 ms

24. Subtyping, Polymorphie und Vererbung

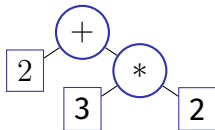
Die nachfolgenden Folien dieses Kapitels sind **nicht klausurrelevant**
Ausdrucksbäume, Aufgabenteilung und Modularisierung, Typhierarchien,
virtuelle Funktionen, dynamische Bindung, Code-Wiederverwendung,
Konzepte der objektorientierten Programmierung

Kurzvorstellung: Ausdrucksbäume

- Ziel: Arithmetische Ausdrücke repräsentieren, z.B.

$$2 + 3 * 2$$

- Arithmetische Ausdrücke bilden eine *Baumstruktur*

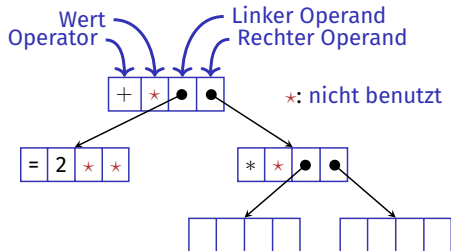


- Ausdrucksbäume bestehen aus *unterschiedlichen* Knoten: Literale (z.B. 2), binäre Operatoren (z.B. +), unäre Operatoren (z.B. $\sqrt{\quad}$), Funktionsanwendungen (z.B. $\cos$), etc.

Ein einziger Knotentyp

Ausdrucksbäume implementiert mittels *eines einzigen* Knotentyps:

```
struct tnode {  
    char op; // Operator ('=' for literals)  
    double val; // Literal's value  
    tnode* left; // Left child (or nullptr)  
    tnode* right; // ...  
    ...  
};
```



Beobachtung: `tnode` ist die „Summe“ aller benötigten Knoten (Konstanten, Addition, ...) $\Rightarrow$ Speicherverschwendung, unelegant...

Nachteile

...und jede Funktion muss diese „Summe“ wieder „auseinander nehmen“, z.B.:

```
double eval(const tnode* n) {  
    if (n->op == '=') return n->val; // n is a constant  
    double l = 0;  
    if (n->left) l = eval(n->left); // n is not a unary operator  
    double r = eval(n->right);  
    switch(n->op) {  
        case '+': return l+r; // n is an addition node  
        case '*': return l*r; // ...  
        ...  
    }
```

⇒ Umständlich und somit fehleranfällig

Nachteile

```
struct tnode {  
    char op;  
    double val;  
    tnode* left;  
    tnode* right;  
    ...  
};
```

```
double eval(const tnode* n) {  
    if (n->op == '=') return n->val;  
    double l = 0;  
    if (n->left) l = eval(n->left);  
    double r = eval(n->right);  
    switch(n->op) {  
        case '+': return l+r;  
        case '*': return l*r;  
        ...  
    }
```

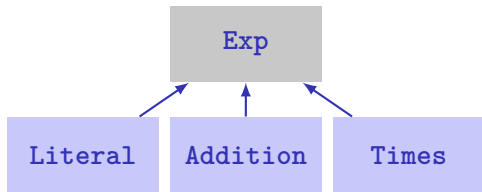
Dieser Code ist nicht *modular* – das ändern wir heute!

Neue Konzepte heute

1. Subtyping

- Typhierarchie: `Exp` repräsentiert allgemeine Ausdrücke, `Literal` etc. sind konkrete Ausdrücke
- Jedes `Literal` etc. ist auch ein `Exp` (Subtyp-Beziehung)
- Deswegen kann ein `Literal` etc. überall dort genutzt werden, wo ein `Exp` erwartet wird:

```
Exp* e = new Literal(132);
```



Neue Konzepte heute

2. Polymorphie und dynamische Bindung

- Eine Variable vom *statischen* Typ `Exp` kann Ausdrücke mit unterschiedlichen *dynamischen* Typen „beherbergen“:

```
Exp* e = new Literal(2); // e is the literal 2  
e = new Addition(e, e); // e is the addition 2 + 2
```

- Ausgeführt werden die Memberfunktionen des *dynamischen* Typs:

```
Exp* e = new Literal(2);  
std::cout << e->eval(); // 2  
  
e = new Addition(e, e);  
std::cout << e->eval(); // 4
```

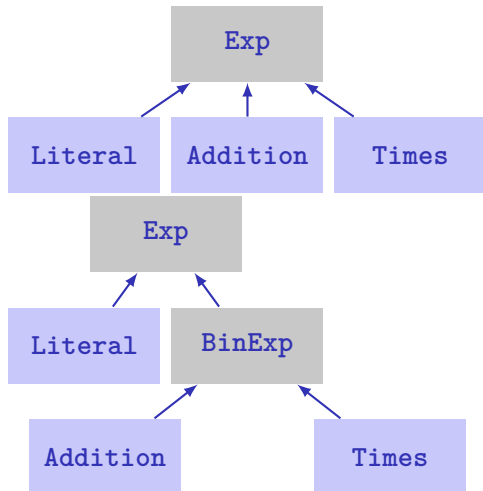
Neue Konzepte heute

3. Vererbung

- Manche Funktionalität ist für mehrere Mitglieder der Typhierarchie gleich
- Z.B. die Berechnung der Grösse (Verschachtelungstiefe) binärer Ausdrücke (Addition, Times):

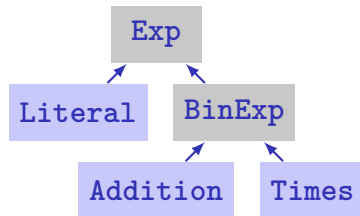
$$1 + \text{size}(\text{left operand}) + \text{size}(\text{right operand})$$

⇒ Funktionalität einmal implementieren und dann an Subtypen *vererben*



Vorteile

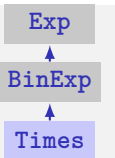
- Subtyping, Polymorphie und dynamische Bindung ermöglichen *Modularisierung durch Spezialisierung*
- Vererbung erlaubt gemeinsamen Code trotz Modularisierung
⇒ *Codeduplikation vermeiden*



```
Exp* e = new Literal(2);  
std::cout << e->eval();  
  
e = new Addition(e, e);  
std::cout << e->eval();
```


Syntax und Terminologie

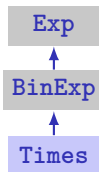
```
struct Exp {  
    ...  
}  
  
struct BinExp : public Exp {  
    ...  
}  
  
struct Times : public BinExp {  
    ...  
}
```



Anmerkung: Wir konzentrieren uns heute auf die neuen Konzepte (Subtyping, ...) und ignorieren den davon unabhängigen Aspekt der Kapselung (class, private vs. public Membervariablen)

Syntax und Terminologie

```
struct Exp {  
    ...  
}  
  
struct BinExp : public Exp {  
    ...  
}  
  
struct Times : public BinExp {  
    ...  
}
```



- BinExp ist eine von Exp *abgeleitete Klasse*¹
- Exp ist die *Basisklasse*² von BinExp
- BinExp *erbt* von Exp
- Die Vererbung von Exp zu BinExp ist *öffentlich* (public), daher ist BinExp ein *Subtyp* von Exp
- Analog: Times und BinExp
- Subtyprelation ist transitiv: Times ist ebenfalls ein Subtyp von Exp

¹Subklasse, Kindklasse

²Superklasse, Elternklasse

Abstrakte Klasse Exp und konkrete Klasse Literal

```
struct Exp {
```

← ...das macht Exp zu einer *abstrakten* Klasse

```
    virtual int size() const = 0;
```

```
    virtual double eval() const = 0;
```

```
};
```

↑ Aktiviert dynamische Bindung

Erzwingt Implementierung durch abgeleitete Klassen ...

```
struct Literal : public Exp {
```

← Literal erbt von Exp ...

```
    double val;
```

```
    Literal(double v);
```

```
    int size() const;
```

```
    double eval() const;
```

```
};
```

← ...ist aber ansonsten eine ganz normale Klasse

Literal: Implementierung

```
Literal::Literal(double v): val(v) {}
```

```
int Literal::size() const {  
    return 1;  
}
```

```
double Literal::eval() const {  
    return this->val;  
}
```

Subtyping: Ein Literal ist ein Ausdruck

Ein Zeiger auf einen Subtyp kann überall dort verwendet werden, wo ein Zeiger auf einen Supertyp gefordert ist:

```
Literal* lit = new Literal(5);  
Exp* e = lit; // OK: Literal is a subtype of Exp
```

Aber nicht umgekehrt:

```
Exp* e = ...  
Literal* lit = e; // ERROR: Exp is not a subtype of Literal
```

Polymorphie: Ein Literal verhält sich wie ein Literal

```
struct Exp {  
    ...  
    virtual double eval();  
};  
  
double Literal::eval() {  
    return this->val;  
}
```

```
Exp* e = new Literal(3);  
std::cout << e->eval(); // 3
```

- *Virtuelle* Memberfunktionen: der *dynamische* Typ (hier: `Literal`) bestimmt die auszuführenden Memberfunktionen
⇒ *dynamische Bindung*
- Ohne `virtual` bestimmt der *statische* Typ (hier: `Exp`) die auszuführende Funktion
- Wir vertiefen das nicht weiter

Weitere Ausdrücke: Addition und Times

```
struct Addition : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```

```
int Addition::size() const {  
    return 1 + left->size()  
           + right->size();  
}
```

```
struct Times : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```

```
int Times::size() const {  
    return 1 + left->size()  
           + right->size();  
}
```



Aufgabenteilung



Codeduplizierung

Gemeinsamkeiten auslagern ...: BinExp

```
struct BinExp : public Exp {  
    Exp* left;  
    Exp* right;  
  
    BinExp(Exp* l, Exp* r);  
    int size() const;  
};
```

```
BinExp::BinExp(Exp* l, Exp* r): left(l), right(r) {}
```

```
int BinExp::size() const {  
    return 1 + this->left->size() + this->right->size();  
}
```

Bemerkung: BinExp implementiert eval nicht und ist daher, genau wie Exp, eine abstrakte Klasse

...Gemeinsamkeiten erben: Addition

```
struct Addition : public BinExp {  
    Addition(Exp* l, Exp* r);  
    double eval() const;  
};
```

← Addition erbt Membervariablen (left, right) und Funktionen (size) von BinExp

```
Addition::Addition(Exp* l, Exp* r): BinExp(l, r) {}
```

```
double Addition::eval() const {  
    return  
        this->left->eval() +  
        this->right->eval();  
}
```

↑ Aufruf des Superkonstruktors (Konstruktor von BinExp) zwecks Initialisierung der Membervariablen left und right

...Gemeinsamkeiten erben: Times

```
struct Times : public BinExp {  
    Times(Exp* l, Exp* r);  
    double eval() const;  
};
```

```
Times::Times(Exp* l, Exp* r): BinExp(l, r) {}
```

```
double Times::eval() const {  
    return  
        this->left->eval() *  
        this->right->eval();  
}
```

Beobachtung: `Additon::eval()` und `Times::eval()` sind sich sehr ähnlich und könnten ebenfalls zusammengelegt werden. Das dafür notwendige Konzept der *funktionalen Programmierung* geht jedoch über diesen Kurs hinaus.

Weitere Ausdrücke und Operationen

- Weitere Ausdrücke, als von `Exp` abgeleitete Klassen, sind möglich, z.B. $-$, $/$, $\sqrt{}$, $\cos$, $\log$
- Eine ehemalige Bonusaufgabe (Teil der heutigen Vorlesungsbeispiele auf Code Expert) veranschaulicht, was alles möglich ist: Variablen, trigonometrische Funktionen, Parsing, Pretty-Printing, numerische Vereinfachungen, symbolische Ableitungen, ...

Mission: Monolithisch → modular ✓

```
struct tnode {  
    char op;  
    double val;  
    tnode* left;  
    tnode* right;  
    ...  
}
```

```
double eval(const tnode* n) {  
    if (n->op == '=') return n->val;  
    double l = 0;  
    if (n->left != 0) l = eval(n->left);  
    double r = eval(n->right);  
    switch(n->op) {  
        case '+': return l + r;  
        case '*': return l * r;  
        case '-': return l - r;  
        case '/': return l / r;  
        default:  
            // unknown operator  
            assert (false);  
    }  
}
```

```
int size (const tnode* n) const { ... }  
...
```

```
struct Literal : public Exp {  
    double val;  
    ...  
    double eval() const {  
        return val;  
    }  
};
```

```
struct Addition : public Exp {  
    ...  
    double eval() const {  
        return left->eval() + right->eval();  
    }  
};
```

```
struct Times : public Exp {  
    ...  
    double eval() const {  
        return left->eval() * right->eval();  
    }  
};
```

+

```
struct Cos : public Exp {  
    ...  
    double eval() const {  
        return std::cos(argument->eval());  
    }  
};
```

Es gibt noch so viel mehr ...

Nicht gezeigt/besprochen:

- Private Vererbung (`class B : public A`)
- Subtyping und Polymorphie ohne Zeiger
- Nicht-virtuelle Memberfunktionen und statische Bindung
(~~`virtual`~~ `double eval()`)
- Überschreiben geerbter Memberfunktionen und Aufrufen der überschriebenen Implementierung
- Mehrfachvererbung (multiple inheritance)
- ...

Gute Neuigkeit: Das heutige Kapitel (24. Subtyping, Polymorphie und Vererbung) ist *nicht* prüfungsrelevant

Objektorientierte Programmierung

Im letzten Kursdrittel wurden einige Konzepte der *objektorientierten Programmierung* vorgestellt, die auf den kommenden Folien noch einmal kurz zusammengefasst werden.

Kapselung (Wochen 10-13):

- Verbergen der Implementierungsdetails von Typen (privater Bereich) vor Benutzern
- Definition einer Schnittstelle (öffentlicher Bereich) zum kontrollierten Zugriff auf Werte und Funktionalität
- Ermöglicht das Sicherstellen von Invarianten, sowie den Austausch von Implementierungen ohne Anpassungen von Benutzercode

Objektorientierte Programmierung

Subtyping (Woche 14):

- Typhierarchien mit Super- und Subtypen können angelegt werden um Verwandtschaftbeziehungen sowie Abstraktionen und Spezialisierungen zu modellieren
- Ein Subtyp unterstützen mindestens die Funktionalität, die auch der Supertyp unterstützt – i.d.R. aber mehr, d.h. Subtypen erweitern die Schnittstelle (den öffentlichen Bereich) ihrer Supertypen
- Daher können Subtypen überall dort eingesetzt werden, wo Supertypen verlangt sind ...
- ...und Funktionen, die auf abstrakteren Typen (Supertypen) operieren können, können auch auf spezialisierteren Typen (Subtypen) operieren
- Die in Woche 7 vorgestellten Streams bilden eine solche Typhierarchie: `ostream` ist der abstrakte Supertyp, `ofstream` etc. sind spezialisierte Subtypen

Objektorientierte Programmierung

Polymorphie und *dynamische Bindung* (Woche 14):

- Ein Zeiger vom statischen Typ T_1 kann zur Laufzeit auf Objekte vom (dynamischen) Typ T_2 zeigen, falls T_2 ein Subtyp von T_1 ist
- Wird eine virtuelle Memberfunktion von einem solchen Zeiger aus aufgerufen, so entscheidet der dynamische Typ darüber, welche Funktion ausgeführt wird
- D.h.: Trotz gleichem statischen Typ kann beim Zugriff auf eine gemeinsame Schnittstelle (Memberfunktionen) eines solchen Zeigers ein anderes Verhalten auftreten
- Zusammen mit Subtyping ermöglicht es dies, neue konkrete Typen (Streams, Ausdrücke, ...) zu einem bestehenden System hinzuzufügen, ohne dieses abändern zu müssen

Objektorientierte Programmierung

Vererbung (Woche 14):

- Abgeleitete Klassen erben die Funktionalität, d.h. die Implementierungen von Memberfunktionen, ihrer Elternklassen
- Dies ermöglicht es, gemeinsam genutzten Code wiederverwenden zu können und vermeidet so Codeduplikation
- Geerbte Implementierungen können auch überschrieben werden, um zu erreichen, dass eine abgeleitete Klasse sich anders verhält als ihre Elternklasse (im Kurs nicht gezeigt)

25. Zusammenfassung

Zweck und Format

Nennung der wichtigsten Stichwörter zu den Kapiteln. Checkliste: „kann ich mit jedem Begriff etwas anfangen?“

- Ⓜ Motivation: Motivierendes Beispiel zum Kapitel
- Ⓚ Konzepte: Konzepte, die nicht von der Implementation (Sprache) C++abhängen
- Ⓢ Sprachlich (C++): alles was mit der gewählten Sprache zusammenhängt
- Ⓑ Beispiele: genannte Beispiele der Vorlesung

Kapitelüberblick

- 1. Einführung
- 2. Erstes C++ Programm
- 3. Ganze Zahlen
- 4. Wahrheitswerte
- 5./6. Kontrollanweisungen
- 7./8. Fließkommazahlen
- 9./10. Funktionen
- 11. Vektoren
- 12. Referenztypen
- 13. Texte und Strings
- 14. Mehrdimensionale Vektoren
- 15./16. Rekursion
- 17./18. Custom Data Types
- 19. Container, Iteratoren und Algorithmen
- 20. Dynamischer Speicher und Zeiger
- 21.-23. Dynamische Datenstrukturen und Speicherverwaltung
- 24. Subtyping, Polymorphie und Vererbung

1. Einführung

Ⓜ

- Euklidischer Algorithmus

Ⓚ

- Algorithmus, Turingmaschine, Programmiersprachen, Kompilation, Syntax und Semantik
- Werte und Effekte, (Fundamental)typen, Literale, Variablen, Bezeichner, Objekte, Ausdrücke, Operatoren, Anweisungen

Ⓢ

- Include-Direktiven `#include <iostream>`
- Hauptfunktion `int main(){...}`
- Kommentare, Layout `// Kommentar`
- Typen, Variablen, L-Wert `a` , R-Wert `a+b`
- Ausdrucksanweisung `b=b*b;` , Deklarationsanweisung `int a;`, Rückgabeeanweisung `return 0;`

2. Erstes C++ Programm

(M)

- Einfache Berechnung von a^8

(K)

- Programmiersprachen-Elemente

(S)

- Kommentare und Layout
- Include-Direktive
- main-Funktion
- Werte, Effekte
- Typen, Funktionalität
- Literale, Variablen, Konstanten
- Bezeichner, Namen
- Ausdrücke
- L- und R-Werte
- Operatoren
- Anweisungen

(B)

- Erstes Programm

3. Ganze Zahlen

- (M) Celsius to Fahrenheit
- (K) Assoziativität und Präzedenz, Stelligkeit
- Ausdrucksbäume, Auswertungsreihenfolge
- Arithmetische Operatoren
- Binärzahldarstellung, Hexadezimale Zahlen, Wertebereich
- Zahlendarstellung mit Vorzeichen, Zweierkomplement
- (S) Arithmetische Operatoren `9 * celsius / 5 + 32`
- Inkrement / Dekrement `expr++`
- Arithmetische Zuweisungen `expr1 += expr2`
- Konversion `int` $\leftrightarrow$ `unsigned int`
- (B) Celsius to Fahrenheit, Ersatzwiderstand

4. Wahrheitswerte

(K)

- Boole'sche Funktionen, Vollständigkeit
- DeMorgan'sche Regeln

(S)

- Der Typ `bool`
- Logische Operationen `a && !b`
- Relationale Operationen `x < y`
- Präzedenzen `7 + x < y && y != 3 * z`
- Kurzschlussauswertung `x != 0 && z / x > y`
- Die `assert`-Anweisung, `#include <cassert>`

(B)

- Div-Mod Identität.

5./6. Kontrollanweisungen

(M)

- Linearer Kontrollfluss vs. interessante Programme, Spaghetti-Code

(K)

- Auswahlanweisungen, Iterationsanweisungen
- (Vermeidung von) Endlosschleifen, Halteproblem
- Sichtbarkeits- und Gültigkeitsbereich, Automatische Speicherdauer
- Äquivalenz von Iterationsanweisungen

(S)

- if Anweisungen `if (a % 2 == 0) {..}`
- for Anweisungen `for (unsigned int i = 1; i <= n; ++i) ...`
- while und do-Anweisungen `while (n > 1) {...}`
- Blöcke, Sprunganweisungen `if (a < 0) continue;`
- Switch Anweisung `switch(grade) {case 6: }`

(B)

- Summenberechnung (Gauss), Primzahltest, Collatz-Folge, Fibonacci Zahlen, Taschenrechner, Notenausgabe

7./8. Fließkommazahlen

Ⓜ

- Richtig Rechnen: Celsius / Fahrenheit

Ⓚ

- Fixkomma- vs. Fließkommazahldarstellung
- (Löcher im) Wertebereich
- Rechnen mit Fließkommazahlen, Umrechnung
- Fließkommazahlensysteme, Normalisierung, IEEE Standard 754
- *Richtlinien für das Rechnen mit Fließkommazahlen*

Ⓢ

- Typen `float`, `double`
- Fließkommaliterale `1.23e-7f`

Ⓑ

- Celsius/Fahrenheit, Euler, Harmonische Zahlen

9./10. Funktionen

(M)

- Potenzberechnung

(K)

- Kapselung von Funktionalität
- Funktionen, formale Argumente, Aufrufargumente
- Gültigkeitsbereich, Vorwärts-Deklaration
- Prozedurales Programmieren, Modularisierung, Getrennte Übersetzung
- *Stepwise Refinement*

(S)

- Funktionsdeklaration, -definition `double pow(double b, int e){ ... }`
- Funktionsaufruf `pow (2.0, -2)`
- Der typ `void`

(B)

- Potenzberechnung, perfekte Zahlen, Minimum, Kalender

11. Vektoren

(M)

- Iteration über Daten: Sieb des Eratosthenes

(K)

- Vektoren, Speicherlayout, Wahlfreier Zugriff
- (Fehlende) Grenzenprüfung
- Vektoren

(S)

- Vektor Typen `std::vector<int> a {4,3,5,2,1};`

(B)

- Sieb des Eratosthenes

12. Referenztypen

(M)

- Funktion Swap

(K)

- Werte-/ Referenzsemantik, Pass by Value / Pass by Reference, Return by Reference
- Lebensdauer von Objekten / Temporäre Objekte
- Konstanten

(S)

- Referenztyp `int& a`
- Call by Reference und Return by Reference `int& increment (int& i)`
- Const-Richtlinie, Const-Referenzen, Referenzrichtlinie

(B)

- Swap, Inkrement

13 Zeichen und Strings

- ④ ■ Caesar Code
- ④ ■ Zeichen: ASCII, UTF8, Texte, Strings
- ④ ■ Zeichen und Texte, der Typ char `char c = 'a';`, Konversion nach `int`
- ④ ■ Caesar-Code

14. Vektoren

- Matrizen, Tabellen
- Mehrdimensionale Vektoren
- Vektoren von Vektoren
- auto
- Summe des unteren Dreiecks

15./16. Rekursion

- Ⓜ ■ Rekursive math. Funktionen, Das n-Queen Problem, Kommandozeilenrechner
- Ⓚ ■ Rekursion
 - Aufrufstapel, Gedächtnis der Rekursion
 - Korrektheit, Terminierung,
 - Rekursion vs. Iteration
 - Decrease and Conquer vs. Divide and Conquer
- Ⓑ ■ Fakultät, GGT, Sudoku-Löser, Taschenrechner, Vektorsumme

17./18. Custom Data Types

(M)

- Datentyp Rationale Zahlen selber bauen

(K)

- Heterogene Datenstruktur
- Funktions- und Operator-Overloading
- Datenkapselung, Konstruktion, Mitgliedsfunktionen

(S)

- Struct Definition `struct rational {int n; int d;};`
 - Mitgliedszugriff `result.n = a.n * b.d + a.d * b.n;`
 - Initialisierung und Zuweisung,
 - Überladen von Funktionen `pow(2)` vs. `pow(3,3);`, Überladen von Operatoren
 - Klassen `class rational { ... };`
 - Zugriffssteuerung `public: / private:`
 - Mitgliedsfunktionen `int rational::denominator () const`
 - Das implizite Argument der Memberfunktionen
- (B)
- rationale Zahlen, komplexe Zahlen

19. Container, Iteratoren und Algorithmen

(M)

- Vektoren sind Container

(K)

- Iterieren mit Zeigern
- Container und Iteratoren
- Algorithmen

(S)

- Iteratoren `std::vector<int>::iterator`
- Algorithmen der Standardbibliothek `std::fill (a, a+5, 1);`
- Einen Iterator implementieren
- Iteratoren und `const`

(B)

- Ausgeben eines Vektors, einer Menge

20. Dynamischer Speicher und Zeiger

- Unsere eigene Datenstruktur
- Allokation, Zeiger-Typen, Verkettete Liste, Allokation, Deallokation, Dynamischer Datentyp
- Die `new` Anweisung
- Zeiger `int* x;`, Nullzeiger `nullptr.`
- Adress-, Dereferenzoperator `int *ip = &i; int j = *ip;`
- Zeiger und Const `const int *a;`
- Verkettete Liste, Stack

21.-23. Dynamische Datenstrukturen und Speicherverwaltung

(M)

- Stack
- Verkettete Liste
- Vektor

(K)

- Richtlinie „Dynamischer Speicher“
- Gemeinsamer Zeiger-Zugriff
- Dynamischer Datentyp

(S)

- `new` und `delete`
- Desktruktor `stack::~~stack()`
- Kopierkonstruktor `stack::stack(const stack& s)`
- Zuweisungsoperator `stack& stack::operator=(const stack& s)`
- Dreierregel

(B)

- Stack, Verkettete Liste, Vektor

24. Subtyping, Polymorphie und Vererbung

(M)

- Erweitern und Verallgemeinern von Ausdrucksbäumen

(K)

- Subtyping
- Polymorphie und dynamische Bindung
- Vererbung

(S)

- Basisklasse `struct Exp{}`
- Abgeleitete Klasse `struct BinExp: public Exp{}`
- Abstrakte Klasse `struct Exp{virtual int size() const = 0...}`
- Polymorphie `virtual double eval()`

(B)

- Ausdrucksknoten und Erweiterungen

Ende

Ende der Vorlesung.